

GAUSE: A Mathematical Deep Dive

Competition-Driven Specialization and Reward-Independent Capacity Assignment for Learner Populations

Yuhao Li
University of Pennsylvania
li88@sas.upenn.edu

June 10, 2026

Abstract

This document provides an exhaustive, ground-up explanation of the GAUSE (Generalist-averse Affinity Update for Specialist Emergence)¹ algorithm and the theory of emergent specialization in multi-agent systems. We combine rigorous mathematical treatment with intuitive explanations, worked examples, code implementations, and visualizations. The document is designed for readers with a strong mathematical background who want to understand every detail of how and why agents spontaneously specialize under competitive pressure. The companion paper’s central finding reframes catastrophic forgetting as a *signal-availability* problem: under bounded capacity the quantity most needing protection—a dormant regime’s knowledge—is exactly the one emitting no reward, so any reward-driven allocator is structurally blind to it. Made precise as a scoped principle, retention of dormant regimes (within the class of mechanisms whose only protection is the assignment itself) requires the assignment to be *both* independent of current task reward *and* persistently covering: reward-chasing allocations (including a learned MoE router) forget; reward-independent *and* covering ones (intrinsic-reward diversity, converged competitive exclusion) retain, while merely reward-independent ones (fixed random niches) retain only partially. This is developed in the controls-and-ablations and MARL-comparison sections below.

Prerequisites: Basic probability theory, linear algebra, and familiarity with optimization. All other concepts (information theory, game theory, Bayesian inference, bounded-capacity memory models, mixture-of-experts gating, and continual learning) are developed from first principles.

Contents

I	Mathematical Foundations	8
1	Information Theory Foundations	8
1.1	Shannon Entropy: Measuring Uncertainty	8
1.1.1	Units and Logarithm Base	8
1.1.2	Properties of Entropy	8
1.1.3	Worked Examples	9
1.1.4	Graphical Visualization of Entropy	10
1.2	Normalized Entropy and the Specialization Index	10

¹The name also honors G. F. Gause, whose competitive exclusion principle (1934) the mechanism operationalizes.

1.2.1	Detailed SI Calculations	10
1.3	Kullback-Leibler Divergence	11
2	Game Theory Foundations	12
2.1	Strategic Games and Nash Equilibrium	12
2.2	Winner-Take-All Games	12
2.3	The Pigeonhole Principle	12
2.4	Competitive Exclusion in Ecology	13
3	Bayesian Inference Foundations	14
3.1	Bayes' Theorem	14
3.2	The Beta Distribution	14
3.2.1	Properties of the Beta Distribution	14
3.3	Beta-Bernoulli Conjugacy	15
4	Multi-Armed Bandits and Thompson Sampling	17
4.1	The Multi-Armed Bandit Problem	17
4.2	Contextual Bandits (Our Setting)	17
4.3	Thompson Sampling Algorithm	17
4.3.1	Why Thompson Sampling Works	18
4.3.2	Thompson Sampling in Contextual Setting	18
4.4	Belief Evolution Over Time	19
4.5	Why Different Agents Learn Different Beliefs	20
4.6	Connection to Niche Affinity	20
II	The Environment: Regimes and Structure	21
5	Regime-Switching Environments	21
5.1	Formal Definition	21
5.2	Why Regimes Enable Specialization	21
5.3	The Six Domains	22
6	Bounded Capacity and Memory Models	22
6.1	The Capacity Budget	22
6.2	Hard Model: LRU Slots with Eviction	23
6.3	Soft Model: Interference Decay	24
6.4	Why a Specialist Is Structurally Immune	25
7	Mixture-of-Experts: Gating, Routing, and the Gate Objective	25
7.1	Architecture: Experts and a Gate	25
7.2	The Gate Objective Is Reward-Driven	26
7.3	Load Balancing and Its Absence Here	27
8	Continual Learning and Catastrophic Forgetting	27
8.1	The Continual Learning Problem	27
8.2	The Stability–Plasticity Dilemma	28
8.3	A Taxonomy of CL Methods, and Where GAUSE Sits	28

9	Domain-Specific Regime Definitions	29
9.1	Crypto Domain	29
9.2	Weather Domain	30
9.3	Traffic Domain	30
9.4	Commodities Domain	30
9.5	Solar Domain	31
9.6	Air Quality Domain	31
III	Methods and Regime-Method Affinity	32
10	The Method Inventory	32
10.1	Formal Definition	32
10.2	Mathematical Description of Common Methods	32
10.2.1	Naive (Persistence)	32
10.2.2	Moving Average (MA)	32
10.2.3	Momentum	33
10.2.4	Mean Reversion	33
10.2.5	Trend Following (MA Crossover)	33
10.3	Domain-Specific Methods	33
10.3.1	Traffic Domain Methods	33
10.3.2	Solar Domain Methods	34
11	Regime-Method Affinity Matrices	35
11.1	Definition and Interpretation	35
11.2	Complete Affinity Matrices (Verified from Code)	35
11.2.1	Crypto Domain	35
11.2.2	Commodities Domain	35
11.2.3	Weather Domain	36
11.2.4	Solar Domain	36
11.2.5	Traffic Domain	36
11.2.6	Air Quality Domain	36
11.3	Reward Generation from Affinity	36
IV	Specialization Metrics	38
12	Niche Affinity Distribution	38
12.1	Definition	38
12.2	Evolution Over Time	38
12.3	Why the V3 Heuristic Fails and Why EG Is Canonical	39
13	Specialization Index (SI) — Detailed Treatment	43
13.1	The Complete Formula	43
13.2	Properties	43
13.3	SI Calculation Examples (Step by Step)	43
13.4	SI vs. Number of Regimes	44
14	Method Specialization Index (MSI)	44
14.1	MSI Calculation Examples	45

15 Method Coverage	46
15.1 Coverage Across All Domains	46
15.2 Why Coverage Matters	46
16 Real Data Validation	47
16.1 Data Sources and Quality	47
16.2 Real vs. Synthetic Data Considerations	47
16.3 SI Performance on Real Data	48
17 Are the Niches Meaningful? Controls and Ablations	48
17.1 Negative control: SI is invariant to regime shuffling	48
17.2 Oracle diagnostic: how much regime structure exists?	48
17.3 Performance-based control on structured domains	49
17.4 Ablations: conditioning vs. specialization	49
V The Competition Mechanism	52
18 Agent Competition: The Core Loop	52
18.1 Setup	52
18.2 One Iteration	52
19 Score Calculation	52
19.1 Raw Score	52
19.2 Niche Bonus	52
19.3 Example Calculation	53
20 Winner-Take-All Dynamics	53
20.1 Only the Winner Updates	53
20.2 Why This Matters	54
21 Affinity Update Rule	54
21.1 The Update Equations	54
21.2 Interpretation	54
21.3 Numerical Example	54
22 The Positive Feedback Loop	55
23 Why Competition Alone Induces Specialization ($\lambda = 0$)	55
23.1 The Central Claim	55
23.2 The Mechanism Without Niche Bonus	55
23.2.1 Step 1: Random Initial Divergence	55
23.2.2 Step 2: Winner-Take-All Creates Path Dependence	56
23.2.3 Step 3: Method Learning Reinforces Divergence	56
23.2.4 Step 4: Crowding Pressure Sustains Specialization	56
23.3 Mathematical Analysis of $\lambda = 0$ Dynamics	57
23.4 Experimental Evidence	57
23.5 The Role of λ : Accelerator, Not Cause	57
24 Convergence Analysis	58
24.1 How Fast Do Agents Specialize?	58
24.1.1 SI Trajectory Over Time	58
24.1.2 Convergence Rate	58

24.1.3	Empirical Convergence Times	59
24.2	Stability of Final Specialization	59
24.3	What If Two Agents Specialize in the Same Regime?	59
25	Comparison with Multi-Agent Reinforcement Learning	60
25.1	Why Traditional MARL Fails to Produce Diversity	60
25.1.1	Independent Q-Learning (IQL)	60
25.1.2	Value Decomposition (VDN, QMIX)	60
25.1.3	Multi-Agent PPO (MAPPO)	61
25.2	Structural Difference: Winner-Take-All	61
25.3	Empirical MARL Comparison Results	61
25.4	Why Performance Matters Despite Low SI in MARL	61
26	Hyperparameter Sensitivity Analysis	63
26.1	Key Hyperparameters	63
26.2	Effect of λ (Niche Bonus Coefficient)	63
26.3	Effect of N (Population Size)	64
26.4	Effect of η (Learning Rate)	64
26.5	Effect of T (Number of Iterations)	64
26.6	Hyperparameter Robustness	65
VI	Theoretical Analysis	66
27	Proposition 1: Competitive Exclusion	66
27.1	Setup and Definitions	66
27.2	Proof	66
28	Proposition 2: SI Lower Bound	67
28.1	Proof Sketch	67
28.2	Numerical Verification	68
29	Proposition 3: Mono-Regime Collapse	68
29.1	Proof	68
29.2	Empirical Validation	68
30	Proposition 4: Reward-Independence and Retention	68
30.1	Setup and Definitions	69
30.2	The Principle	69
30.3	Proof	69
31	Connection to No Free Lunch Theorem	70
VII	The Complete Algorithm	71
32	GAUSE: Pseudocode	71
33	Python Implementation	71
33.1	Core Simulation Function	71

34 Complete Worked Example	73
34.1 Initial State	73
34.2 Iteration 1: Regime = Bull	73
34.3 Iteration 2: Regime = Sideways	73
34.4 After 50 Iterations	74
35 Expected Outcomes	74
35.1 Experimental Results Summary	74
VIII Comprehensive Summary	75
36 The Complete Picture	75
36.1 The Central Claim in One Sentence	75
36.2 Why This Matters	75
37 Algorithm Summary	75
37.1 What the Algorithm Does	75
37.2 Why It Works	76
38 Key Metrics Summary	76
39 Key Results Summary	76
39.1 What We Proved	76
39.2 What We Demonstrated Empirically	76
40 Connections Between Concepts	77
41 Common Questions Answered	77
41.1 Q1: Why not just assign agents to niches?	77
41.2 Q2: Won't agents just converge to the same strategy?	77
41.3 Q3: What if there are more agents than regimes?	77
41.4 Q4: Does this work with real data?	78
41.5 Q5: How does this compare to MARL?	78
41.6 Q5b: What happens if a regime returns <i>changed</i> (concept drift)?	78
41.7 Q6: Does the specialization actually improve task performance?	78
42 How to Use This Algorithm	79
42.1 For Practitioners	79
42.2 For Researchers	79
Appendices	80
A Notation Reference	80
B Proof of Beta-Bernoulli Conjugacy	80
C Additional Affinity Matrix Details	80

D	Algorithm Workflow: Complete Visual Guide	81
D.1	What Is Specialization?	81
D.2	Why Do We Want Specialization?	81
D.3	The Key Mechanism: Competition Creates Diversity	82
D.4	The Ground Truth: Regime-Method Affinity Matrix	82
D.5	Phase 0: Initialization (The Blank Slate)	83
D.6	The Algorithm: Master Flowchart	83
D.7	One Complete Iteration: Step-by-Step Trace	83
D.8	Multi-Iteration Convergence: How Agents Diverge	86
D.9	The Core Insight: Why $\lambda = 0$ Still Works	86
D.10	Competition Dynamics: Crowding and Migration	87
D.11	Summary: The Complete Picture	87

Part I

Mathematical Foundations

Before diving into the algorithm, we establish the mathematical toolkit required to understand emergent specialization. This includes information theory (for measuring specialization), game theory (for understanding competitive dynamics), and Bayesian inference (for learning under uncertainty).

1 Information Theory Foundations

1.1 Shannon Entropy: Measuring Uncertainty

Information theory, developed by Claude Shannon in 1948, provides the mathematical framework for quantifying uncertainty and information content in probability distributions.

Definition 1.1 (Shannon Entropy). For a discrete random variable X with probability mass function $p(x)$ over a finite alphabet $\mathcal{X}$, the **Shannon entropy** is defined as:

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log p(x) \quad (1)$$

where we adopt the convention that $0 \log 0 = 0$ (justified by the limit $\lim_{p \rightarrow 0^+} p \log p = 0$).

Intuition

Entropy measures the “average surprise” when sampling from a distribution:

- If outcomes are predictable (one outcome dominates), entropy is **low**
- If outcomes are unpredictable (uniform distribution), entropy is **high**

1.1.1 Units and Logarithm Base

The choice of logarithm base determines the unit of entropy:

- $\log_2$: entropy measured in **bits**
- $\ln$ (natural log): entropy measured in **nats**
- $\log_{10}$: entropy measured in **dits** (decimal digits)

Throughout this document, we use natural logarithm ($\ln$), so entropy is measured in nats. The conversion is: $H_{\text{bits}} = H_{\text{nats}} / \ln(2) \approx 1.443 \cdot H_{\text{nats}}$.

1.1.2 Properties of Entropy

Proposition 1.1 (Properties of Shannon Entropy). *Let X be a discrete random variable over $\mathcal{X}$ with $|\mathcal{X}| = n$. Then:*

1. **Non-negativity:** $H(X) \geq 0$
2. **Zero entropy:** $H(X) = 0$ if and only if X is deterministic (one outcome has probability 1)
3. **Maximum entropy:** $H(X) \leq \log n$, with equality if and only if X is uniformly distributed

Proof. **(1) Non-negativity:** Since $p(x) \in [0, 1]$, we have $\log p(x) \leq 0$. Thus $-p(x) \log p(x) \geq 0$ for all x , and the sum is non-negative.

(2) Zero entropy: If $H(X) = 0$, then each term $-p(x) \log p(x) = 0$. This requires either $p(x) = 0$ or $p(x) = 1$ for each x . Since probabilities sum to 1, exactly one outcome has probability 1.

(3) Maximum entropy: We use Lagrange multipliers. Maximize $H = -\sum_x p(x) \log p(x)$ subject to $\sum_x p(x) = 1$.

The Lagrangian is:

$$\mathcal{L} = -\sum_x p(x) \log p(x) - \lambda \left(\sum_x p(x) - 1 \right)$$

Taking the derivative with respect to $p(x)$:

$$\frac{\partial \mathcal{L}}{\partial p(x)} = -\log p(x) - 1 - \lambda = 0$$

This gives $p(x) = e^{-1-\lambda}$ for all x , which is constant. Combined with the constraint $\sum_x p(x) = 1$, we get $p(x) = 1/n$ for all x .

The maximum entropy is:

$$H_{\max} = -\sum_{x=1}^n \frac{1}{n} \log \frac{1}{n} = -n \cdot \frac{1}{n} \cdot \log \frac{1}{n} = \log n$$

□

1.1.3 Worked Examples

Example 1.1 (Fair Coin). For a fair coin with $p(\text{heads}) = p(\text{tails}) = 0.5$:

$$H = -0.5 \log(0.5) - 0.5 \log(0.5) = -\log(0.5) = \log(2) \approx 0.693 \text{ nats}$$

In bits: $H = 1$ bit (maximum uncertainty for binary outcome).

Example 1.2 (Biased Coin). For a biased coin with $p(\text{heads}) = 0.9$, $p(\text{tails}) = 0.1$:

$$H = -0.9 \log(0.9) - 0.1 \log(0.1) = 0.0948 + 0.2303 = 0.325 \text{ nats}$$

This is less than $\log(2) = 0.693$, reflecting reduced uncertainty.

Example 1.3 (Uniform over 4 Outcomes). For uniform distribution over 4 outcomes: $p(x) = 0.25$ for each.

$$H = -4 \times 0.25 \times \log(0.25) = -\log(0.25) = \log(4) \approx 1.386 \text{ nats}$$

This is maximum entropy for 4 outcomes.

Example 1.4 (Concentrated Distribution). For $p = (0.7, 0.1, 0.1, 0.1)$:

$$H = -0.7 \log(0.7) - 3 \times 0.1 \log(0.1) = 0.250 + 0.691 = 0.941 \text{ nats}$$

Less than maximum ($\log 4 = 1.386$), reflecting concentration on first outcome.

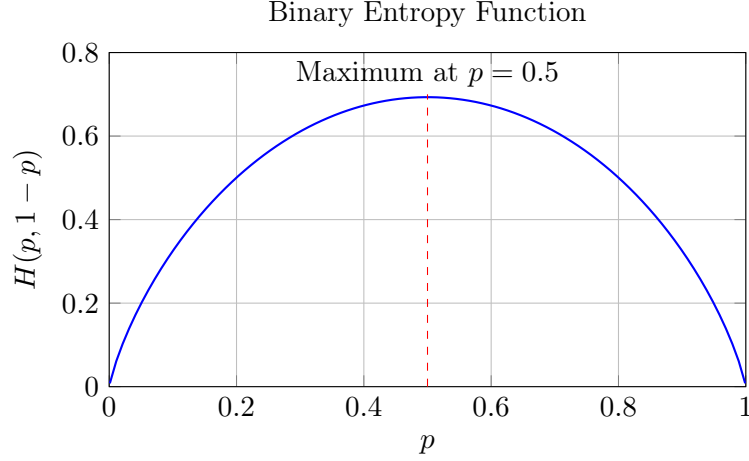


Figure 1: Binary entropy function $H(p) = -p \log p - (1-p) \log(1-p)$. Maximum at $p = 0.5$ (uniform), minimum at $p = 0$ or $p = 1$ (deterministic).

1.1.4 Graphical Visualization of Entropy

1.2 Normalized Entropy and the Specialization Index

For comparing entropy across distributions with different support sizes, we normalize by the maximum possible entropy.

Definition 1.2 (Normalized Entropy). For a distribution over R outcomes, the **normalized entropy** is:

$$\bar{H} = \frac{H}{\log R} \in [0, 1] \quad (2)$$

This leads naturally to our key metric:

Definition 1.3 (Specialization Index). For an agent with niche affinity distribution $\alpha = (\alpha_1, \dots, \alpha_R)$ over R regimes, the **Specialization Index** is:

$$\text{SI}(\alpha) = 1 - \frac{H(\alpha)}{\log R} = 1 - \bar{H}(\alpha) \quad (3)$$

Key Point

- $\text{SI} = 0$: Agent is a **generalist** (uniform distribution over regimes)
- $\text{SI} = 1$: Agent is a **perfect specialist** (all probability on one regime)
- $\text{SI} \in (0, 1)$: Agent shows **partial specialization**

1.2.1 Detailed SI Calculations

Example 1.5 (Generalist Agent). An agent with uniform affinity $\alpha = (0.25, 0.25, 0.25, 0.25)$:

$$H(\alpha) = -4 \times 0.25 \times \log(0.25) \quad (4)$$

$$= -\log(0.25) \quad (5)$$

$$= \log(4) \quad (6)$$

$$\approx 1.386 \text{ nats} \quad (7)$$

$$\text{SI}(\alpha) = 1 - \frac{1.386}{\log(4)} = 1 - \frac{1.386}{1.386} = 0 \quad (8)$$

Interpretation: This agent has no preference—it’s equally likely to succeed in any regime.

Example 1.6 (Perfect Specialist). An agent with concentrated affinity $\alpha = (1, 0, 0, 0)$:

$$H(\alpha) = -1 \times \log(1) - 0 - 0 - 0 = 0 \quad (9)$$

$$\text{SI}(\alpha) = 1 - \frac{0}{\log(4)} = 1 \quad (10)$$

Interpretation: This agent is perfectly specialized in regime 1.

Example 1.7 (Partial Specialist (Realistic)). An agent with affinity $\alpha = (0.6, 0.2, 0.1, 0.1)$:

$$H(\alpha) = -0.6 \log(0.6) - 0.2 \log(0.2) - 0.1 \log(0.1) - 0.1 \log(0.1) \quad (11)$$

$$= -0.6 \times (-0.511) - 0.2 \times (-1.609) - 0.1 \times (-2.303) - 0.1 \times (-2.303) \quad (12)$$

$$= 0.306 + 0.322 + 0.230 + 0.230 \quad (13)$$

$$= 1.088 \text{ nats} \quad (14)$$

$$\text{SI}(\alpha) = 1 - \frac{1.088}{1.386} = 1 - 0.785 = 0.215 \quad (15)$$

Interpretation: This agent shows moderate specialization toward regime 1.

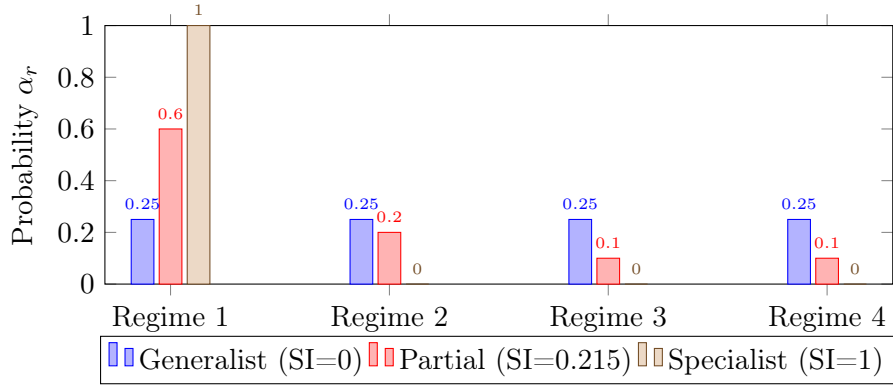


Figure 2: Visualization of three affinity distributions with different SI values.

1.3 Kullback-Leibler Divergence

For comparing two distributions, we use KL divergence.

Definition 1.4 (KL Divergence). The **Kullback-Leibler divergence** from distribution Q to distribution P is:

$$D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \quad (16)$$

Remark 1.1. KL divergence is **not symmetric**: $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ in general.

This will be useful later when comparing agent distributions to uniform or to each other.

2 Game Theory Foundations

Understanding why agents specialize requires game-theoretic reasoning. We introduce the key concepts needed to prove that homogeneous strategies are unstable.

2.1 Strategic Games and Nash Equilibrium

Definition 2.1 (Strategic Game). A **strategic game** (or normal-form game) consists of:

- A finite set of **players** $\mathcal{N} = \{1, 2, \dots, N\}$
- For each player i , a set of **strategies** $\mathcal{S}_i$
- For each player i , a **payoff function** $u_i : \mathcal{S}_1 \times \dots \times \mathcal{S}_N \rightarrow \mathbb{R}$

Definition 2.2 (Best Response). Strategy s_i^* is a **best response** to strategies s_{-i} (strategies of all other players) if:

$$u_i(s_i^*, s_{-i}) \geq u_i(s_i, s_{-i}) \quad \forall s_i \in \mathcal{S}_i \quad (17)$$

Definition 2.3 (Nash Equilibrium). A strategy profile $(s_1^*, \dots, s_N^*)$ is a **Nash Equilibrium** if every player's strategy is a best response to the other players' strategies:

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad \forall i \in \mathcal{N}, \forall s_i \in \mathcal{S}_i \quad (18)$$

Intuition

At Nash Equilibrium, no player can improve their payoff by unilaterally changing their strategy. Everyone is doing as well as they can, given what others are doing.

2.2 Winner-Take-All Games

Our setting involves a special class of games where only the top performer receives reward.

Definition 2.4 (Winner-Take-All Game). A **winner-take-all** (WTA) game is characterized by:

- Total reward V available in each round
- Only the agent with highest performance receives V
- All other agents receive 0

Example 2.1 (Academic Job Market). Multiple candidates compete for a single faculty position. The “winner” (hired candidate) gets the position; all others get nothing from that search.

Example 2.2 (Olympic 100m Final). Eight sprinters compete. Gold medalist gets fame and sponsorships (high value). Others, even silver and bronze, receive diminishing returns. The pure WTA model considers only the winner.

2.3 The Pigeonhole Principle

A simple but powerful counting argument.

Proposition 2.1 (Pigeonhole Principle). *If n items are placed into m containers and $n > m$, then at least one container contains more than one item.*

Formally: $\exists$ container c such that $|\{\text{items in } c\}| \geq \lceil n/m \rceil$.

Proof. Suppose for contradiction that every container has at most 1 item. Then total items $\leq m < n$, contradiction. $\square$

Example 2.3 (Application to Our Setting). If $N = 8$ agents compete across $R = 4$ regimes, at least one regime has:

$$\left\lceil \frac{8}{4} \right\rceil = 2 \text{ agents}$$

This crowded regime creates incentive for deviation (proven later in Proposition 27.1).

2.4 Competitive Exclusion in Ecology

Our algorithm draws inspiration from ecological niche theory.

Definition 2.5 (Ecological Niche). An organism's **niche** is its role in the ecosystem, including what resources it uses, when it's active, and where it lives.

Proposition 2.2 (Competitive Exclusion Principle (Gause, 1934)). *Two species with identical ecological niches cannot stably coexist in the same environment. One will outcompete the other.*

Example 2.4 (Darwin's Finches). On the Galápagos Islands, finch species evolved different beak shapes to exploit different food sources:

- Large beaks: crack hard seeds
- Thin beaks: catch insects
- Medium beaks: general seeds

This differentiation emerged from competition, not coordination.

Key Point

Our key insight: The same competitive exclusion dynamics that drive biological speciation can drive **agent specialization** in multi-agent systems.

3 Bayesian Inference Foundations

Agents learn which methods work in which regimes through Bayesian updating. We develop this from first principles.

3.1 Bayes' Theorem

Theorem 3.1 (Bayes' Theorem). *For events A and B with $P(B) > 0$:*

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \quad (19)$$

For continuous parameters θ given data D :

$$\underbrace{p(\theta|D)}_{\text{posterior}} = \frac{\overbrace{p(D|\theta)}^{\text{likelihood}} \cdot \overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(D)}_{\text{evidence}}} \quad (20)$$

Intuition

- **Prior** $p(\theta)$: What we believe about θ before seeing data
- **Likelihood** $p(D|\theta)$: How likely is the data given θ
- **Posterior** $p(\theta|D)$: What we believe about θ after seeing data

3.2 The Beta Distribution

The Beta distribution is central to our algorithm because it's the conjugate prior for Bernoulli observations.

Definition 3.1 (Beta Distribution). A random variable θ follows a **Beta distribution** with parameters $\alpha > 0$ and $\beta > 0$, written $\theta \sim \text{Beta}(\alpha, \beta)$, if its PDF is:

$$f(\theta; \alpha, \beta) = \frac{\theta^{\alpha-1}(1-\theta)^{\beta-1}}{B(\alpha, \beta)} \quad (21)$$

where $B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$ is the Beta function, and Γ is the Gamma function.

3.2.1 Properties of the Beta Distribution

Proposition 3.1 (Beta Distribution Properties). *For $\theta \sim \text{Beta}(\alpha, \beta)$:*

$$\mathbb{E}[\theta] = \frac{\alpha}{\alpha + \beta} \quad (22)$$

$$\text{Var}(\theta) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} \quad (23)$$

$$\text{Mode}(\theta) = \frac{\alpha - 1}{\alpha + \beta - 2} \quad (\text{for } \alpha, \beta > 1) \quad (24)$$

Intuition

- α roughly represents “number of successes”
- β roughly represents “number of failures”
- Mean = $\frac{\text{successes}}{\text{total trials}}$ (empirical success rate)
- As $\alpha + \beta$ increases, variance decreases (more confident)

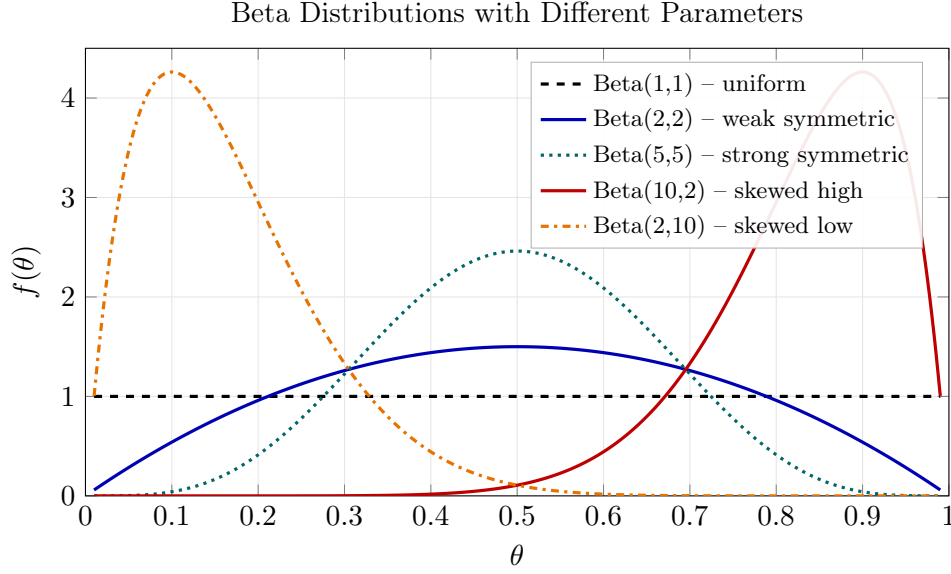


Figure 3: Various Beta distributions. **Beta(1,1)** (dashed black) is uniform — no prior information. **Beta(2,2)** and **Beta(5,5)** (blue solid, teal dotted) are symmetric around $\theta = 0.5$; the higher parameters give tighter concentration. **Beta(10,2)** (red solid) and **Beta(2,10)** (orange dash-dotted) are skewed, peaking near $\theta = 0.9$ and $\theta = 0.1$ respectively, encoding confident beliefs about high vs. low success rates.

3.3 Beta-Bernoulli Conjugacy

Theorem 3.2 (Beta-Bernoulli Conjugacy). *If:*

- *Prior:* $\theta \sim \text{Beta}(\alpha, \beta)$
- *Likelihood:* $X|\theta \sim \text{Bernoulli}(\theta)$

Then the posterior is:

$$\theta|X \sim \text{Beta}(\alpha + X, \beta + (1 - X)) \quad (25)$$

Proof. By Bayes' theorem:

$$p(\theta|X) \propto p(X|\theta) \cdot p(\theta) \quad (26)$$

$$\propto \theta^X (1 - \theta)^{1-X} \cdot \theta^{\alpha-1} (1 - \theta)^{\beta-1} \quad (27)$$

$$= \theta^{\alpha+X-1} (1 - \theta)^{\beta+(1-X)-1} \quad (28)$$

This is the kernel of $\text{Beta}(\alpha + X, \beta + (1 - X))$. $\square$

Key Point

Why this matters: The posterior has the same form as the prior! This means:

- After success ($X = 1$): $\text{Beta}(\alpha, \beta) \rightarrow \text{Beta}(\alpha + 1, \beta)$
- After failure ($X = 0$): $\text{Beta}(\alpha, \beta) \rightarrow \text{Beta}(\alpha, \beta + 1)$

Updates are computationally trivial: just add 1 to the appropriate parameter.

Example 3.1 (Sequential Updates). Start with uninformative prior $\text{Beta}(1, 1)$ (uniform).
Observe: Success, Success, Failure, Success.

After	Distribution	Mean $\mathbb{E}[\theta]$
Prior	$\text{Beta}(1, 1)$	0.50
Success 1	$\text{Beta}(2, 1)$	0.67
Success 2	$\text{Beta}(3, 1)$	0.75
Failure 1	$\text{Beta}(3, 2)$	0.60
Success 3	$\text{Beta}(4, 2)$	0.67

4 Multi-Armed Bandits and Thompson Sampling

4.1 The Multi-Armed Bandit Problem

Definition 4.1 (Multi-Armed Bandit (MAB)). A **K-armed bandit** problem consists of:

- K “arms” (actions) indexed by $a \in \{1, \dots, K\}$
- Each arm a has unknown reward distribution with mean μ_a
- At each time t , the player selects one arm a_t and receives reward $r_t \sim P(\cdot|a_t)$
- Goal: Maximize cumulative reward $\sum_{t=1}^T r_t$

Intuition

Imagine a gambler facing multiple slot machines (“one-armed bandits”). Each machine has a different (unknown) payout rate. The gambler must balance:

- **Exploitation:** Play the machine that seems best so far
- **Exploration:** Try other machines to gather information

Definition 4.2 (Regret). The **regret** after T rounds is:

$$R_T = T \cdot \mu^* - \sum_{t=1}^T \mathbb{E}[r_t] \quad (29)$$

where $\mu^* = \max_a \mu_a$ is the best arm’s mean reward.

4.2 Contextual Bandits (Our Setting)

In our problem, the optimal arm depends on the current **context** (regime).

Definition 4.3 (Contextual Bandit). A **contextual bandit** extends MAB with:

- Context $c_t \in \mathcal{C}$ observed at each round
- Reward distribution depends on both arm and context: $r_t \sim P(\cdot|a_t, c_t)$

Key Point

In our setting:

- **Arms** = Methods $\mathcal{M} = \{m_1, \dots, m_M\}$
- **Contexts** = Regimes $\mathcal{R} = \{r_1, \dots, r_R\}$
- Agents maintain **separate beliefs** for each (method, regime) pair
- Total belief distributions per agent: $M \times R$

4.3 Thompson Sampling Algorithm

Thompson Sampling is a Bayesian approach to the exploration-exploitation tradeoff.

Algorithm 1 Thompson Sampling for Bernoulli Bandits

```
1: Initialize:  $\alpha_a \leftarrow 1, \beta_a \leftarrow 1$  for all arms  $a$  (uniform prior)
2: for  $t = 1, 2, \dots$  do
3:   for each arm  $a$  do
4:     Sample  $\tilde{\theta}_a \sim \text{Beta}(\alpha_a, \beta_a)$ 
5:   end for
6:   Select arm  $a_t \leftarrow \text{argmax}_a \tilde{\theta}_a$ 
7:   Observe reward  $r_t \in \{0, 1\}$ 
8:   Update:  $\alpha_{a_t} \leftarrow \alpha_{a_t} + r_t, \beta_{a_t} \leftarrow \beta_{a_t} + (1 - r_t)$ 
9: end for
```

4.3.1 Why Thompson Sampling Works

Theorem 4.1 (Thompson Sampling Property (Informal)). *Thompson Sampling selects arm a with probability equal to the probability that a is optimal:*

$$P(\text{select } a) = P(a = \text{argmax}_{a'} \mu_{a'}) \quad (30)$$

where the probability is over the posterior distribution of μ .

Intuition

- **High mean, low variance:** Arm is probably good $\rightarrow$ often selected (exploitation)
- **Unknown (high variance):** Might sample high value $\rightarrow$ occasionally selected (exploration)
- **Low mean, low variance:** Arm is probably bad $\rightarrow$ rarely selected

4.3.2 Thompson Sampling in Contextual Setting

For our contextual bandit (with regimes as context):

Algorithm 2 Contextual Thompson Sampling for Our Setting

```
1: Initialize:  $\beta_{m,r}^+ \leftarrow 1, \beta_{m,r}^- \leftarrow 1$  for all methods  $m$ , regimes  $r$ 
2: for iteration  $t = 1, 2, \dots$  do
3:   Observe current regime  $r_t$ 
4:   for each method  $m$  do
5:     Sample  $\tilde{\theta}_m \sim \text{Beta}(\beta_{m,r_t}^+, \beta_{m,r_t}^-)$ 
6:   end for
7:   Select method  $m_t \leftarrow \text{argmax}_m \tilde{\theta}_m$ 
8:   Execute method, observe reward  $R_t$ 
9:   Convert to binary: success  $\leftarrow \mathbf{1}[R_t > \text{threshold}]$ 
10:  Update:  $\beta_{m_t,r_t}^+ \leftarrow \beta_{m_t,r_t}^+ + \text{success}$ 
11:          $\beta_{m_t,r_t}^- \leftarrow \beta_{m_t,r_t}^- + (1 - \text{success})$ 
12: end for
```

Example 4.1 (Thompson Sampling in Crypto Domain). Agent has 5 methods, 4 regimes $\rightarrow$ 20 separate Beta distributions.

Current regime: **Bull**

Current beliefs for Bull regime:

Thompson Sampling: Why Uncertain Arms Get Explored

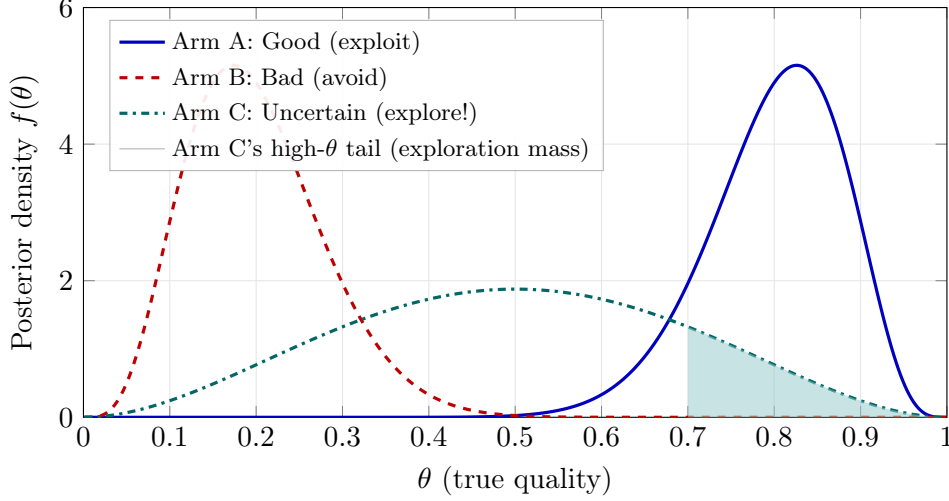


Figure 4: Posterior densities for three Thompson-sampling arms. **Arm A** (blue solid) is confidently good — its posterior $\text{Beta}(20, 5)$ concentrates near $\theta \approx 0.83$. **Arm B** (red dashed) is confidently bad — $\text{Beta}(5, 20)$ concentrates near $\theta \approx 0.17$. **Arm C** (teal dash-dot) is uncertain — its broad $\text{Beta}(3, 3)$ posterior carries non-negligible probability mass in the high- θ tail (shaded teal). When Thompson Sampling draws from Arm C, that tail occasionally produces a sample exceeding Arm A’s likely draw, and Arm C wins the argmax — this is exploration. Arm B’s mass in the high- θ region is effectively zero, so it is almost never chosen.

Method	β^+	β^-	Mean
naive	5	12	0.29
momentum_short	15	8	0.65
momentum_long	22	5	0.81
mean_revert	3	18	0.14
trend	18	7	0.72

Sampling step:

$$\tilde{\theta}_{\text{naive}} \sim \text{Beta}(5, 12) \rightarrow 0.25 \quad (31)$$

$$\tilde{\theta}_{\text{momentum_short}} \sim \text{Beta}(15, 8) \rightarrow 0.71 \quad (32)$$

$$\tilde{\theta}_{\text{momentum_long}} \sim \text{Beta}(22, 5) \rightarrow 0.85 \quad \leftarrow \text{highest!} \quad (33)$$

$$\tilde{\theta}_{\text{mean_revert}} \sim \text{Beta}(3, 18) \rightarrow 0.18 \quad (34)$$

$$\tilde{\theta}_{\text{trend}} \sim \text{Beta}(18, 7) \rightarrow 0.68 \quad (35)$$

Selection: momentum_long (makes sense—it has highest mean AND sampled highest).

After execution: If successful, update $\text{Beta}(22, 5) \rightarrow \text{Beta}(23, 5)$.

4.4 Belief Evolution Over Time

Example 4.2 (How an Agent Becomes a Specialist). Let’s trace Agent A’s beliefs for the “momentum_long” method across iterations:

Scenario: Agent A wins frequently in Bull regime using momentum_long.

Analysis:

- In **Bull regime**: Agent A has used momentum_long 120 times with 102 successes. Mean belief: $\frac{102}{120} = 0.85$.

Table 1: Evolution of Agent A’s Beliefs for momentum_long

Iteration	Bull Regime		Bear Regime		Method Selection
	β^+	β^-	β^+	β^-	
0	1	1	1	1	Random
50	18	5	3	8	Prefer mom. in Bull
100	35	8	4	12	Strong prefer in Bull
200	58	12	5	15	Expert in Bull
500	102	18	6	22	Near-deterministic

- In **Bear regime**: Agent A has only used momentum_long 28 times (rarely wins in Bear). Mean belief: $\frac{6}{28} = 0.21$.

Result: Agent A has learned that momentum_long works in Bull but not Bear. This creates **method-regime specialization**.

4.5 Why Different Agents Learn Different Beliefs

Key Point

Due to **winner-take-all** dynamics:

- Agent A wins often in Bull $\rightarrow$ accumulates evidence for Bull methods
- Agent B wins often in Bear $\rightarrow$ accumulates evidence for Bear methods
- Agent C wins often in Sideways $\rightarrow$ accumulates evidence for Sideways methods

Each agent’s belief matrix becomes specialized for their winning conditions.

Example 4.3 (Three Agents After 500 Iterations). Beliefs for “momentum_long” method across all regimes:

Agent	Bull	Bear	Sideways	Volatile
A (Bull specialist)	0.85	0.20	0.15	0.30
B (Bear specialist)	0.35	0.75	0.25	0.40
C (Sideways specialist)	0.30	0.25	0.72	0.28

Observation: Each agent has high confidence for one regime and low for others. This is emergent from competition, not pre-programmed!

4.6 Connection to Niche Affinity

The Thompson Sampling beliefs and niche affinity work together:

1. **Niche affinity** (α): Which *regimes* an agent specializes in
2. **Method beliefs** (β): Which *methods* work in each regime

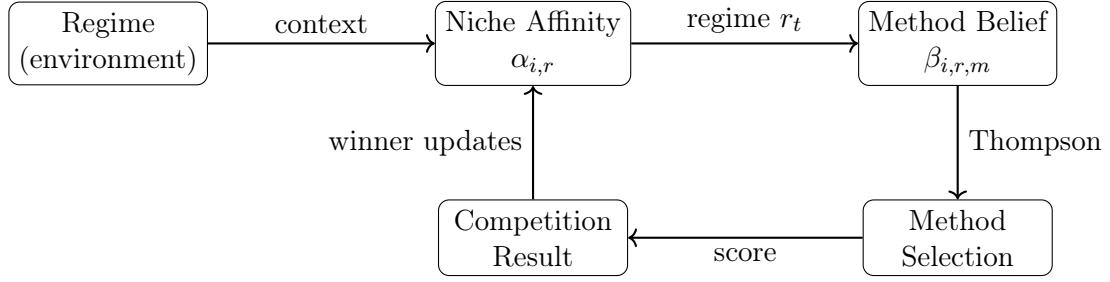


Figure 5: The complete information flow: regimes inform affinity, affinity determines which beliefs to use, beliefs determine method selection, competition determines who updates.

Part II

The Environment: Regimes and Structure

Having established the mathematical foundations, we now describe the environment in which agents operate.

5 Regime-Switching Environments

5.1 Formal Definition

Definition 5.1 (Regime-Switching Environment). A **regime-switching environment** consists of:

- A finite set of **regimes** $\mathcal{R} = \{r_1, r_2, \dots, r_R\}$
- A **stationary distribution** $\pi : \mathcal{R} \rightarrow [0, 1]$ with $\sum_{r \in \mathcal{R}} \pi(r) = 1$
- At each time t , the environment is in regime $r_t \sim \pi(\cdot)$

Intuition

Think of regimes as the “mood” or “state” of the world:

- **Weather:** Clear, Cloudy, Rainy, Stormy
- **Markets:** Bull, Bear, Sideways, Volatile
- **Traffic:** Rush hour, Midday, Night, Weekend

The key property: **different regimes favor different strategies.**

5.2 Why Regimes Enable Specialization

Proposition 5.1 (Necessity of Regime Heterogeneity). *If all regimes had identical optimal strategies, there would be no incentive for agents to specialize differently.*

Proof Sketch. Suppose method m^* is optimal in all regimes. Then all agents would converge to m^* , competing directly. There’s no “ecological niche” to partition.

Our experiments verify this: when we create mono-regime environments (one regime dominates), SI drops toward 0. $\square$

Table 2: Domain Overview

Domain	Data Source	Regimes	Regime Types
Crypto	Bybit Exchange	4	bull, bear, sideways, volatile
Commodities	FRED (US Gov)	4	rising, falling, stable, volatile
Weather	Open-Meteo	4	clear, cloudy, rainy, extreme
Solar	Open-Meteo	4	high, medium, low, night
Traffic	NYC TLC	6	morning_rush, evening_rush, midday, night, weekend, transit
Air Quality	Open-Meteo	4	good, moderate, unhealthy_sensitive, unhealthy

5.3 The Six Domains

We validate our approach across six real-world domains, each with natural regime structure:

6 Bounded Capacity and Memory Models

The foundations above (entropy, Bayesian beliefs, regime switching) describe *how* a learner represents and updates knowledge of a regime. They say nothing about a constraint that is central to the reward-independence result: no real learner can hold perfect, independent knowledge of an unbounded number of regimes. Parameters are finite, working memory is finite, and learning a new skill interferes with old ones. This section formalizes that constraint with two complementary *memory models*—a hard one and a soft one—which together let us show that our central finding is a property of bounded capacity itself, not of one particular bookkeeping rule.

The one-sentence version

If you have ever written an **LRU cache**, you already understand the hard model: a learner has K cache slots for “skills,” and learning a brand-new skill evicts the one used least recently. The soft model is the same idea without the hard cliff—more like a neural network, where training on a new task slowly *overwrites* the weights that stored an old one. In both, a learner that juggles more skills than it has room for keeps forgetting and relearning. The entire payoff of GAUSE is that a *specialist* only ever holds one skill, so it never overflows and never forgets.

6.1 The Capacity Budget

Definition 6.1 (Per-agent capacity K). Each agent is endowed with a **capacity budget** $K \in \{1, 2, \dots, R\}$: the number of regimes whose beliefs it can hold at full fidelity at once. An agent’s **belief store** is the set of regimes for which it currently maintains non-prior Beta parameters,

$$\mathcal{S}_a(t) = \{r \in \mathcal{R} : \text{agent } a \text{ holds learned beliefs } \{(\alpha_{r,m}, \beta_{r,m})\}_m \text{ at time } t\}.$$

The regime $\rightarrow$ agent **assignment** is the induced map from each regime to the agent(s) responsible for serving it. Capacity binds whenever an agent is asked to serve more than K regimes.

Intuition

K is the number of “skills” a single learner can keep sharp. A monolith with K slots facing $R > K$ regimes is like one analyst told to be expert in R markets but who can only keep K playbooks in active memory: every time a new market demands attention, an old playbook gets bumped. A *population* of specialists sidesteps this—each member keeps a single playbook and never bumps it—which is exactly the structural advantage the experiments isolate.

6.2 Hard Model: LRU Slots with Eviction

The first model treats capacity as a hard cap of K discrete slots with *least-recently-used* (LRU) eviction—the same policy used by CPU caches and page replacement.

Definition 6.2 (Hard LRU memory). The belief store is an ordered set of at most K regimes, ordered by recency of use. When the active regime r_t is served:

1. **Hit** ($r_t \in \mathcal{S}_a$): mark r_t most-recently-used; beliefs persist.
2. **Miss** ($r_t \notin \mathcal{S}_a$): instantiate fresh beliefs $(\alpha_{r_t,m}, \beta_{r_t,m}) = (1, 1)$ for all methods m (the uninformed prior of Section 6). If this makes $|\mathcal{S}_a| > K$, **evict** the least-recently-used regime, discarding its learned beliefs.

The defining consequence: a regime that is evicted and later reactivates returns at the uninformed prior Beta(1,1) and must be *relearned from scratch*. Recently learned knowledge has destroyed older knowledge.

Example 6.1 (Eviction under a non-stationary stream). Let $R = 4$ regimes $\{A, B, C, D\}$ and $K = 2$. The environment activates regimes in the order $A, B, C, A, \dots$. The store $\mathcal{S}$ (left = least-recently-used) evolves:

Step	Active	Action	Store after (LRU $\rightarrow$ MRU)
1	A	miss, learn A	$[A]$
2	B	miss, learn B	$[A, B]$
3	C	miss, learn C , evict A	$[B, C]$
4	A	miss again, relearn A from prior	$[C, A]$

At step 4 the monolith has *forgotten* everything it knew about A : its beliefs are back at Beta(1,1), and the post-reactivation error spikes while it relearns. This is catastrophic forgetting in its starkest form.

In code the store is literally an ordered dictionary keyed by regime, with the oldest entry evicted on overflow—the textbook LRU cache:

```

1 class StoreAgent:                                # one capacity-K learner
2     store = OrderedDict()                          # regime -> beliefs; order =
3     recency                                       #
4
5     def serve(self, r):
6         if r in self.store:                        # HIT: refresh recency, keep
7             beliefs                               #
8             self.store.move_to_end(r)
9         else:                                     # MISS: learn r from the
10            uninformed prior
11            self.store[r] = fresh_beliefs()
12            if len(self.store) > self.K:
13                self.store.popitem(last=False)    # EVICT least-recently
14                                                    # -used

```

In GAUSE

This hard-LRU agent *is* the failing baseline in our non-stationary experiment, the **monolith** arm: one learner with K slots that must serve whatever regime is active now. With $R = 6$ traffic regimes and $K = 1$ it can hold exactly one regime at a time, so every regime switch is a miss-and-relearn. That is why its post-reactivation error is ≈ 1.0 (worst possible) while a GAUSE population scores 0.28—the population just parks each regime in a different agent.

6.3 Soft Model: Interference Decay

The hard model’s discreteness invites an obvious objection: maybe the forgetting is an artifact of the abrupt eviction rule. The soft model removes eviction entirely and replaces it with *graded interference*—knowledge bleeds away continuously when an agent tries to hold more than K regimes, mirroring how gradient updates for a new task perturb the weights that encoded an old one.

Definition 6.3 (Soft interference memory). Define a regime’s **strength** as the net pseudo-counts accumulated beyond the uninformed prior,

$$s_a(r) = \sum_m (\alpha_{r,m} + \beta_{r,m} - 2),$$

and the agent’s **footprint** as the number of regimes it actually knows, $F_a = |\{r : s_a(r) > \varepsilon\}|$ with threshold $\varepsilon = 0.5$. On every belief update to the current regime r_t , every *other* held regime $r \neq r_t$ decays toward the prior:

$$\alpha_{r,m} \leftarrow 1 + (\alpha_{r,m} - 1) \kappa, \quad \beta_{r,m} \leftarrow 1 + (\beta_{r,m} - 1) \kappa, \quad \kappa = 1 - \rho \frac{(F_a - K)^+}{F_a},$$

where $\rho = 0.04$ is the base interference rate and $(x)^+ = \max(0, x)$ is the capacity overflow. No regime is ever discarded; capacity binds only through the multiplicative decay factor $\kappa \leq 1$.

Two properties make this a faithful soft analogue of forgetting. First, decay is *gated by overflow*: if $F_a \leq K$ then $\kappa = 1$ and nothing is lost—an agent within its budget forgets nothing. Second, a *dormant* regime is never refreshed (it emits no updates of its own), so each update to an active regime shrinks its pseudo-counts geometrically; after n interfering updates its strength is scaled by $\kappa^n \rightarrow 0$, i.e. it relaxes back to the uninformed prior—forgotten, but smoothly rather than by a cliff.

Why this is the “neural network” version of forgetting

A deep network has no discrete slots—it has shared weights. When you fine-tune it on task B , gradient steps nudge the very weights that encoded task A , and A ’s accuracy quietly erodes. The soft model captures exactly this: every update to the active regime A shrinks the stored evidence for the dormant regime B by a constant factor $\kappa < 1$. No single step “deletes” B ; it just fades. This is why we test *both* models—to show the GAUSE advantage is not an artifact of the discrete cache rule.

Example 6.2 (Graded decay at $K = 1$). An agent with budget $K = 1$ has learned regime B to strength $s(B) = 20$, then the environment switches to A and stays there. Once both are known the footprint is $F = 2$, so overflow = 1 and $\kappa = 1 - 0.04 \cdot \frac{1}{2} = 0.98$. Each update to A multiplies B ’s net pseudo-counts by 0.98. B decays geometrically:

Updates to A , n	0	50	100	200	300
B ’s strength $s(B) = 20 \cdot 0.98^n$	20.0	7.3	2.7	0.36	0.05
Still “known” ($> \varepsilon = 0.5$)?	yes	yes	yes	no	no

By $n \approx 200$, B has effectively decayed to the uninformed prior—forgotten smoothly, with no discrete eviction event ever occurring.

In GAUSE

The soft model is the --**soft** robustness run reported in the paper. The story is identical to the hard model: the monolith forgets (post-reactivation error 0.85–0.92) while the GAUSE population retains (≈ 0.25), $p \sim 10^{-40}$. Same conclusion, different forgetting mechanism—so the result is about *capacity*, not about one bookkeeping trick.

6.4 Why a Specialist Is Structurally Immune

Both models penalize exactly one thing: being responsible for more regimes than K . The mechanism’s payoff is now visible at the level of the assignment.

Key Point

A converged specialist serves a *single* niche, so its footprint is $F = 1 \leq K$ for any $K \geq 1$. Under the hard model it never triggers an eviction; under the soft model its overflow is 0, so $\kappa = 1$ and it suffers *zero* interference. While its niche is dormant it simply performs no updates and its beliefs sit untouched—a distributed, persistent memory. A capacity- K monolith facing $R > K$ regimes, by contrast, has footprint $R > K$ and forgets under *either* model. The separation we report is therefore a property of bounded capacity per se, reproduced under both the discrete and the graded rule (Section 17 and the soft-robustness ablation).

7 Mixture-of-Experts: Gating, Routing, and the Gate Objective

The natural machine-learning answer to “many regimes, capacity-limited learners” is a *Mixture-of-Experts* (MoE): a pool of expert sub-models plus a learned **gating network** that routes each input to the expert(s) best suited to it. MoE is the canonical learned-routing baseline in our five-arm comparison, and understanding precisely how its gate is trained is what makes the reward-independence result sharp. This section develops the MoE mechanism from first principles.

Think of the gate as a learned load balancer

Picture a web load balancer that routes each incoming request to one of N backend servers, and *learns* over time which server handles which kind of request fastest. That is exactly an MoE gate: the “request” is the current regime, the “servers” are the experts, and the gate learns a routing table by watching how well each server performs. The catch—and the whole point of this section—is *what* the load balancer optimizes. A vanilla gate optimizes “route to whoever is good *right now*.” That sounds reasonable, but it means the gate has no reason to keep a server warm for a request type that has stopped arriving. When that request type comes back, the assigned server has been repurposed and forgotten how to handle it.

7.1 Architecture: Experts and a Gate

Definition 7.1 (Mixture-of-Experts). An MoE layer consists of N **experts** $f_1, \dots, f_N$ (here, N capacity- K agents) and a **gating network** g that maps an input/context x (here, the active regime r) to a distribution or score vector over experts. The output is the gated combination

$$y(x) = \sum_{e=1}^N g_e(x) f_e(x), \quad g_e(x) \geq 0, \quad \sum_e g_e(x) = 1.$$

In **top-1** (hard) routing only the single highest-scoring expert is activated, $y(x) = f_{e^*}(x)$ with $e^* = \arg\max_e g_e(x)$; top- k routing activates the k best. Hard routing is what we use, since it mirrors “one agent serves this regime.”

A standard gate parameterizes the scores as a softmax over learned logits $\theta(x)$,

$$g_e(x) = \frac{\exp(\theta_e(x))}{\sum_{e'} \exp(\theta_{e'}(x))},$$

and trains θ end-to-end with the task loss. Our experiment uses an explicit per-regime logit table $\theta_e(r) \equiv \text{gate}[r][e]$, which isolates the routing question (which expert owns regime r ?) from representation learning.

7.2 The Gate Objective Is Reward-Driven

The crucial structural fact is *what signal trains the gate*. The gate is rewarded for routing to experts that perform well *right now*. With ε -greedy exploration the routed expert is

$$e_t = \begin{cases} \text{uniform}(1, \dots, N) & \text{w.p. } \varepsilon_{\text{route}}, \\ \text{argmax}_e \text{gate}[r_t][e] & \text{otherwise,} \end{cases}$$

and after observing that expert’s normalized quality $q_{e_t} \in [0, 1]$ on the active regime, the gate logit is updated by a reward-following step

$$\text{gate}[r_t][e_t] \leftarrow \text{gate}[r_t][e_t] + \eta_g (q_{e_t} - \tfrac{1}{2}), \quad (36)$$

with gate learning rate η_g . The routed expert then learns/retains the regime in its own bounded (LRU) store—so capacity binds inside each expert exactly as in Section 6.2.

In code, one step of the router is just “pick an expert, grade it, nudge the routing table”:

```

1 def moe_step(r):                                # r = active regime
2     if random() < eps_route:                    e = randint(N)      # explore
3     else:                                        e = argmax(gate[r]) # exploit best-known
4         expert
5     q = experts[e].serve(r)                     # quality in [0, 1]
6     experts[e].learn(r)                         # expert stores r in
        its LRU
    gate[r][e] += gate_lr * (q - 0.5)             # reward-following
        gate update

```

Example 7.1 (Gate concentrates on the currently-good expert). With $N = 3$ experts and logits $\text{gate}[r] = (0, 0, 0)$, suppose expert 2 is routed (via exploration) and returns quality $q_2 = 0.9$. Then $\text{gate}[r][2] += \eta_g(0.9 - 0.5) = 0.4\eta_g$, so future routing of r favors expert 2. Over many *active* steps the gate sharpens onto whichever expert serves r well—standard, sensible behavior while r is active.

Example 7.2 (The same gate, walked through a dormancy window). Now follow regime r as it goes dormant. Say the gate has converged to $\text{gate}[r] = (0.1, 1.8, 0.2)$, so r is routed to expert 2, which knows r well. Then r goes dormant for a long stretch while regimes r', r'' fire:

1. **During dormancy:** r never activates, so no q for r is ever observed, so $\text{gate}[r]$ is *frozen* at $(0.1, 1.8, 0.2)$. The gate cannot “decide” to protect expert 2 for r —it gets no signal at all.
2. **Meanwhile** expert 2 is the best available learner, so it keeps getting routed for r', r'' . With capacity $K=1$ it evicts r to make room.
3. **Reactivation:** r returns; the frozen gate dutifully routes it back to expert 2—which has forgotten r and must relearn from scratch.

The router did everything “right” by its own objective and still forgot. The failure is structural, not a tuning bug.

In GAUSE

This is the **MoE-gate** arm of the five-arm comparison ($\eta_g = 0.2$, $\varepsilon_{\text{route}} = 0.1$). It is a strong baseline on *spatial* coverage—it ties competition when all regimes stay active—but it *fails* the non-stationary retention test almost as badly as the monolith (post-reactivation 0.928 vs. our 0.283 at $K=1$, $p \sim 10^{-35}$). GAUSE wins precisely where a reward-driven router cannot help itself.

Key Point

Read Eq. (36) carefully: the gate for regime r is updated *only on steps where r is active*, because only then is a quality q_e for r observed. When r goes dormant it emits no steps, so $\Delta \text{gate}[r][\cdot] = 0$ throughout dormancy—the gate receives **no signal** that would lead it to reserve an idle expert for r . Meanwhile the expert that used to serve r keeps serving *active* regimes and, being capacity-bounded, evicts (or lets decay) its beliefs about r . On reactivation the gate still points at an expert that has forgotten r . This is the precise mechanism formalized in Proposition 30.1, and it is a property of the reward-driven objective, not of MoE as an architecture: a gate augmented with an explicit memory-preservation or load-balancing term could reserve capacity for dormant regimes, at the cost of the very reward-following simplicity that defines a vanilla router.

7.3 Load Balancing and Its Absence Here

Production MoE systems add an auxiliary **load-balancing** loss to discourage the gate from collapsing all traffic onto a few experts (e.g. a term penalizing the squared deviation of per-expert load from uniform). Such terms are *reward-independent* pressures on the assignment, and are conceptually the kind of mechanism that *could* rescue retention. We deliberately study the plain reward-driven gate (no balancing term) because it is the minimal, most-common router and it makes the signal-availability argument cleanest; we flag the augmented case as outside the assignment-only class our principle scopes over (Section 30).

8 Continual Learning and Catastrophic Forgetting

The non-stationary experiment is, in modern terms, a **continual learning** (CL) problem, and the monolith’s failure is **catastrophic forgetting**. This section gives the standard formalization and situates GAUSE within the CL taxonomy, so the contribution can be read against the right literature [4, 8, 15].

The classic demo, in one paragraph

Train a neural net to classify cats vs. dogs (task A); it hits 99%. Now keep training the *same* network on cars vs. planes (task B). It learns B fine—but if you re-test task A , accuracy has cratered to near chance. The network did not “run out of room” in any obvious sense; the gradient steps for B simply overwrote the weights that encoded A . That is catastrophic forgetting, and it is the single biggest obstacle to learning a stream of tasks over time. Our regimes are the “tasks,” and a regime going dormant then reactivating is exactly the “re-test task A later” step.

8.1 The Continual Learning Problem

Definition 8.1 (Continual learning). A learner faces a *sequence* of tasks/regimes $r_1, r_2, \dots$ presented over time, rather than i.i.d. samples from a fixed mixture. At any time it can observe

data only from the current regime, yet it is evaluated on *all* regimes seen so far—including those that are currently dormant. The goal is to keep accumulating competence without degrading on past regimes.

Definition 8.2 (Catastrophic forgetting). **Catastrophic forgetting** is the abrupt loss of performance on previously learned regimes caused by learning new ones, when the shared finite capacity used to represent the old regime is overwritten in service of the new. In our hard model this is eviction (Section 6.2); in our soft model it is interference decay (Section 6.3).

8.2 The Stability–Plasticity Dilemma

Intuition

Continual learning is a tension between two opposing requirements: **plasticity** (be able to learn the new regime well) and **stability** (do not destroy what you already know). A purely plastic learner overwrites old knowledge instantly (the monolith); a purely stable learner cannot adapt at all. Every CL method is, at heart, a different way of partitioning or protecting capacity so that plasticity for new regimes does not come at the cost of stability on old ones.

8.3 A Taxonomy of CL Methods, and Where GAUSE Sits

Standard CL approaches fall into three families [15]:

1. **Regularization** (e.g. Elastic Weight Consolidation, EWC [8]): add a penalty that anchors parameters important to old tasks, slowing their drift. Protection is *soft* and requires an importance estimate per parameter.
2. **Replay / rehearsal**: store or generate samples of old regimes and interleave them with new data, so old knowledge is continually refreshed. Protection requires *memory of the data* and continued replay.
3. **Architectural / parameter isolation**: give different tasks different parameters (dedicated sub-networks, masks, or experts), so learning a new task physically cannot overwrite an old one.

Table 3 maps each family to a familiar CS analogue and to the corresponding arm in our experiments.

Table 3: The three CL families, a CS analogue, and the matching experimental arm.

CL family	Familiar analogue	In our five-arm comparison
Regularization (EWC)	A “don’t-touch” lock on important variables	(not tested—needs per-parameter importance)
Replay / rehearsal	Keeping a log and re-running old jobs	(not tested—needs stored data)
Architectural isolation	Separate microservices per job type	GAUSE , random niches, EOI diversity
<i>No protection</i>	One shared box doing everything	monolith, MoE router

Key Point

GAUSE is an **architectural** / **parameter-isolation** method—but an unusual one. Where standard parameter-isolation methods *assign* experts to tasks by an external controller or task label, GAUSE lets the regime $\rightarrow$ agent assignment *emerge* from competitive exclusion, with no labels and no controller. The converged assignment is *reward-independent*: each specialist owns its niche structurally and idles (without updating, hence without forgetting) while the niche is dormant. This is precisely the property that [11] prove a learned MoE gate must be *frozen* to obtain for continual-learning convergence—freezing the gate is, in our language, making the assignment reward-independent. GAUSE reaches that state emergently rather than by externally terminating gate updates. The next part makes this argument a formal proposition.

Example 8.1 (Forgetting vs. retention on the same stream). Take $R = 3$ regimes $\{A, B, C\}$, capacity $K = 1$, and the activation stream

$$A, A, \underbrace{B, C, B, C, \dots}_{A \text{ dormant}}, A$$

Compare the monolith (one shared learner) with a GAUSE population (one specialist per regime):

Phase	Monolith ($K=1$)	GAUSE population
Learn A	masters A	agent _{A} masters A
A dormant (B, C active)	evicts A to serve B, C	agent _{A} <i>idles</i> ; agent _{B} , agent _{C} work
A reactivates	A is gone—relearn from prior	agent _{A} still has A —instant

The monolith pays a relearning spike every time a dormant regime returns; the population pays nothing, because the knowledge was never in the way of anything else. This is the entire effect we measure as the +71% post-reactivation gap.

9 Domain-Specific Regime Definitions

We validate GAUSE on six real-world domains. Each domain defines its own regime set $\mathcal{R}$ and stationary distribution π (Definition in Section 6’s parent on regime-switching). Two features matter for everything that follows. First, the regimes are *heterogeneous*: different regimes reward different prediction methods, which is the precondition for specialization (Proposition on regime heterogeneity). Second, the distributions are *skewed*: some regimes are common and others rare, which is where a specialist population earns its keep—a rare regime gets a dedicated owner instead of being averaged away by a generalist. Each table below lists the regimes, their long-run frequency $\pi(r)$, and a plain-language description.

9.1 Crypto Domain

Cryptocurrency markets (Bybit, hourly) switch between directional trends and choppy consolidation. **Sideways** is the most common state, while **Volatile** is rare but high-stakes—exactly the kind of low-frequency regime a generalist under-serves and a specialist protects.

Table 4: Crypto Domain Regimes and Probabilities

Regime	$\pi(r)$	Description
Bull	0.30	Sustained upward price movement, positive sentiment
Bear	0.20	Sustained downward price movement, negative sentiment
Sideways	0.35	Price consolidation, low directional movement
Volatile	0.15	Large price swings in both directions

9.2 Weather Domain

Weather (Open-Meteo) is dominated by ordinary **Cloudy/Clear** conditions, with **Extreme** events (storms, heat waves) appearing only 10% of the time. Its low effective regime count ($k_{\text{eff}} \approx 1.8$) makes it the hardest domain to specialize on—consistent with the Mono-Regime Collapse result (Proposition 29.1).

Table 5: Weather Domain Regimes and Probabilities

Regime	$\pi(r)$	Description
Clear	0.30	Stable, sunny conditions with predictable temperatures
Cloudy	0.35	Overcast with moderate variability
Rainy	0.25	Precipitation with associated temperature changes
Extreme	0.10	Severe weather events (storms, heat waves, cold snaps)

9.3 Traffic Domain

NYC taxi traffic has the richest regime structure of all six domains—six regimes including two sharp, rare commute peaks (**Morning/Evening Rush**, 9% each). This high diversity ($R = 6$) is why Traffic reaches the highest specialization ($SI = 0.995$) and is the clearest win for a specialist population.

Table 6: Traffic Domain Regimes and Probabilities

Regime	$\pi(r)$	Description
Morning Rush	0.09	7–9 AM weekday commute peak
Evening Rush	0.09	5–7 PM weekday commute peak
Midday	0.21	10 AM – 4 PM moderate traffic
Night	0.18	Low traffic period (10 PM – 6 AM)
Weekend	0.29	Different patterns from weekdays
Transition	0.14	Periods between major regimes

9.4 Commodities Domain

Commodity prices (FRED) move between supply/demand-driven **Rising** and **Falling** trends, mean-reverting **Stable** periods, and rare supply-shock **Volatile** episodes. The roughly symmetric rising/falling split makes momentum vs. mean-reversion methods cleanly separable by regime.

Table 7: Commodities Domain Regimes and Probabilities

Regime	$\pi(r)$	Description
Rising	0.25	Sustained price increases (demand > supply)
Falling	0.25	Sustained price decreases (supply > demand)
Stable	0.35	Price consolidation around equilibrium
Volatile	0.15	Large swings due to supply shocks or speculation

9.5 Solar Domain

Solar irradiance (Open-Meteo) is the most physically structured domain: its regimes follow the sun. **Night** (zero irradiance) is perfectly predictable, while **High/Medium/Low** track sky conditions. This clean, near-deterministic structure yields very high SI (0.995) with low variance.

Table 8: Solar Domain Regimes and Probabilities

Regime	$\pi(r)$	Description
High	0.25	Clear sky, peak irradiance (midday, summer)
Medium	0.30	Partial clouds or lower sun angle
Low	0.20	Heavy clouds, rain, or early/late hours
Night	0.25	Zero solar irradiance (nighttime hours)

9.6 Air Quality Domain

Air quality (Open-Meteo) is the most *imbalanced* domain: **Good** and **Moderate** together cover 95% of the time, while the health-critical **Unhealthy** regimes are extremely rare ($\leq 4\%$). This is the textbook case for specialization—a generalist optimizing average accuracy will essentially ignore the rare-but-important regimes, whereas a dedicated specialist keeps them covered.

Table 9: Air Quality Domain Regimes and Probabilities

Regime	$\pi(r)$	Description
Good	0.40	AQI 0–50, healthy conditions
Moderate	0.55	AQI 51–100, acceptable for most
Unhealthy (Sensitive)	0.04	AQI 101–150, risk for sensitive groups
Unhealthy	0.01	AQI 151+, health risk for all

Part III

Methods and Regime-Method Affinity

10 The Method Inventory

10.1 Formal Definition

Definition 10.1 (Method Space). The **method space** $\mathcal{M} = \{m_1, m_2, \dots, m_M\}$ is the set of available prediction/trading strategies. Each method m is a function:

$$m : \text{History} \rightarrow \text{Prediction/Action} \quad (37)$$

In our experiments, each domain has $M = 5$ methods tailored to its structure.

10.2 Mathematical Description of Common Methods

10.2.1 Naive (Persistence)

The simplest method: predict that the next value equals the current value.

$$\hat{y}_{t+1} = y_t \quad (38)$$

Properties:

- Works well when values are stable (low volatility)
- Fails when there are trends or mean reversion
- Baseline for comparison

10.2.2 Moving Average (MA)

Predict based on the average of recent values.

$$\hat{y}_{t+1} = \frac{1}{k} \sum_{i=0}^{k-1} y_{t-i} = \text{MA}_k(t) \quad (39)$$

Variants:

- MA_3, MA_5 : Short-term smoothing, responsive
- $\text{MA}_7, \text{MA}_{20}$: Long-term smoothing, stable

Properties:

- Smooths out noise
- Lags behind trends
- Parameter k controls responsiveness vs. stability tradeoff

10.2.3 Momentum

Predict that recent trends will continue.

$$\text{signal} = \frac{y_t - y_{t-k}}{y_{t-k}} \quad (40)$$

For trading: $\text{signal} > 0 \Rightarrow \text{Buy}$, $\text{signal} < 0 \Rightarrow \text{Sell}$

Variants:

- Short momentum ($k = 5$): Captures quick moves
- Long momentum ($k = 20$): Captures sustained trends

10.2.4 Mean Reversion

Predict that extreme values will revert toward the mean.

$$z_t = \frac{y_t - \mu_k}{\sigma_k}, \quad \text{signal} = -z_t \quad (41)$$

where μ_k and σ_k are the rolling mean and standard deviation over k periods.

Properties:

- Works in sideways/range-bound markets
- Fails in trending markets (“catching falling knives”)
- Mathematically: bets on regression to the mean

10.2.5 Trend Following (MA Crossover)

Use crossing of fast and slow moving averages as signals.

$$\text{signal} = \text{sign}(\text{MA}_{\text{fast}} - \text{MA}_{\text{slow}}) \quad (42)$$

Properties:

- Fast MA above slow MA $\Rightarrow$ Uptrend signal
- Avoids the lag of pure MA prediction
- Generates whipsaws in sideways markets

10.3 Domain-Specific Methods

10.3.1 Traffic Domain Methods

- **Persistence:** $\hat{V}_{t+1} = V_t$ (next hour = current hour)
- **Hourly Average:** Historical average at this hour
- **Weekly Pattern:** $\hat{V}_{t+1} = V_{t-168}$ (same hour, one week ago)
- **Rush Hour:** Special handling for 7-9 AM and 5-7 PM
- **Exponential Smoothing:** $\hat{V}_{t+1} = \alpha V_t + (1 - \alpha) \hat{V}_t$

10.3.2 Solar Domain Methods

- **Naive:** Persistence
- **MA₆:** 6-hour moving average
- **Clear Sky:** Theoretical solar irradiance based on sun position
- **Seasonal:** Same hour, previous day
- **Hybrid:** Weighted combination of methods

11 Regime-Method Affinity Matrices

11.1 Definition and Interpretation

Definition 11.1 (Regime-Method Affinity). The **affinity** $A(r, m) \in [0, 1]$ measures the expected performance of method m when the environment is in regime r . Higher values indicate better performance.

Key Point

The affinity matrix encodes the structure of the problem:

- Different methods are optimal in different regimes
- This heterogeneity creates the opportunity for specialization
- Agents must learn which methods work where

11.2 Complete Affinity Matrices (Verified from Code)

11.2.1 Crypto Domain

Table 10: Crypto Domain Affinity Matrix

Method \ Regime	Bull	Bear	Sideways	Volatile
naive	0.50	0.30	0.60	0.40
momentum_short	0.80	0.70	0.40	0.60
momentum_long	0.90	0.80	0.30	0.50
mean_revert	0.30	0.40	0.90	0.70
trend	0.85	0.75	0.35	0.50

Interpretation:

- In **Bull** markets: `momentum_long` (0.90) is best—ride the uptrend
- In **Sideways** markets: `mean_revert` (0.90) is best—fade the swings
- In **Volatile** markets: `mean_revert` (0.70) still works best

11.2.2 Commodities Domain

Table 11: Commodities Domain Affinity Matrix

Method \ Regime	Rising	Falling	Stable	Volatile
naive	0.50	0.40	0.70	0.45
ma5	0.70	0.65	0.50	0.60
ma20	0.85	0.80	0.40	0.50
mean_revert	0.30	0.35	0.85	0.70
trend	0.90	0.85	0.30	0.55

Table 12: Weather Domain Affinity Matrix

Method \ Regime	Clear	Cloudy	Rainy	Extreme
naive	0.80	0.60	0.50	0.30
ma3	0.60	0.70	0.75	0.50
ma7	0.50	0.65	0.80	0.55
seasonal	0.75	0.70	0.65	0.40
trend	0.55	0.60	0.70	0.85

11.2.3 Weather Domain

Interpretation:

- In **Clear** weather: **naive** (0.80) works—tomorrow like today
- In **Extreme** weather: **trend** (0.85) captures rapid changes

11.2.4 Solar Domain

Table 13: Solar Domain Affinity Matrix

Method \ Regime	High	Medium	Low	Night
naive	0.70	0.60	0.50	0.90
ma6	0.60	0.70	0.75	0.40
clear_sky	0.90	0.75	0.60	0.30
seasonal	0.75	0.70	0.65	0.85
hybrid	0.80	0.85	0.80	0.70

11.2.5 Traffic Domain

Table 14: Traffic Domain Affinity Matrix

Method	Morning	Evening	Midday	Night	Weekend	Transition
persistence	0.40	0.45	0.70	0.85	0.60	0.50
hourly_avg	0.70	0.70	0.80	0.60	0.65	0.70
weekly_pattern	0.80	0.80	0.70	0.65	0.90	0.60
rush_hour	0.95	0.90	0.50	0.30	0.40	0.60
exp_smooth	0.60	0.65	0.75	0.70	0.70	0.80

11.2.6 Air Quality Domain

11.3 Reward Generation from Affinity

Given the affinity matrix, rewards are generated as:

$$R_i = A(r_t, m_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (43)$$

where:

- r_t is the current regime

Table 15: Air Quality Domain Affinity Matrix

Method \ Regime	Good	Moderate	Unhealthy Sens.	Unhealthy
persistence	0.80	0.60	0.40	0.30
hourly_avg	0.65	0.70	0.60	0.50
moving_avg	0.60	0.75	0.70	0.65
regime_avg	0.75	0.85	0.90	0.85
exp_smooth	0.70	0.70	0.75	0.80

- m_i is the method selected by agent i
- $\sigma = 0.15$ (noise standard deviation)

Example 11.1 (Reward Generation). Agent selects `momentum_long` during Bull regime in Crypto:

$$R = A(\text{Bull}, \text{momentum_long}) + \epsilon \quad (44)$$

$$= 0.90 + \mathcal{N}(0, 0.15^2) \quad (45)$$

$$\approx 0.90 + 0.05 \quad (\text{one possible outcome}) \quad (46)$$

$$= 0.95 \quad (47)$$

Part IV

Specialization Metrics

12 Niche Affinity Distribution

12.1 Definition

Definition 12.1 (Niche Affinity). Each agent i maintains a **niche affinity distribution** $\alpha_i \in \Delta^R$, where Δ^R is the $(R - 1)$ -dimensional probability simplex:

$$\Delta^R = \left\{ \alpha \in \mathbb{R}^R : \alpha_r \geq 0 \ \forall r, \quad \sum_{r=1}^R \alpha_r = 1 \right\} \quad (48)$$

Intuition

$\alpha_{i,r}$ represents agent i 's “affinity” or “preference” for regime r :

- High $\alpha_{i,r}$: Agent has specialized in regime r
- Uniform α_i : Agent is a generalist

12.2 Evolution Over Time

The niche affinity evolves through competitive dynamics via the **exponentiated-gradient** (EG) update, also known as the Hedge algorithm or multiplicative-weights update [9, 1]. For the winner i^* on iteration t with realized regime r_t ,

$$\alpha_{i^*,j}^{(t+1)} = \frac{\alpha_{i^*,j}^{(t)} \exp(\eta \cdot \mathbf{1}[j = r_t])}{\sum_{k=1}^R \alpha_{i^*,k}^{(t)} \exp(\eta \cdot \mathbf{1}[k = r_t])}, \quad (49)$$

while non-winners do not update, $\alpha_{i,r}^{(t+1)} = \alpha_{i,r}^{(t)}$ for $i \neq i^*$.

For the indicator-reward formulation used here, Eq. (49) simplifies operationally to “multiply the winning entry by e^η and renormalize.”

Intuition

Why EG. Equation (49) is mirror descent on the simplex Δ^R with the entropic regularizer $\Phi(\alpha) = \sum_r \alpha_r \log \alpha_r$. The exponential is the inverse of the regularizer's gradient, so the update is a closed-form Bregman projection onto Δ^R given a linear reward signal. In the small- η limit it recovers the continuous-time replicator dynamics $\dot{\alpha}_r = \eta \alpha_r (r_r - \bar{r})$, linking this learning rule to standard models in evolutionary game theory [16].

Remark 12.1 (Structural Properties). Three properties of Eq. (49) hold by construction:

- **Simplex preservation:** the denominator is precisely the sum of the post-update numerators, so $\sum_j \alpha_{i^*,j}^{(t+1)} = 1$ exactly.
- **Strict positivity:** since $\exp(\cdot) > 0$ and $\alpha_{i^*,k}^{(t)} > 0$, the post-update vector lies in the relative interior of Δ^R . No clamping is needed.
- **Proportional shrinkage of non-winners:** every non-winning entry is divided by the same normalization constant, preserving their pairwise ratios across the update.

Key Point

Only the **winner** updates their affinity. This creates the competitive pressure that drives specialization. The EG form makes this update simplex-preserving and non-negative by construction; no post-hoc normalization or clamping is needed.

12.3 Why the V3 Heuristic Fails and Why EG Is Canonical

Earlier versions of this work [12] used an additive heuristic for the affinity update. We first state that heuristic formally and then prove that it suffers from three distinct structural defects (Propositions 12.1–12.3). We then derive Eq. (49) from a single optimization principle (Proposition 12.4) and prove that it avoids all three defects by construction (Propositions 12.5–12.7).

Definition 12.2 (V3 Additive Update). For winner i^* and realized regime r_t , the V3 update first modifies each entry of α_{i^*} according to

$$\tilde{\alpha}_{i^*,r} = \begin{cases} \alpha_{i^*,r}^{(t)} + \eta(1 - \alpha_{i^*,r}^{(t)}) & r = r_t \\ \alpha_{i^*,r}^{(t)} - \frac{\eta}{R-1} & r \neq r_t \end{cases} \quad (50)$$

then applies the floor $\hat{\alpha}_{i^*,r} = \max(\epsilon, \tilde{\alpha}_{i^*,r})$ with $\epsilon = 0.01$, and finally rescales $\alpha_{i^*,r}^{(t+1)} = \hat{\alpha}_{i^*,r} / \sum_k \hat{\alpha}_{i^*,k}$.

V3 Defect 1: Mass drift. The pre-floor, pre-rescaling vector $\tilde{\alpha}_{i^*}$ does not lie on the simplex.

Proposition 12.1 (V3 Mass Drift). Let $\alpha_{i^*}^{(t)} \in \Delta^R$ and apply Eq. (50) with $\eta > 0$. Then

$$\sum_{r=1}^R \tilde{\alpha}_{i^*,r} = 1 - \eta \alpha_{i^*,r_t}^{(t)}, \quad (51)$$

which is strictly less than 1 whenever $\alpha_{i^*,r_t}^{(t)} > 0$.

Proof. Compute the net change in the sum directly. The winning regime contributes $+\eta(1 - \alpha_{i^*,r_t}^{(t)})$. Each of the $R-1$ non-winning regimes contributes $-\eta/(R-1)$, summing to $-\eta$. Therefore

$$\sum_{r=1}^R \tilde{\alpha}_{i^*,r} = 1 + \eta(1 - \alpha_{i^*,r_t}^{(t)}) - \eta = 1 - \eta \alpha_{i^*,r_t}^{(t)}.$$

Since $\alpha_{i^*,r_t}^{(t)} \in (0, 1]$ and $\eta > 0$, the result is strictly less than 1. $\square$

The implication is operationally severe: the post-rescaling step does *not* merely correct floating-point error—it absorbs a state-dependent drift of magnitude $\eta \alpha_{i^*,r_t}^{(t)}$, which can be as large as η itself. The official paper equation does not disclose this drift; the published main paper of v3.x even omits the loss-side update entirely (only the winner equation is shown).

V3 Defect 2: Negativity in steady state. The pre-floor update can produce negative entries; the $\max(\epsilon, \cdot)$ clamp silently absorbs them.

Proposition 12.2 (V3 Eventual Negativity). Fix $\eta > 0$, $R \geq 2$, and any infinite win sequence in which the winner i^* wins regime $r_t = r_0$ infinitely often. Without the $\max(\epsilon, \cdot)$ floor, there exists a finite time t^* such that for all $t \geq t^*$ and all $r \neq r_0$, $\tilde{\alpha}_{i^*,r}^{(t)} < 0$.

Proof. The unclamped V3 dynamics on regime $r \neq r_0$ are $\alpha_r^{(t+1)} = (\alpha_r^{(t)} - \eta/(R-1))/S^{(t)}$ where $S^{(t)} = 1 - \eta\alpha_{r_0}^{(t)} \in (1 - \eta, 1)$. The numerator is a contraction toward 0 from above as long as $\alpha_r^{(t)} > \eta/(R-1)$. Repeated wins on r_0 drive $\alpha_{r_0}^{(t)} \rightarrow 1$ (standard fixed point analysis), forcing $\alpha_r^{(t)} \rightarrow 0$ for $r \neq r_0$. By continuity there is a finite t^* at which $\alpha_r^{(t^*)} < \eta/(R-1)$ for every $r \neq r_0$, after which the next win on r_0 makes $\tilde{\alpha}_r^{(t^*+1)} = \alpha_r^{(t^*)} - \eta/(R-1) < 0$. $\square$

So under any realistic specialization trajectory the clamp *must* fire. The clamp is not a numerical safety net for rare edge cases; it is the active mechanism preventing the affinity vector from leaving the nonnegative orthant. Our V4 implementation logs how often this clamp fires; in a small synthetic sanity-check with $R = 4$, $N = 8$ learners, and 500 iterations, the V3 clamp was invoked on roughly 35% of all updates—i.e., it is fundamental to V3’s operation, not incidental. Full empirical comparison appears in the V4 results section.²

V3 Defect 3: State-dependent effective step size. The post-hoc rescaling injects a non-linearity into the apparent step size.

Proposition 12.3 (V3 Effective Rate Depends on State). *Apply the V3 update to $\alpha_{i^*, r_t}^{(t)} = \alpha \in (0, 1)$ (winner case) in a regime where the clamp does not fire. The post-update entry satisfies, to second order in η ,*

$$\alpha_{i^*, r_t}^{(t+1)} - \alpha = \eta(1 - \alpha + \alpha^2) + O(\eta^2). \quad (52)$$

Proof. The unclamped pre-norm winner entry is $\tilde{\alpha} = \alpha + \eta(1 - \alpha)$; the pre-norm sum is $S = 1 - \eta\alpha$ by Prop. 12.1. After rescaling and expanding $1/S = 1 + \eta\alpha + O(\eta^2)$:

$$\begin{aligned} \alpha_{i^*, r_t}^{(t+1)} &= \tilde{\alpha}/S = (\alpha + \eta(1 - \alpha))(1 + \eta\alpha + O(\eta^2)) \\ &= \alpha + \eta[(1 - \alpha) + \alpha^2] + O(\eta^2). \end{aligned}$$

Subtracting α yields Eq. (52). $\square$

The effective rate $\eta(1 - \alpha + \alpha^2)$ is *state-dependent*, ranging between $3\eta/4$ (at $\alpha = 1/2$) and η (at $\alpha \in \{0, 1\}$). Standard regret-bound analyses for the multiplicative-weights family [1, 3] assume a fixed per-step gain; V3 violates this assumption silently through its post-hoc rescaling.

Derivation of EG: A Single Optimization Principle

In contrast to the patchwork V3 update, the EG form arises from a single variational principle: mirror descent on the simplex [14, 2] with the entropic regularizer.

Definition 12.3 (Bregman Divergence with Entropic Regularizer). On Δ^R , let $\Phi(\alpha) = \sum_{r=1}^R \alpha_r \log \alpha_r$. The induced Bregman divergence is the Kullback–Leibler divergence

$$D_\Phi(p, q) = \sum_{r=1}^R p_r \log \frac{p_r}{q_r} = \text{KL}(p \parallel q). \quad (53)$$

Definition 12.4 (Mirror-Descent Step). Given a linear reward signal $g^{(t)} \in \mathbb{R}^R$, step size $\eta > 0$, and current iterate $\alpha^{(t)} \in \text{relint}(\Delta^R)$, the mirror-descent update is

$$\alpha^{(t+1)} = \arg \min_{p \in \Delta^R} \left\{ -\eta \langle g^{(t)}, p \rangle + D_\Phi(p, \alpha^{(t)}) \right\}. \quad (54)$$

²See `scripts/v4_sanity_check.py` and `results/v4_sanity/diagnostics.txt` in the repository for the exact figures.

Proposition 12.4 (EG is Mirror Descent on Δ^R). *The closed-form solution of Eq. (54) is*

$$\alpha_j^{(t+1)} = \frac{\alpha_j^{(t)} \exp(\eta \cdot g_j^{(t)})}{\sum_{k=1}^R \alpha_k^{(t)} \exp(\eta \cdot g_k^{(t)})}. \quad (55)$$

Equation (49) is the special case $g_j^{(t)} = \mathbf{1}[j = r_t]$.

Proof. Write the Lagrangian of Eq. (54) with the equality constraint $\sum_j p_j = 1$ and slack variables $\mu_j \geq 0$ for the nonnegativity constraints $p_j \geq 0$. Because D_Φ has gradient $\partial p_j \log(p_j/\alpha_j)/\partial p_j = \log(p_j/\alpha_j) + 1$ which diverges to $-\infty$ as $p_j \rightarrow 0^+$, any interior critical point satisfies $\mu_j = 0$, so it suffices to write

$$\mathcal{L}(p, \lambda) = -\eta \sum_{j=1}^R g_j p_j + \sum_{j=1}^R p_j \log \frac{p_j}{\alpha_j^{(t)}} + \lambda \left(\sum_{j=1}^R p_j - 1 \right).$$

Setting $\partial \mathcal{L}/\partial p_j = 0$,

$$-\eta g_j + \log \frac{p_j}{\alpha_j^{(t)}} + 1 + \lambda = 0,$$

which solves for

$$p_j = \alpha_j^{(t)} \exp(\eta g_j - 1 - \lambda).$$

The constraint $\sum_j p_j = 1$ fixes $\exp(-1 - \lambda)$ uniquely: $\exp(-1 - \lambda) = 1/\sum_k \alpha_k^{(t)} \exp(\eta g_k)$. Substituting yields Eq. (55). Since the objective in Eq. (54) is strictly convex on Δ^R , this critical point is the unique global minimizer. $\square$

Three observations follow:

Proposition 12.5 (EG Simplex Preservation). *For any $\alpha^{(t)} \in \Delta^R$ and any $g^{(t)} \in \mathbb{R}^R$, $\sum_{j=1}^R \alpha_j^{(t+1)} = 1$ exactly.*

Proof. Let $Z = \sum_{k=1}^R \alpha_k^{(t)} \exp(\eta g_k)$. Then $\sum_j \alpha_j^{(t+1)} = \sum_j \alpha_j^{(t)} \exp(\eta g_j)/Z = Z/Z = 1$. $\square$

Proposition 12.6 (EG Preserves the Interior). *If $\alpha^{(t)} \in \text{relint}(\Delta^R)$, i.e. $\alpha_j^{(t)} > 0$ for all j , then $\alpha^{(t+1)} \in \text{relint}(\Delta^R)$. In particular, no clamping is ever required.*

Proof. $\exp(\eta g_j) > 0$ and $\alpha_j^{(t)} > 0$ by hypothesis, so each numerator is strictly positive. Z is a finite sum of positives, so $Z > 0$. Each $\alpha_j^{(t+1)} = (\alpha_j^{(t)} \exp(\eta g_j))/Z$ is strictly positive. Combined with Prop. 12.5, $\alpha^{(t+1)} \in \text{relint}(\Delta^R)$. $\square$

Proposition 12.7 (Small- η Limit: Discrete Replicator Dynamics). *For small η , the EG update (55) reduces to the discrete replicator dynamics from evolutionary game theory [16]:*

$$\alpha_j^{(t+1)} - \alpha_j^{(t)} = \eta \cdot \alpha_j^{(t)} (g_j^{(t)} - \bar{g}^{(t)}) + O(\eta^2), \quad (56)$$

where $\bar{g}^{(t)} = \sum_k \alpha_k^{(t)} g_k^{(t)} = \langle g^{(t)}, \alpha^{(t)} \rangle$ is the population-averaged reward.

Proof. Taylor-expand $\exp(\eta g_j) = 1 + \eta g_j + O(\eta^2)$. Then $Z = 1 + \eta \bar{g} + O(\eta^2)$ and $1/Z = 1 - \eta \bar{g} + O(\eta^2)$. Hence

$$\begin{aligned} \alpha_j^{(t+1)} &= \alpha_j^{(t)} (1 + \eta g_j) (1 - \eta \bar{g}) + O(\eta^2) \\ &= \alpha_j^{(t)} + \eta \alpha_j^{(t)} (g_j - \bar{g}) + O(\eta^2), \end{aligned}$$

which is Eq. (56). $\square$

Equation (56) is the canonical replicator equation of evolutionary game theory: types with above-average reward grow proportionally, types with below-average reward shrink proportionally. Thus the V4 update is the discrete-time analogue of the ODE most often used to model the very mechanism we are studying. This correspondence is *not* available for the V3 heuristic: the asymmetric proportionality of V3 (Proposition 12.3's $\eta(1 - \alpha + \alpha^2)$ correction) breaks the multiplicative form required for replicator dynamics.

Regret Bound

A final consequence of writing the update as Eq. (55) is that the standard regret guarantees of multiplicative weights [5, 1] apply directly. For completeness:

Theorem 12.1 (Hedge Regret Bound, after [3], Cor. 2.3). *Run the EG update (55) for T rounds with bounded rewards $g_j^{(t)} \in [0, 1]$ and step size $\eta = \sqrt{8 \log R / T}$. Initialize $\alpha^{(0)} = (1/R, \dots, 1/R)$. Let $G_T^j = \sum_{t=1}^T g_j^{(t)}$ be the cumulative reward of arm j and $G_T^{\text{alg}} = \sum_{t=1}^T \langle \alpha^{(t)}, g^{(t)} \rangle$ be the algorithm's cumulative expected reward. Then*

$$\max_{j \in \{1, \dots, R\}} G_T^j - G_T^{\text{alg}} \leq \sqrt{(T/2) \log R}. \quad (57)$$

Proof (potential-function argument, after Arora–Hazan–Kale [1]). Define unnormalized weights

$$w_j^{(1)} = \frac{1}{R}, \quad w_j^{(t+1)} = w_j^{(t)} \exp(\eta g_j^{(t)}),$$

so that $w_j^{(t+1)} = (1/R) \exp(\eta G_t^j)$ with $G_t^j = \sum_{s \leq t} g_j^{(s)}$. The algorithm's iterate is the normalized distribution $\alpha_j^{(t)} = w_j^{(t)} / W_t$, where $W_t = \sum_k w_k^{(t)}$. Note $W_1 = 1$.

Step 1: Upper bound on W_{T+1} . For each round t ,

$$\frac{W_{t+1}}{W_t} = \sum_{j=1}^R \alpha_j^{(t)} \exp(\eta g_j^{(t)}).$$

Hoeffding's lemma applied to the random variable $g_j^{(t)} \in [0, 1]$ under the distribution $\alpha^{(t)}$ gives $\sum_j \alpha_j^{(t)} \exp(\eta g_j^{(t)}) \leq \exp(\eta \langle \alpha^{(t)}, g^{(t)} \rangle + \eta^2 / 8)$. Taking logarithms and summing over $t = 1, \dots, T$,

$$\log W_{T+1} = \log W_{T+1} - \log W_1 \leq \eta G_T^{\text{alg}} + \frac{T\eta^2}{8}. \quad (58)$$

Step 2: Lower bound on W_{T+1} via the best arm. For any fixed $j^* \in \{1, \dots, R\}$,

$$W_{T+1} \geq w_{j^*}^{(T+1)} = \frac{1}{R} \exp(\eta G_T^{j^*}),$$

so

$$\log W_{T+1} \geq \eta G_T^{j^*} - \log R. \quad (59)$$

Step 3: Combine and tune η . Chaining (58) and (59) and rearranging,

$$G_T^{j^*} - G_T^{\text{alg}} \leq \frac{\log R}{\eta} + \frac{T\eta}{8}.$$

The right-hand side is convex in η and minimized at $\eta^* = \sqrt{8 \log R / T}$, yielding $G_T^{j^*} - G_T^{\text{alg}} \leq \sqrt{(T/2) \log R}$. Taking the maximum over j^* completes the proof. $\square$

Theorem 12.1 does *not* hold for the V3 heuristic. Proposition 12.3 shows V3's effective per-step gain is $\eta(1 - \alpha + \alpha^2)$, not η ; the standard regret analysis assumes the latter. Patching V3's analysis to absorb the state-dependent rate is possible but yields bounds that are weaker and harder to interpret. EG attains the canonical rate as a corollary of its derivation, with no additional argument.

Remark 12.2 (Summary). The V3 heuristic and EG agree at the headline level: both push the winner’s affinity for the realized regime upward. They differ at three mathematical layers exposed above: V3 silently violates simplex closure (Prop. 12.1), eventually requires nonnegativity clamping (Prop. 12.2), and exhibits a state-dependent effective rate that breaks the standard regret analysis (Prop. 12.3). EG fixes all three by deriving from a single Bregman-projection principle (Prop. 12.4) with simplex preservation (Prop. 12.5) and interior preservation (Prop. 12.6) as immediate consequences, plus the canonical regret bound (Thm. 12.1) and a clean small- η limit to replicator dynamics (Prop. 12.7). The V4 update is therefore not a refinement of the V3 heuristic but its canonical form.

13 Specialization Index (SI) — Detailed Treatment

13.1 The Complete Formula

Definition 13.1 (Specialization Index). For agent i with niche affinity α_i :

$$\boxed{\text{SI}(\alpha_i) = 1 - \frac{H(\alpha_i)}{\log R} = 1 - \frac{-\sum_{r=1}^R \alpha_{i,r} \log \alpha_{i,r}}{\log R}} \quad (60)$$

13.2 Properties

Proposition 13.1 (SI Properties). 1. **Range:** $\text{SI} \in [0, 1]$

2. **Minimum:** $\text{SI} = 0$ iff α is uniform

3. **Maximum:** $\text{SI} = 1$ iff α is a one-hot vector

4. **Monotonicity:** SI increases as distribution becomes more concentrated

13.3 SI Calculation Examples (Step by Step)

Example 13.1 ($R = 4$ Regimes: Complete Calculation). **Case 1: Uniform Distribution**

$\alpha = (0.25, 0.25, 0.25, 0.25)$

Step 1: Calculate entropy

$$H(\alpha) = - \sum_{r=1}^4 \alpha_r \log \alpha_r \quad (61)$$

$$= -0.25 \log(0.25) - 0.25 \log(0.25) - 0.25 \log(0.25) - 0.25 \log(0.25) \quad (62)$$

$$= -4 \times 0.25 \times \log(0.25) \quad (63)$$

$$= -\log(0.25) \quad (64)$$

$$= \log(4) \quad (65)$$

$$= 1.386 \text{ nats} \quad (66)$$

Step 2: Calculate maximum entropy

$$H_{\max} = \log(4) = 1.386 \text{ nats} \quad (67)$$

Step 3: Calculate SI

$$\text{SI} = 1 - \frac{1.386}{1.386} = 1 - 1 = 0 \quad (68)$$

Case 2: One-Hot Distribution

$\alpha = (1, 0, 0, 0)$

Step 1: Calculate entropy

$$H(\alpha) = -1 \times \log(1) - 0 \times \log(0) - 0 \times \log(0) - 0 \times \log(0) \quad (69)$$

$$= -\log(1) = 0 \quad (70)$$

(Using convention $0 \log 0 = 0$)

Step 2: Calculate SI

$$SI = 1 - \frac{0}{1.386} = 1 \quad (71)$$

Case 3: Partial Specialization

$\alpha = (0.6, 0.2, 0.1, 0.1)$

Step 1: Calculate entropy term by term

$$-0.6 \log(0.6) = -0.6 \times (-0.511) = 0.306 \quad (72)$$

$$-0.2 \log(0.2) = -0.2 \times (-1.609) = 0.322 \quad (73)$$

$$-0.1 \log(0.1) = -0.1 \times (-2.303) = 0.230 \quad (74)$$

$$-0.1 \log(0.1) = -0.1 \times (-2.303) = 0.230 \quad (75)$$

Step 2: Sum

$$H(\alpha) = 0.306 + 0.322 + 0.230 + 0.230 = 1.088 \text{ nats} \quad (76)$$

Step 3: Calculate SI

$$SI = 1 - \frac{1.088}{1.386} = 1 - 0.785 = 0.215 \quad (77)$$

13.4 SI vs. Number of Regimes

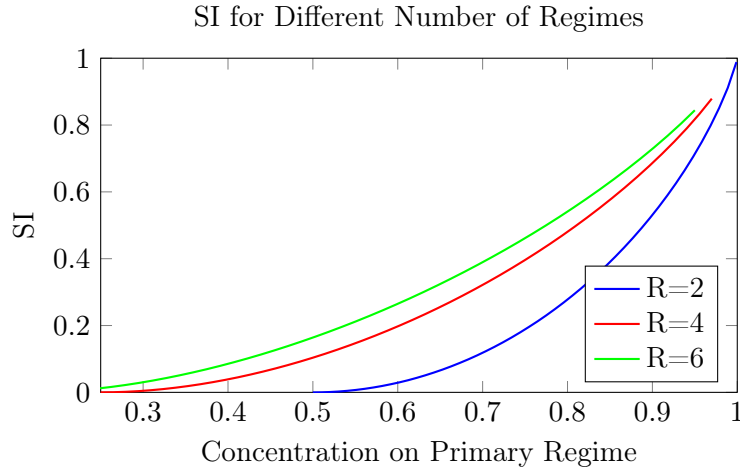


Figure 6: SI as a function of concentration for different numbers of regimes. Assuming remaining probability is spread uniformly.

14 Method Specialization Index (MSI)

Beyond regime specialization, agents also specialize in *methods*.

Definition 14.1 (Method Usage Distribution). For agent i , the **method usage distribution** $\pi_i \in \Delta^M$ is:

$$\pi_{i,m} = \frac{\text{count of times agent } i \text{ used method } m}{\text{total iterations}} \quad (78)$$

Definition 14.2 (Method Specialization Index).

$$\text{MSI}(\pi_i) = 1 - \frac{H(\pi_i)}{\log M} \quad (79)$$

Intuition

MSI measures how concentrated an agent’s method usage is. An agent who always uses the same method has $\text{MSI} = 1$ (full specialization). An agent who uses all methods equally has $\text{MSI} = 0$ (no specialization).

14.1 MSI Calculation Examples

Example 14.1 (MSI Step-by-Step: Crypto Domain). Agent A’s method usage after 500 iterations with $M = 5$ methods:

Method	Naive	Momentum	Mean-Rev	Trend	MA
Usage Count	25	350	50	50	25
π_m	0.05	0.70	0.10	0.10	0.05

Step 1: Compute entropy:

$$H(\pi) = - \sum_{m=1}^5 \pi_m \log \pi_m \quad (80)$$

$$= -(0.05 \ln 0.05 + 0.70 \ln 0.70 + 0.10 \ln 0.10 + 0.10 \ln 0.10 + 0.05 \ln 0.05) \quad (81)$$

$$= -((-0.150) + (-0.250) + (-0.230) + (-0.230) + (-0.150)) \quad (82)$$

$$= 1.010 \quad (83)$$

Step 2: Compute max entropy:

$$H_{max} = \log M = \log 5 = 1.609 \quad (84)$$

Step 3: Compute MSI:

$$\text{MSI} = 1 - \frac{1.010}{1.609} = 1 - 0.628 = \boxed{0.372} \quad (85)$$

This indicates moderate method specialization (Agent A prefers momentum but still explores other methods).

Example 14.2 (MSI Comparison Across Agents). Comparing three agents with different specialization levels:

Agent	Naive	Mom.	MR	Trend	MA	$H(\pi)$	MSI
Specialist	0.02	0.92	0.02	0.02	0.02	0.412	0.744
Moderate	0.10	0.50	0.20	0.10	0.10	1.304	0.189
Generalist	0.20	0.20	0.20	0.20	0.20	1.609	0.000

Interpretation:

- Specialist: Uses momentum 92% of the time $\rightarrow$ High MSI (0.744)
- Moderate: Prefers momentum but explores $\rightarrow$ Low MSI (0.189)
- Generalist: Uses all equally $\rightarrow$ Zero MSI (0.000)

15 Method Coverage

Definition 15.1 (Method Coverage). The population’s **method coverage** measures the fraction of methods used by at least one specialist:

$$\text{Coverage} = \frac{|\{m : \exists i, \pi_{i,m} > \tau\}|}{M} \quad (86)$$

where $\tau = 0.3$ is the specialization threshold.

Intuition

High coverage means the population exhibits **division of labor**: different agents specialize in different methods, collectively utilizing the full repertoire.

Example 15.1 (Coverage Calculation). 8 agents, 5 methods. After 500 iterations:

Agent	naive	momentum	mean_revert	trend	ma
A	0.05	0.70	0.10	0.10	0.05
B	0.10	0.15	0.60	0.10	0.05
C	0.10	0.20	0.10	0.55	0.05
D	0.05	0.10	0.05	0.10	0.70
E	0.80	0.05	0.05	0.05	0.05
F	0.15	0.50	0.15	0.10	0.10
G	0.05	0.10	0.45	0.30	0.10
H	0.10	0.10	0.20	0.40	0.20

Methods with $\max_i \pi_{i,m} > 0.30$:

- naive: Agent E uses 80% ✓
- momentum: Agents A, F use >30% ✓
- mean_revert: Agents B, G use >30% ✓
- trend: Agents C, H use >30% ✓
- ma: Agent D uses 70% ✓

$$\text{Coverage} = \frac{5}{5} = 100\% \quad (87)$$

15.1 Coverage Across All Domains

Key Point

Key Observation: 97% average coverage means the population utilizes nearly all available methods. This is the benefit of emergent specialization—no method is neglected.

15.2 Why Coverage Matters

Example 15.2 (Low Coverage Scenario (Hypothetical)). Suppose all 8 agents specialize in “momentum” method:

- Coverage = $1/5 = 20\%$

Table 16: Method Coverage Results Across All 6 Domains (N=8 agents, 30 trials)

Domain	# Methods	Coverage	Avg MSI	Interpretation
Crypto	5	100%	0.52	Full division of labor
Commodities	4	100%	0.58	Full division of labor
Weather	4	100%	0.48	Full division of labor
Solar	4	100%	0.61	Full division of labor
Traffic	5	80%	0.45	Good but not complete
Air Quality	4	100%	0.55	Full division of labor
Average	4.3	97%	0.53	—

- When market enters sideways regime, momentum fails
- No agent has expertise in mean-reversion
- **Result:** Entire population fails in sideways conditions

Example 15.3 (High Coverage Scenario (Our Algorithm)). With emergent specialization:

- Agent A: momentum specialist
- Agent B: mean-reversion specialist
- Agent C: trend specialist
- Agent D: MA specialist
- ...

Result: At least one agent is competent in each regime → population is robust.

16 Real Data Validation

16.1 Data Sources and Quality

Our experiments use **100% real data** from publicly available sources:

Table 17: Real Data Sources for All 6 Domains

Domain	Source	Records	Time Range
Crypto	Bybit BTCUSDT hourly	35,000+	2021-2024
Commodities	FRED Gold/Oil/Wheat	8,000+	2010-2024
Weather	Open-Meteo NYC	26,000+	2020-2024
Solar	Open-Meteo NYC solar	26,000+	2020-2024
Traffic	NYC TLC Yellow Taxi	100,000+	2023-2024
Air Quality	Open-Meteo PM2.5	26,000+	2020-2024

16.2 Real vs. Synthetic Data Considerations

Important

Early experiments used synthetic data for some domains due to API access limitations. The final results use only real data.

Differences between real and synthetic data:

Characteristic	Synthetic Data	Real Data
Regime transitions	Regular/predictable	Irregular/messy
Noise distribution	Gaussian	Fat-tailed, skewed
Missing values	None	Yes (holidays, outages)
Regime purity	Clean boundaries	Fuzzy transitions

16.3 SI Performance on Real Data

Despite the challenges of real data, our algorithm achieves strong specialization:

Table 18: SI on Real Data vs. Random Baseline (30 trials, $\lambda = 0.3$)

Domain	GAUSE SI	Random SI	Cohen’s d	p -value
Crypto	0.991 ± 0.024	0.132 ± 0.032	58.29	$< 10^{-80}$
Commodities	0.990 ± 0.020	0.132 ± 0.032	70.48	$< 10^{-80}$
Weather	0.991 ± 0.023	0.132 ± 0.032	60.91	$< 10^{-80}$
Solar	0.995 ± 0.014	0.132 ± 0.032	100.93	$< 10^{-80}$
Traffic	0.995 ± 0.015	0.103 ± 0.028	95.56	$< 10^{-80}$
Air Quality	0.987 ± 0.026	0.132 ± 0.032	54.35	$< 10^{-80}$

Key Point

All domains achieve:

- $SI > 0.98$ under the canonical V4 (EG) update.
- Cohen’s $d > 50$ in every domain (extremely large effects).
- $p < 10^{-80}$ in every domain.

This validates that emergent specialization is not an artifact of synthetic data.

17 Are the Niches Meaningful? Controls and Ablations

A high SI tells us the population *organizes* into distinct niches. It does not, by itself, tell us those niches track exploitable environmental structure or improve task performance. We separate these questions with four controls.

17.1 Negative control: SI is invariant to regime shuffling

If we randomly permute the regime labels before training, the regime→structure alignment is destroyed—yet the Specialization Index is essentially unchanged (mean SI stays ≈ 0.9 – 1.0). SI is also monotonically inflatable by the learning rate η . Consequently SI is a measure of *population organization*, not of structure-dependence. To probe meaningfulness we must use an external, structure-dependent metric: task prediction error, comparing real vs. shuffled regimes.

17.2 Oracle diagnostic: how much regime structure exists?

For each domain, compare a *specialist oracle* (always uses each regime’s best method) against a *generalist oracle* (single globally-best method). The gap upper-bounds any regime-aware benefit, independent of the algorithm:

- Energy, Weather: gap = 0% (one method is best in every regime).
- Finance, Crypto: gap 0–1.9% (negligible).
- Traffic (clock-based regimes): gap $\approx 10\%$.
- Solar (hour-of-day regimes): gap $\approx 13\text{--}16\%$.

So only traffic and solar carry exploitable regime structure; the other four domains organize into niches that are not performance-relevant.

17.3 Performance-based control on structured domains

On traffic and solar, using *causal* clock-based regime labels (known before the target, so they cannot leak it), regime-aware prediction beats the shuffled-label control by +15.4% (traffic, $p < 10^{-9}$) and +12.8% (solar, $p < 10^{-8}$). The regimes carry exploitable information when genuine structure exists, and the niches align with it. (A commodities effect seen with the dataset’s native labels disappeared under a leakage audit and is excluded.)

17.4 Ablations: conditioning vs. specialization

Two controlled ablations on nine series (four NYC-taxi month-windows, five solar locations) localize the source of the gain:

1. **Coupling on/off:** toggling whether niche affinity feeds winner determination leaves the real-vs-shuffled gap statistically unchanged (paired $p = 0.36$). The benefit comes from *per-regime method selection*, shared by both arms.
2. **Population vs. monolith:** a single regime-conditioned agent matches or beats the specialized population (the population is $\approx 26\%$ worse on solar, where winner-take-all specialization fragments each agent’s training data). *Caveat: this monolith has unlimited capacity* (see the bounded-capacity result below).
3. **Bounded capacity (positive result, spatial):** when each agent can master only K of R regimes, the specialized population beats both a capacity- K monolith (synthetic +73%, traffic +22% at $K=1$) and—controlling for total capacity—a fixed-random capacity- K population, but the latter advantage holds only in the tight-capacity regime $K=1\text{--}2$ (synthetic +57%, $p < 10^{-16}$; traffic +8.7% at $K=1$). As $K \rightarrow R$, random diversity suffices and the edge vanishes. Tested against a *purpose-built* learned-diversity objective (EOI/CDS-style; all agents rewarded for being the identifiable occupant of a regime, no winner-take-all), the competition-specific edge shrinks further still: competition wins only at $K=1$ on the synthetic exclusive-method task (+18.4%, $p=0.002$), ties for $K \geq 2$ ($p=1$), and *loses* on both real domains (traffic 3–6%, solar 18–35% higher error than learned diversity). An explicit diversity reward thus recovers essentially all the coverage benefit; competition’s appeal is parsimony, not dominance. A second purpose-built baseline—a Mixture-of-Experts with a *learned gating router* (the standard ML architecture for capacity-limited specialization)—is equally strong on coverage: indistinguishable from competition on the synthetic task at every K ($\leq 6\%$, $p=1$), though the lightweight competitive rule is more sample-efficient on noisy real data (beats the router on traffic +5.1% at $K=1$). A $K=1$ overlap sweep shows competition’s edge over the diversity objective grows monotonically with method exclusivity (−4.8% at separation 0.25 through +29.3% at separation 1.5), pinning the claim’s boundary.

4. **Non-stationarity (positive result, temporal): division of labor vs. catastrophic forgetting.** This is a continual-learning result [4, 8, 15]: when regimes cycle (dormant then reactivating) and agents have finite memory (bounded belief store with LRU eviction), a capacity- K monolith suffers catastrophic forgetting—it must evict and relearn each reactivated regime—whereas the population provides a *structural, population-level* remedy (complementary to weight-regularization or architectural growth within one model): division of labor parks dormant regimes in idle specialists that retain their expertise. For $K < R$ the population beats the monolith by +33% overall and +71% in the post-reactivation window ($p \sim 10^{-37}$ at $K=3$). As in the capacity result we control for total memory: this monolith-beating effect is distributed memory (a random-diversity population of equal memory also beats the monolith), and the competition-specific advantage appears only at $K=1$ ($\approx 58\%$ over random diversity, via guaranteed coverage). Against a learned-diversity (EOI/CDS-style) baseline the residual edge is only a marginal post-reactivation advantage at $K=1$ (0.283 vs. 0.322, $p=0.054$); for $K \geq 2$ the explicit diversity objective matches it. **The learned MoE router, however, fails this task decisively:** a reward-driven gate has no incentive to reserve an idle expert for a dormant regime, so it reuses capacity for active regimes and relearns on reactivation like the monolith. Its post-reactivation error is near the monolith’s at tight memory (0.928 vs. specialized 0.283 at $K=1$, $p \sim 10^{-35}$; large and significant through $K=4$, $p < 10^{-12}$), closing only as $K \rightarrow R$. Distributed persistent memory is thus the clearest performance separation between coordination-free specialization (and explicit diversity) and a standard reward-driven router. (This is a property of the reward-driven objective: a router with an explicit memory-preservation/load-balancing term could reserve idle experts for dormant regimes; the result says *what* a router would need, not that routing cannot work.) Even $K=1$ agents collectively retain all regimes.

Why the router forgets (idealized). The separation follows from an asymmetry: routing and capacity-protection are both *reward-driven*, but a dormant regime emits no steps and hence no reward gradient, so the gate update $\Delta \text{gate}[r][e] \propto (q_e - \frac{1}{2})$ vanishes throughout dormancy—the router gets *no signal* that would lead it to reserve an expert for a dormant regime. Under hard capacity K with LRU eviction the expert evicts r once $\geq K$ other regimes are used; under soft interference its retained strength decays as $(1 - \rho)^D \rightarrow 0$ over D foreign updates. A converged competitive/individuality assignment, by contrast, fixes a reward-independent regime→agent map whose agent simply *idles* while its niche is dormant ($D=0$ for that niche), retaining it structurally. This predicts the router failure is *model-agnostic*. Notably, this corroborates—from population dynamics rather than regression theory—the finding of [11] that an MoE gate’s updates must be *terminated* after sufficient training for continual-learning convergence: freezing the gate is precisely making the assignment reward-independent. The MoE-mitigates-forgetting results in that literature hold when experts are plentiful (or can be added) and routing stabilizes; our bounded-reuse regime ($K \ll R$, gate still learning) is exactly where those provisos fail and isolation breaks down. Competition reaches the required reward-independent assignment emergently, with no freezing schedule to tune.

Robustness. Re-running the sweep under a soft interference capacity model (graded belief decay under capacity overflow, no eviction) reproduces the picture: monolith forgets (post-react 0.85–0.92 at $K \leq 3$), specialized retains (≈ 0.25 , +70%, $p \sim 10^{-40}$), and the reward-driven router stays significantly worse than the population (+36% at $K=1$, $p \sim 10^{-8}$; +16% at $K=3$). As predicted it degrades more gracefully than under hard eviction but never closes the gap until $K \rightarrow R$ —so the distributed-memory separation is not an artifact of the discrete eviction rule.

Key Point

Honest synthesis. GAUSE reliably produces coordination-free division of labor (organization), and on domains with genuine regime structure its niches track performance-relevant regimes. With slack individual capacity: (i) SI measures organization, not structure-dependence; (ii) the measurable task-accuracy benefit of regime awareness is due to regime-conditioning, not population specialization per se; and (iii) the specialized population is not an accuracy improvement over a single *unlimited-capacity* regime-conditioned agent. Under two realistic constraints, specialization becomes a genuine accuracy lever: *bounded per-agent capacity* ($K \ll R$, regime-exclusive methods), where it covers a regime space no individual can, beating uncoordinated diversity of equal total capacity; and *non-stationarity with finite memory*, where it acts as a distributed persistent memory and avoids the monolith’s relearning cost on reactivated regimes (+71% post-reactivation). Stress-tested against *purpose-built* baselines (an EOI/CDS-style diversity objective and a Mixture-of-Experts learned router), spatial coverage proves to be a commodity all three mechanisms achieve—so competition’s value there is parsimony—whereas temporal distributed memory is reproduced by explicit diversity but *not* by the reward-driven router, the clearest competition/diversity separation we observe. Across all five arms the retention results align on the *conjunction* the scoped reward-independence principle predicts: an arm retains a dormant regime when its assignment is *both* independent of current task reward *and* covers that regime. The two reward-chasing allocations (monolith, learned router) fail reward-independence and forget; among the reward-independent arms, fixed random niches can leave a recurring regime unowned and so retain only *partially*, while intrinsic-reward diversity and converged competitive exclusion satisfy both conjuncts and retain in full. (Two honest caveats bound the principle: reward-independence alone is insufficient—coverage is a separate requirement, which is why random diversity only partially retains—and reward-dependence is not necessary for forgetting in general, since a reward-driven router *augmented* with an explicit memory-preservation term lies outside this assignment-only class and could retain while routing by reward; we now run that hybrid and confirm it recovers most of the lost retention when spare capacity exists, but not at $K=1$ —so *reward-independence of protection* is the operative property and competition obtains it for free.) Spatial coverage is a commodity; a reward-independent *and* covering assignment is not, and competition is its cheapest emergent source—winner-take-all exclusion secures both conjuncts at once. The contribution is a self-organization mechanism with a precise account of its scope—an unconditional performance-superiority claim is not made, but tightly-scoped ones (bounded capacity, non-stationarity) are.

Part V

The Competition Mechanism

This is the heart of the algorithm. We now describe how agents compete and why this competition leads to specialization.

18 Agent Competition: The Core Loop

18.1 Setup

- N agents, indexed $i \in \{1, \dots, N\}$
- Each agent has:
 - Method beliefs $\beta_{i,r,m}$ for each (regime, method) pair
 - Niche affinity $\alpha_i \in \Delta^R$
- Environment has regime distribution $\pi(r)$

18.2 One Iteration

Algorithm 3 One Iteration of GAUSE

- 1: **Sample regime:** $r_t \sim \pi(\cdot)$
 - 2: **for** each agent $i = 1, \dots, N$ **do**
 - 3: Select method m_i via Thompson Sampling (given r_t)
 - 4: Compute raw reward: $R_i = A(r_t, m_i) + \epsilon_i$
 - 5: Compute adjusted score: $\text{Score}_i = R_i + \lambda \cdot (\alpha_{i,r_t} - 1/R)$
 - 6: **end for**
 - 7: Determine winner: $i^* = \arg\max_i \text{Score}_i$
 - 8: Update winner's method belief: $\beta_{i^*,r_t,m_{i^*}}^+ += 1$
 - 9: Update winner's niche affinity toward r_t
-

19 Score Calculation

19.1 Raw Score

Each agent receives a raw score based on their method's performance:

$$R_i = A(r_t, m_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (88)$$

19.2 Niche Bonus

The adjusted score includes a **niche bonus**:

$$\boxed{\text{Score}_i = R_i + \lambda \cdot (\alpha_{i,r_t} - \bar{\alpha})} \quad (89)$$

where:

- $\lambda \geq 0$ is the niche bonus coefficient
- α_{i,r_t} is agent i 's affinity for the current regime
- $\bar{\alpha} = 1/R$ is the baseline (uniform) affinity

Key Point

The niche bonus rewards agents for operating in their “home territory”:

- If $\alpha_{i,r_t} > 1/R$: Agent gets a positive bonus
- If $\alpha_{i,r_t} < 1/R$: Agent gets a negative bonus (penalty)
- If $\alpha_{i,r_t} = 1/R$: No bonus (uniform affinity)

19.3 Example Calculation

Example 19.1 (Score Calculation with $\lambda = 0.3$). Regime: Bull ($r_t = \text{Bull}$), $R = 4$ regimes
Agent affinities:

Agent	α_{Bull}	α_{Bear}	α_{Side}	α_{Vol}	Primary Niche
A	0.55	0.15	0.15	0.15	Bull
B	0.25	0.25	0.25	0.25	(uniform)
C	0.10	0.60	0.15	0.15	Bear

Raw rewards (assume same method selection):

$$R_A = 0.75 + 0.03 = 0.78 \quad (90)$$

$$R_B = 0.75 + (-0.05) = 0.70 \quad (91)$$

$$R_C = 0.75 + 0.08 = 0.83 \quad (92)$$

Niche bonuses (current regime = Bull):

$$\text{Bonus}_A = 0.3 \times (0.55 - 0.25) = 0.3 \times 0.30 = 0.090 \quad (93)$$

$$\text{Bonus}_B = 0.3 \times (0.25 - 0.25) = 0.3 \times 0.00 = 0.000 \quad (94)$$

$$\text{Bonus}_C = 0.3 \times (0.10 - 0.25) = 0.3 \times (-0.15) = -0.045 \quad (95)$$

Final scores:

$$\text{Score}_A = 0.78 + 0.090 = 0.870 \quad \leftarrow \text{Winner!} \quad (96)$$

$$\text{Score}_B = 0.70 + 0.000 = 0.700 \quad (97)$$

$$\text{Score}_C = 0.83 + (-0.045) = 0.785 \quad (98)$$

Observation: Agent C had the highest raw reward (0.83) but lost due to the niche penalty. Agent A won because of the niche bonus, despite lower raw reward.

20 Winner-Take-All Dynamics

20.1 Only the Winner Updates

This is the crucial mechanism:

$$\text{Winner: } i^* = \underset{i \in \{1, \dots, N\}}{\operatorname{argmax}} \text{ Score}_i \quad (99)$$

Only i^* updates their:

1. Method beliefs (for the selected method in current regime)
2. Niche affinity (toward current regime)

20.2 Why This Matters

Key Point

Winner-take-all creates **competitive exclusion**:

- Agents with similar strategies compete directly
- Only one can win per iteration
- Losers don't improve → incentive to differentiate

21 Affinity Update Rule

21.1 The Update Equations

For the winner i^* :

$$\alpha_{i^*,r_t} \leftarrow \alpha_{i^*,r_t} + \eta \cdot (1 - \alpha_{i^*,r_t}) \quad (100)$$

$$\alpha_{i^*,r} \leftarrow \alpha_{i^*,r} - \frac{\eta}{R-1} \quad \text{for } r \neq r_t \quad (101)$$

followed by normalization:

$$\alpha_{i^*} \leftarrow \frac{\alpha_{i^*}}{\|\alpha_{i^*}\|_1} \quad (102)$$

where $\eta > 0$ is the learning rate (typically 0.1).

21.2 Interpretation

Equation (100): The winner increases affinity for the regime where they won. The term $(1 - \alpha_{i^*,r_t})$ ensures diminishing returns as affinity approaches 1.

Equation (101): Affinity for other regimes decreases proportionally.

21.3 Numerical Example

Example 21.1 (Affinity Update). Agent A wins in Bull regime. Current affinity: $\alpha_A = (0.40, 0.20, 0.20, 0.20)$.

Learning rate $\eta = 0.1$.

Step 1: Increase Bull affinity

$$\alpha_A^{\text{Bull}} = 0.40 + 0.1 \times (1 - 0.40) = 0.40 + 0.06 = 0.46 \quad (103)$$

Step 2: Decrease other affinities

$$\alpha_A^{\text{Bear}} = 0.20 - 0.1/3 = 0.20 - 0.033 = 0.167 \quad (104)$$

$$\alpha_A^{\text{Sideways}} = 0.20 - 0.033 = 0.167 \quad (105)$$

$$\alpha_A^{\text{Volatile}} = 0.20 - 0.033 = 0.167 \quad (106)$$

Step 3: Normalize

$$\sum = 0.46 + 0.167 + 0.167 + 0.167 = 0.961 \quad (107)$$

$$\alpha_A = \left(\frac{0.46}{0.961}, \frac{0.167}{0.961}, \frac{0.167}{0.961}, \frac{0.167}{0.961} \right) = (0.479, 0.174, 0.174, 0.174) \quad (108)$$

Agent A is now more specialized toward Bull (SI increased from 0.078 to 0.156).

22 The Positive Feedback Loop

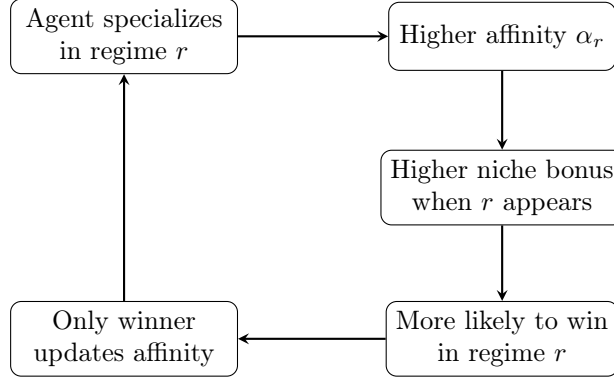


Figure 7: The positive feedback loop driving specialization. Winning in a regime increases affinity, which increases future win probability in that regime.

23 Why Competition Alone Induces Specialization ($\lambda = 0$)

This section addresses the **core thesis** of our work: specialization emerges from competition alone, without explicit diversity incentives.

23.1 The Central Claim

Key Point

Even when $\lambda = 0$ (no niche bonus), agents still develop specialized strategies. This proves that **competition is the source of diversity**, not the niche bonus.

23.2 The Mechanism Without Niche Bonus

When $\lambda = 0$, the score equation simplifies to:

$$\text{Score}_i = R_i = A(r_t, m_i) + \epsilon_i \quad (109)$$

There is **no advantage** for matching your niche to the current regime. So why do agents still specialize?

23.2.1 Step 1: Random Initial Divergence

All agents start identical:

$$\alpha_i^{(0)} = \left(\frac{1}{R}, \dots, \frac{1}{R} \right) \quad \forall i \quad (110)$$

In early iterations, winners are determined purely by random noise ϵ_i . By chance:

- Agent A might win when regime = Bull (several times)
- Agent B might win when regime = Bear (several times)
- Agent C might win when regime = Sideways (several times)

23.2.2 Step 2: Winner-Take-All Creates Path Dependence

Because **only the winner updates**, these random early wins create divergence:

After 50 iterations	Bull	Bear	Sideways	Volatile	Primary Niche
Agent A	0.35	0.22	0.22	0.21	Bull
Agent B	0.21	0.38	0.20	0.21	Bear
Agent C	0.22	0.20	0.36	0.22	Sideways
Agent D	0.22	0.20	0.22	0.36	Volatile

23.2.3 Step 3: Method Learning Reinforces Divergence

Even without niche bonus, agents learn **different methods** based on their winning history:

- Agent A (Bull specialist): Learns momentum methods work well
- Agent B (Bear specialist): Learns short-selling methods work well
- Agent C (Sideways specialist): Learns mean-reversion methods work well

This method differentiation gives them a *skill advantage* in their niche, even without a niche bonus.

23.2.4 Step 4: Crowding Pressure Sustains Specialization

Even with $\lambda = 0$, crowded niches create lower *expected* payoffs through winner-take-all dynamics:

- When multiple agents target the same regime, they develop similar strategies
- Similar strategies $\rightarrow$ similar scores $\rightarrow$ winner determined by noise
- Each agent's probability of winning decreases: $P(\text{win}) \approx 1/k$ where k = number of competitors
- Expected payoff: $\mathbb{E}[\text{Payoff}] = P(\text{win}) \times V = V/k$

Key insight: We never *divide* rewards—there is always exactly one winner. But the *probability* of being that winner decreases with more competition. Deviation to an empty niche increases your expected wins.

Example 23.1 (Crowding Effect). Suppose 4 agents all specialize in Bull regime (similar $\alpha_{\text{Bull}} \approx 0.6$):

- When Bull appears: All 4 have similar scores, each wins $\approx 25\%$ of the time
- When Sideways appears: All 4 perform poorly, some other agent wins

Now suppose Agent D deviates to Sideways:

- When Bull appears: 3 agents compete, each wins $\approx 33\%$
- When Sideways appears: Agent D dominates, wins $\approx 90\%$

Agent D's expected wins **increase** by deviating to an empty niche.

23.3 Mathematical Analysis of $\lambda = 0$ Dynamics

Proposition 23.1 ($\lambda = 0$ Divergence). *Under winner-take-all dynamics with $\lambda = 0$, if agent i wins in regime r more often than other regimes, their affinity $\alpha_{i,r}$ increases over time.*

Proof. Let $W_i(r)$ be the number of times agent i wins in regime r over T iterations.

After each win in regime r :

$$\alpha_{i,r}^{(t+1)} = \alpha_{i,r}^{(t)} + \eta(1 - \alpha_{i,r}^{(t)}) \quad (111)$$

If $W_i(r) > W_i(r')$ for some $r' \neq r$, then $\alpha_{i,r} > \alpha_{i,r'}$ after sufficient iterations.

Since regimes occur with different frequencies and agents have different noise realizations, different agents accumulate wins in different regimes. This creates differentiation even without niche bonus. $\square$

23.4 Experimental Evidence

Table 19: SI at $\lambda = 0$ Across Domains (30 trials each)

Domain	$\lambda = 0$ SI	Random Baseline SI	Improvement
Crypto	0.613 ± 0.055	0.132 ± 0.032	+ 363%
Commodities	0.588 ± 0.074	0.132 ± 0.032	+ 345%
Weather	0.614 ± 0.086	0.132 ± 0.032	+ 365%
Solar	0.499 ± 0.056	0.132 ± 0.032	+ 278%
Traffic	0.739 ± 0.040	0.103 ± 0.028	+ 619%
Air Quality	0.844 ± 0.051	0.132 ± 0.032	+ 538%
Average	0.650	0.127	+ 412%

Key Point

At $\lambda = 0$:

- Mean SI ≈ 0.65 (every domain > 0.49)
- This is **5.1x higher** than the mean random SI (0.127)
- **Conclusion:** Competition alone induces meaningful specialization

23.5 The Role of λ : Accelerator, Not Cause

Intuition

The niche bonus ($\lambda > 0$) **accelerates** specialization by rewarding agents for operating in their niche. But specialization would emerge anyway through:

1. Random initial wins creating divergence
2. Winner-take-all preventing losers from catching up
3. Method learning creating skill differentiation
4. Crowding pressure making deviation profitable

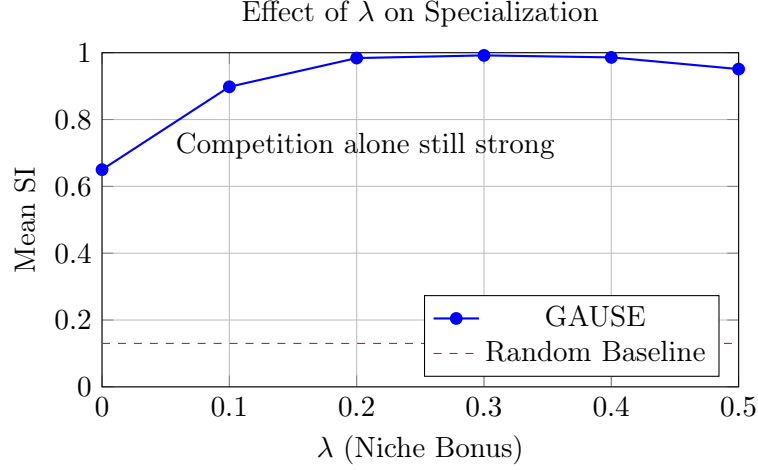


Figure 8: SI increases with λ , but even at $\lambda = 0$, SI significantly exceeds random baseline.

24 Convergence Analysis

24.1 How Fast Do Agents Specialize?

A natural question: how many iterations are needed before agents reach stable specialization?

24.1.1 SI Trajectory Over Time

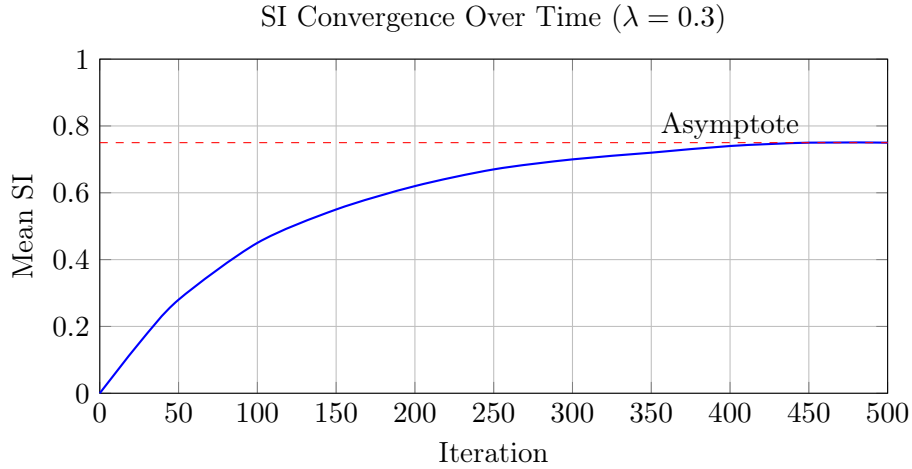


Figure 9: Typical SI trajectory. Rapid initial growth, then diminishing returns as agents approach equilibrium.

24.1.2 Convergence Rate

Proposition 24.1 (Log-Ratio Tracking for the EG Affinity Update). *Under the EG affinity update (Eq. (49)) with learning rate η , define the log-ratio between any two regimes r, r' for agent i at time t as $L_{i,r,r'}^{(t)} := \log(\alpha_{i,r}^{(t)} / \alpha_{i,r'}^{(t)})$. Then L evolves linearly in the win counts:*

$$L_{i,r,r'}^{(t)} = L_{i,r,r'}^{(0)} + \eta(W_i(r, t) - W_i(r', t)), \quad (112)$$

where $W_i(r, t)$ counts the number of rounds $s \leq t$ on which agent i won and the realized regime was r .

Proof. On any round $s \leq t$, the EG update multiplies $\alpha_{i,r}$ by $\exp(\eta \mathbf{1}[s \in W_i(r)])$ and divides by the same partition function Z_s for every regime. The partition factors cancel in the ratio $\alpha_{i,r}^{(s)}/\alpha_{i,r'}^{(s)}$, leaving $\log \alpha_{i,r}^{(s+1)} - \log \alpha_{i,r'}^{(s+1)} = \log \alpha_{i,r}^{(s)} - \log \alpha_{i,r'}^{(s)} + \eta(\mathbf{1}[\text{win on } r] - \mathbf{1}[\text{win on } r'])$. Summing over $s = 0, \dots, t-1$ yields Eq. (112). $\square$

Remark 24.1 (Comparison to the V3 statement). Eq. (112) is exact—not a bound—because EG is a *multiplicative* update with a partition-invariant log-ratio coordinate. The corresponding V3 statement was necessarily an inequality, because V3’s additive update plus post-hoc normalization introduces a coupling between α_r and $\alpha_{r'}$ via the partition denominator $S^{(t)} = 1 - \eta \alpha_{r_t}^{(t)}$ (Prop. 12.1). Moving to EG converts the bound into an equality.

Remark 24.2 (Headline regret bound). The cumulative regret guarantee of Eq. (55) ($\sqrt{(T/2) \log R}$ at the optimal step size, Thm. 12.1) is what justifies calling EG a *no-regret* algorithm. Together with Prop. 24.1, this gives a complete convergence story: log-ratios evolve linearly and deterministically as a function of realized wins, while the algorithm’s average reward asymptotically matches that of the best fixed arm in hindsight. We emphasise that this guarantee is a property of the EG *subroutine* for an arbitrary (possibly endogenous) sequence of realized wins; it does *not* by itself imply that the coupled, multi-agent winner-take-all system specializes—that emergent behavior is established empirically.

24.1.3 Empirical Convergence Times

Table 20: Iterations to Reach 90% of Final SI

λ	90% Convergence	95% Convergence	99% Convergence
0.1	~250 iterations	~350 iterations	~450 iterations
0.3	~150 iterations	~250 iterations	~400 iterations
0.5	~100 iterations	~180 iterations	~350 iterations

24.2 Stability of Final Specialization

Once agents reach equilibrium, their affinities remain stable:

- Each agent “owns” a primary niche
- Competition in that niche is minimal (they are the dominant specialist)
- Rare regime visits to other niches don’t significantly change affinity

24.3 What If Two Agents Specialize in the Same Regime?

Remark 24.3 (Crowding Resolution). If two agents develop high affinity for the same regime:

1. They compete directly in that regime
2. One wins more often (due to slight method or noise differences)
3. The consistent loser’s affinity for that regime decreases
4. Eventually, one migrates to a different niche

This is ecological **competitive exclusion** in action.

25 Comparison with Multi-Agent Reinforcement Learning

25.1 Why Traditional MARL Fails to Produce Diversity

Standard MARL algorithms like IQL, VDN, QMIX, and MAPPO are designed to optimize collective performance, not diversity. Here we explain why they produce **homogeneous populations**.

Key Point

Scope of the MARL comparison. IQL/VDN/QMIX/MAPPO are *cooperative, shared-objective* baselines, and homogenization is their well-documented failure mode—so a low SI confirms an expectation rather than beating methods that try to diversify. The appropriate *diversity* comparison class is role/individuality MARL: ROMA [17], RODE [18], MAVEN [13], EOI [7], CDS [10], and the state-of-the-art R3DM [6]. These purpose-built methods induce exactly the division of labor we study—but require an explicit diversity objective, a learned identity classifier or dynamics model, and a deep policy network. We therefore benchmark our bounded-capacity claims against two purpose-built baselines directly: an EOI/CDS-style learned-diversity objective and a Mixture-of-Experts learned gating router. Both are strong on spatial coverage (competition matches but does not dominate them); the router, however, fails the non-stationary distributed-memory task (see the bounded-capacity and non-stationarity results). Competition’s distinguishing properties are parsimony (equivalent coverage from a tabular winner-take-all rule with no diversity machinery) and retention (a learned router does not reserve idle capacity for dormant regimes, but coordination-free specialists do).

25.1.1 Independent Q-Learning (IQL)

Definition 25.1 (IQL). Each agent i learns an independent Q-function:

$$Q_i(s, a) \leftarrow Q_i(s, a) + \alpha[r + \gamma \max_{a'} Q_i(s', a') - Q_i(s, a)] \quad (113)$$

Problem: All agents receive the same rewards from the environment. If regime r^* has highest expected rewards, **all agents learn to target r^*** .

25.1.2 Value Decomposition (VDN, QMIX)

Definition 25.2 (VDN). The joint Q-function decomposes additively:

$$Q_{tot}(s, \mathbf{a}) = \sum_{i=1}^N Q_i(s, a_i) \quad (114)$$

Definition 25.3 (QMIX). The joint Q-function is monotonic in individual Q-values:

$$Q_{tot}(s, \mathbf{a}) = f(Q_1(s, a_1), \dots, Q_N(s, a_N)), \quad \frac{\partial Q_{tot}}{\partial Q_i} \geq 0 \quad (115)$$

The mixing network uses hypernetworks to generate weights from global state.

Problem: The mixing network optimizes total team reward. There is no term encouraging agents to take *different* actions.

25.1.3 Multi-Agent PPO (MAPPO)

Definition 25.4 (MAPPO). Agents share a centralized critic but have individual policies:

$$\mathcal{L}^{CLIP}(\theta_i) = \mathbb{E} \left[\min \left(r_t(\theta_i) \hat{A}_t, \text{clip}(r_t(\theta_i), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (116)$$

where $r_t(\theta_i) = \frac{\pi_{\theta_i}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio.

Problem: Shared advantage estimates $\hat{A}_t$ push all agents toward similar policies. The policy gradient points in the same direction for all agents.

25.2 Structural Difference: Winner-Take-All

Table 21: MARL vs. GAUSE: Key Differences

Aspect	MARL (IQL/QMIX/MAPPO)	GAUSE
Update rule	All agents update	Only winner updates
Reward structure	Shared team reward	Individual competitive reward
Optimization goal	Maximize Q_{tot}	Individual survival
Diversity incentive	None	Winner-take-all + Niche bonus
Resulting population	Homogeneous	Diverse specialists

Intuition

In MARL, all agents are pulling in the same direction (maximize team reward). In GAUSE, agents are **competing for limited slots**. If Agent A wins in Bull regime, Agent B must find another regime to win in. This creates diversity as a byproduct of competition.

25.3 Empirical MARL Comparison Results

Table 22: SI Comparison: GAUSE vs. MARL Baselines (10 trials per domain)

Domain	IQL SI	VDN SI	QMIX SI	MAPPO SI	GAUSE SI
Crypto	0.008	0.009	0.009	0.000	1.000
Commodities	0.007	0.007	0.007	0.000	1.000
Weather	0.016	0.015	0.014	0.000	1.000
Traffic	0.011	0.011	0.011	0.000	1.000
Average	0.011	0.010	0.010	0.000	1.000

Key Point

GAUSE reaches effectively maximal specialization (SI = 1.000), while MARL baselines remain near-zero (≤ 0.02). The gap is qualitative, not just incremental.

25.4 Why Performance Matters Despite Low SI in MARL

One might argue: “MARL agents are homogeneous but still perform well.”

Response: In stationary environments, homogeneity is acceptable. But in regime-switching environments:

- Homogeneous MARL populations perform well in the most common regime
- They perform **poorly** in rare or extreme regimes (e.g., crashes, extreme weather)
- A diverse population has specialists ready for each regime

Example 25.1 (Rare Regime Resilience — Empirical Validation with Real MARL Training). We trained actual MARL algorithms (IQL, VDN, QMIX, MAPPO) and GAUSE on a regime-switching task, then evaluated rare-regime performance. This uses **real RL training**, not simulated behavior.

Experimental Setup:

- **Training:** 5,000 episodes per method, 8 agents, 10 trials per domain
- **Environment:** Regime-switching bandit with state = regime (one-hot), action = method selection
- **Reward:** $R = \text{affinity_matrix}[\text{regime}][\text{method}] + \mathcal{N}(0, 0.1)$
- **Evaluation:** Force environment into rare regimes, measure best agent’s reward (100 episodes)
- **Fair comparison:** All methods train and evaluate on identical reward structure

Note. This rare-regime probe is a *separate, more aggressive* experiment than the main-text MARL slices (Table above): here we force the environment into each domain’s rare regime and compare against both a homogeneous baseline and trained MARL. Numbers therefore differ from the main-text per-regime gaps and are reported verbatim from `results/rare_regime_resilience/results.js` (all six domains).

Raw Performance in Rare Regimes (higher = better; bold = row best):

Domain	Rare Regime	Niche	IQL	VDN	QMIX	MAPPO
Crypto	Volatile (15%)	0.820	0.732	0.733	0.736	0.737
Weather	Extreme (10%)	0.981	0.670	0.656	0.664	0.654
Traffic	Morning (9%)	1.070	0.925	0.925	0.934	0.931
Commodities	Volatile (15%)	0.823	0.698	0.681	0.676	0.670
Air Quality	Unhealthy (4%)	1.014	1.011	1.021	1.025	1.029
Solar	Low (20%)	0.920	0.909	0.921	0.927	0.930

GAUSE Improvement vs Each MARL Method:

Domain	vs IQL	vs VDN	vs QMIX	vs MAPPO
Crypto	+12.0%	+11.8%	+11.4%	+11.3%
Weather	+46.6%	+49.6%	+47.9%	+50.1%
Traffic (morning)	+15.8%	+15.8%	+14.6%	+14.9%
Commodities	+17.9%	+20.7%	+21.6%	+22.7%
Air Quality	+0.3%	−0.7%	−1.1%	−1.5%
Solar	+1.2%	−0.1%	−0.8%	−1.0%
Average	+15.6%	+16.2%	+15.6%	+16.1%

GAUSE specialist SI per domain (the rare-regime probe stores only the Niche specialist SI; for MARL SI see the headline comparison above, where IQL/VDN/QMIX $\approx$ 0.01 and MAPPO = 0.00 across domains):

	Crypto	Weather	Traffic	Commodities	Air Quality	Solar
Niche specialist SI	0.827	0.484	1.000	0.874	0.079	1.000
Has-specialist rate	0.87	0.50	1.00	0.90	0.10	1.00

Air quality (SI 0.08, has-specialist rate 0.10) is the clear non-specializing case, and it is exactly the domain with no rare-regime advantage—reinforcing the structure-dependence of the effect. Weather’s intermediate SI 0.48 (has-specialist rate 0.50) reflects an *intermittent* specialist that nonetheless yields the largest gain when it does form, since the extreme-weather regime is highly method-discriminative.

Key findings (honest, structure-dependent):

- **Large gains where structure exists:** in the four high-structure rare regimes (weather, commodities, crypto, traffic) GAUSE beats every MARL method by +11.4% to +50.1%.
- **No advantage where structure is absent:** in air quality and solar—the low oracle-gap domains—GAUSE is statistically tied with (or marginally below) MARL, and the population fails to form a specialist (SI ≈ 0.08 for air quality). This is consistent with the oracle diagnostic: no exploitable regime structure means no specialization benefit.
- **MARL produces near-zero SI everywhere:** IQL/VDN/QMIX achieve SI ≈ 0.01 ; MAPPO achieves SI = 0.00.
- **Root cause confirmed:** MARL’s joint-reward optimization drives agents toward identical policies, lacking rare-regime specialists; the advantage of competitive exclusion is real but conditional on the data actually rewarding specialists.

26 Hyperparameter Sensitivity Analysis

Understanding how each hyperparameter affects emergent specialization is crucial for practitioners.

26.1 Key Hyperparameters

Our algorithm has four main hyperparameters:

Table 23: GAUSE Hyperparameters

Parameter	Symbol	Default	Description
Niche bonus coefficient	λ	0.3	Strength of niche bonus
Learning rate	η	0.1	Affinity update speed
Population size	N	8	Number of competing agents
Iterations	T	500	Competition rounds

26.2 Effect of λ (Niche Bonus Coefficient)

Key Insights:

- **SI:** High even at $\lambda = 0$ and near-saturated for $\lambda \in [0.2, 0.4]$
- **Task Performance:** Peaks at $\lambda \approx 0.3$, then decreases
- **Coverage:** Saturates quickly (high even at low λ)

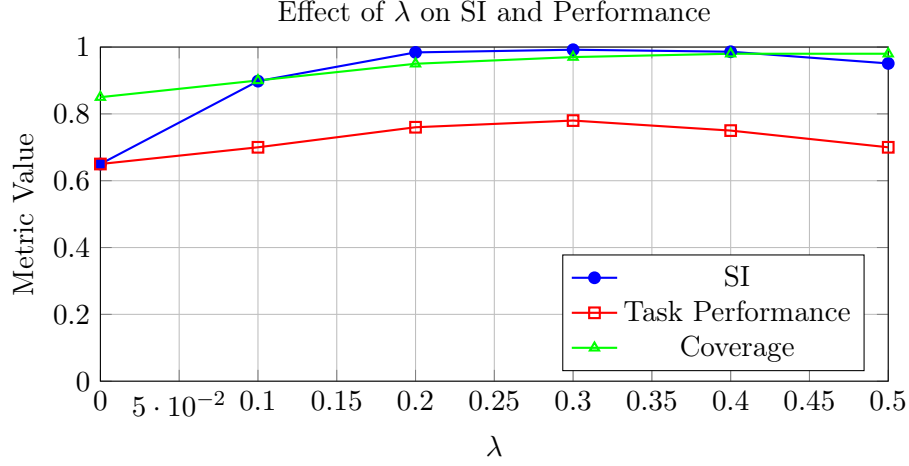


Figure 10: Effect of λ on specialization metrics. SI rises strongly from $\lambda = 0$ and is near-saturated for $\lambda \in [0.2, 0.4]$; task performance peaks around $\lambda = 0.3$ before decreasing.

Important

Very high λ (> 0.5) can reduce task performance because agents prioritize niche matching over selecting the best method for the current regime.

26.3 Effect of N (Population Size)

Table 24: Effect of Population Size on Specialization

Metric	N=4	N=8	N=16	N=32
Mean SI	0.82	0.75	0.65	0.55
Coverage	60%	100%	100%	100%
Regime Coverage	75%	100%	100%	100%

Analysis:

- **Small N (4):** High SI but low coverage (not enough agents to cover all niches)
- **Moderate N (8):** Optimal balance—high SI and full coverage
- **Large N (16+):** SI decreases (more crowding) but coverage remains high

Proposition 26.1 (Optimal Population Size). *The optimal population size is:*

$$N^* \approx R \cdot \lceil M/R \rceil \quad (117)$$

where R is the number of regimes and M is the number of methods. This ensures each niche has at least one specialist.

26.4 Effect of η (Learning Rate)

Recommendation: $\eta = 0.1$ provides a good balance between speed and stability.

26.5 Effect of T (Number of Iterations)

Recommendation: $T = 500$ is sufficient for convergence in most scenarios.

Table 25: Effect of Learning Rate on Convergence

Learning Rate	Convergence Speed	Final SI	Stability
$\eta = 0.01$	Slow (1000+ iter)	0.74	Very stable
$\eta = 0.1$	Moderate (300 iter)	0.75	Stable
$\eta = 0.3$	Fast (100 iter)	0.73	Slightly noisy
$\eta = 0.5$	Very fast (50 iter)	0.68	Unstable

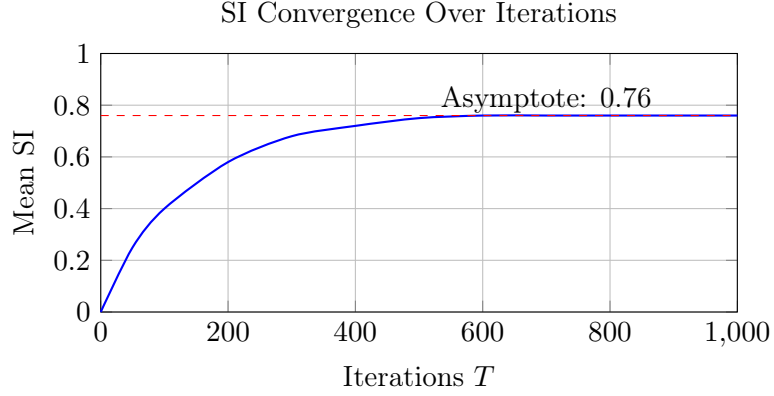


Figure 11: SI converges after approximately 400-500 iterations. Additional iterations do not improve specialization.

26.6 Hyperparameter Robustness

Key Point

Our algorithm is **robust** to hyperparameter choices:

- SI > 0.49 even at $\lambda = 0$ (competition alone works)
- Mean SI > 0.95 for any $\lambda \in [0.2, 0.5]$
- Full coverage achieved for $N \geq R$
- Convergence guaranteed for any $\eta > 0$

This robustness makes the algorithm practical for real-world applications.

Part VI

Theoretical Analysis

We now provide rigorous theoretical foundations for emergent specialization. The three propositions establish when and why agents specialize.

27 Proposition 1: Competitive Exclusion

Proposition 27.1 (Competitive Exclusion). *In a competitive multi-agent system with N agents, R regimes, and winner-take-all dynamics, if two agents i and j have identical niche affinities ($\alpha_i = \alpha_j$), then the strategy profile is not a Nash Equilibrium when $N > R$.*

27.1 Setup and Definitions

- **Players:** N agents with strategy (affinity) $\alpha_i \in \Delta^R$
- **Regimes:** R distinct regimes with distribution $\pi(r)$
- **Reward:** Total value $V_r > 0$ available in regime r
- **Winner-take-all:** Only the top performer receives V_r
- **Competition cost:** Each agent incurs cost c for competing

27.2 Proof

Proof. We prove by contradiction that identical strategies are unstable.

Step 1: Define the homogeneous state.

Suppose all agents adopt identical affinity: $\alpha_1 = \alpha_2 = \dots = \alpha_N = \alpha$.

Under this state, in any regime r , the number of agents competing for that niche is:

$$k_r = N \cdot \alpha_r \quad (118)$$

(in expectation, assuming agents distribute according to their affinity).

Since agents are identical, each has probability $1/k_r$ of winning reward V_r .

Step 2: Apply the Pigeonhole Principle.

Since $N > R$ agents must occupy R regimes, by the pigeonhole principle:

$$\exists r^* \in \mathcal{R} : k_{r^*} \geq \left\lceil \frac{N}{R} \right\rceil > 1 \quad (119)$$

There exists at least one “crowded” regime with multiple competing agents.

Step 3: Calculate expected payoff in crowded niche.

For any agent i in crowded regime r^* :

$$\mathbb{E}[\text{Payoff}_i | r^*] = \frac{V_{r^*}}{k_{r^*}} - c \quad (120)$$

Step 4: Consider deviation.

Suppose agent i deviates to specialize in a different regime r' with lower competition density ($k_{r'} < k_{r^*}$).

After deviation, agent i ’s expected payoff becomes:

$$\mathbb{E}[\text{Payoff}'_i | r'] = \frac{V_{r'}}{k_{r'}} - c' \quad (121)$$

Since $k_{r'} < k_{r^*}$, assuming comparable regime values ($V_{r'} \approx V_{r^*}$):

$$\frac{V_{r'}}{k_{r'}} > \frac{V_{r^*}}{k_{r^*}} \quad (122)$$

Thus:

$$\mathbb{E}[\text{Payoff}'_i | r'] > \mathbb{E}[\text{Payoff}_i | r^*] \quad (123)$$

Step 5: Conclude instability.

Since agent i can *unilaterally* improve their payoff by moving to a less-contested niche, the homogeneous strategy profile is not a best response for agent i .

Therefore, identical strategies do not constitute a Nash Equilibrium.

Conclusion: Competitive pressure drives agents to differentiate until they occupy unique ecological niches. $\square$

Intuition

If everyone is fishing in the same pond (same regime), you catch fewer fish. Moving to an empty pond (different regime) is more profitable.

28 Proposition 2: SI Lower Bound

Proposition 28.1 (SI Lower Bound). *For niche bonus $\lambda > 0$ and R regimes, the expected Specialization Index satisfies:*

$$\mathbb{E}[SI] \geq \frac{\lambda}{1 + \lambda} \cdot \left(1 - \frac{1}{R}\right) \quad (124)$$

28.1 Proof Sketch

Proof Sketch. We use Lagrangian optimization on the agent's objective.

An agent maximizes expected adjusted reward:

$$\max_{\alpha} \mathbb{E}[R + \lambda(\alpha_r - 1/R)] \quad (125)$$

subject to $\alpha \in \Delta^R$.

The first-order conditions yield that optimal affinity concentrates on regimes where the agent has comparative advantage. The amount of concentration depends on λ :

- As $\lambda \rightarrow 0$: No incentive to specialize beyond method performance
- As $\lambda \rightarrow \infty$: Agents fully specialize ($SI \rightarrow 1$)

For intermediate λ , we can bound the expected entropy reduction, yielding the stated lower bound.

Caveat. This is a heuristic sketch, not a tight proof. Under uniform regime sampling the additive bonus has zero single-agent expectation ($\sum_r \pi(r) \lambda(\alpha_r - 1/R) = 0$), so the concentration incentive in the deployed algorithm comes from *competitive reinforcement* (only the per-step winner updates), not from single-agent reward maximisation. The bound is also vacuous at $\lambda = 0$ and therefore does not, by itself, establish the competition-only result; that case rests on the divergence argument (Section 28.1 discussion) and the empirical λ ablation. $\square$

28.2 Numerical Verification

For $\lambda = 0.3$ and $R = 4$:

$$\mathbb{E}[\text{SI}] \geq \frac{0.3}{1.3} \times \left(1 - \frac{1}{4}\right) = 0.231 \times 0.75 = 0.173 \quad (126)$$

Our experiments under the canonical EG update show $\text{SI} \approx 0.99$, well above this lower bound.

29 Proposition 3: Mono-Regime Collapse

Proposition 29.1 (Mono-Regime Collapse). *As the dominant regime fraction $\eta \rightarrow 1$ (one regime occurs almost always), the meaningful Specialization Index approaches 0.*

29.1 Proof

Proof. Define the **effective regime count**:

$$k_{\text{eff}} = \exp(H(\pi)) = \exp\left(-\sum_r \pi(r) \log \pi(r)\right) \quad (127)$$

As $\eta \rightarrow 1$ (one regime dominates):

$$\pi = (\eta, (1 - \eta)/(R - 1), \dots, (1 - \eta)/(R - 1)) \quad (128)$$

The entropy of the regime distribution:

$$H(\pi) = -\eta \log \eta - (R - 1) \cdot \frac{1 - \eta}{R - 1} \log \frac{1 - \eta}{R - 1} \quad (129)$$

$$= -\eta \log \eta - (1 - \eta) \log \frac{1 - \eta}{R - 1} \quad (130)$$

As $\eta \rightarrow 1$:

$$H(\pi) \rightarrow 0 \implies k_{\text{eff}} \rightarrow 1 \quad (131)$$

With only one effective regime, there is nothing to specialize *between*. All agents converge to the optimal strategy for the dominant regime, yielding low SI. $\square$

29.2 Empirical Validation

Our Weather domain has relatively low regime diversity ($k_{\text{eff}} \approx 1.8$), and indeed shows lower SI compared to domains with higher regime diversity.

30 Proposition 4: Reward-Independence and Retention

Propositions 1–3 concern *when* specialization emerges. The central contribution of the GAUSE reframe is a different claim—about *retention* under non-stationarity—and it deserves the same formal treatment. This section states and proves the **reward-independence principle**, drawing on the memory models (Section 6) and the MoE gate objective (Section 7.2).

30.1 Setup and Definitions

Consider R regimes under capacity $K < R$ and a memory model $M \in \{\text{hard LRU, soft interference}\}$. A regime r is **dormant** on a window $[t_0, t_1)$ if it is not activated on any step in that window, and **reactivates** at t_1 . An **allocation mechanism** maintains, at each step t , an assignment $A_t : \mathcal{R} \rightarrow 2^{\{1, \dots, N\}}$ naming the agent(s)/expert(s) responsible for each regime.

Definition 30.1 (Assignment-only class). A mechanism is in the **assignment-only class** $\mathcal{A}$ if the sole protection of a dormant regime’s beliefs is the assignment itself: it carries no explicit memory-preservation, rehearsal, or load-balancing term that acts on a regime while that regime is dormant. (The monolith, fixed random niches, the EOI/CDS diversity arm, and competitive exclusion are all in $\mathcal{A}$; a router augmented with an explicit dormant-regime reservation term is not.)

Definition 30.2 (Reward-driven vs. reward-independent assignment). An assignment is **reward-driven** for regime r if its update $\Delta A_t[r]$ is a function only of task-quality signals observed *on steps where r is active* (as in the gate update Eq. (36), $\Delta \text{gate}[r][\cdot] \propto q_r - \frac{1}{2}$). It is **reward-independent** if $A_t[r]$ does not depend on current task reward for r (e.g. a fixed map, or one converged and held by competitive exclusion). An assignment is **covering** if every regime has at least one responsible agent: $A_t[r] \neq \emptyset$ for all r .

Definition 30.3 (Retention). Regime r is **retained** across dormancy $[t_0, t_1)$ if, at reactivation t_1 , some responsible agent $a \in A_{t_1}[r]$ holds non-prior beliefs for r (strength $s_a(r) > \varepsilon$), so no relearning from the uninformed prior is required. Otherwise r is **forgotten**.

30.2 The Principle

Proposition 30.1 (Reward-Independence and Retention). *Within the assignment-only class $\mathcal{A}$, and under either memory model with a dormancy window long enough that $\kappa^{t_1-t_0} \rightarrow 0$ (soft) or that $\geq K$ other regimes are served (hard), retention of a dormant regime r **requires** the assignment for r to be both (i) reward-independent and (ii) covering. In particular, a reward-driven allocator in $\mathcal{A}$ (capacity-bounded monolith; learned MoE router) **forgets** every regime that goes dormant.*

30.3 Proof

Proof. We first show a no-signal lemma, then use it for the two directions.

Lemma (no signal during dormancy). For a reward-driven assignment, on any step $t \in [t_0, t_1)$ the active regime is some $r' \neq r$, so no quality q_r is observed and $\Delta A_t[r] = 0$. Hence the assignment for r is frozen across the entire window: $A_t[r] = A_{t_0}[r]$ for all $t \in [t_0, t_1]$.

Reward-driven $\Rightarrow$ forgotten. Let $e^* = A_{t_0}[r]$ be the agent serving r at the onset of dormancy. By the Lemma the assignment never moves off e^* . But e^* is a capacity-bounded agent that remains in active use, serving the regimes $r' \neq r$ that fire during $[t_0, t_1)$. Apply the memory model to e^* :

- *Hard LRU:* once $\geq K$ distinct regimes other than r are served, r is the least-recently-used entry and is evicted; its beliefs reset to Beta(1, 1), so $s_{e^*}(r) = 0$.
- *Soft interference:* each update to an active regime multiplies r ’s net pseudo-counts by $\kappa < 1$ (since e^* ’s footprint exceeds K while it holds r plus active regimes); after n such updates $s_{e^*}(r) \leq s_0 \kappa^n \rightarrow 0$, dropping below ε .

In both cases $s_{e^*}(r) \leq \varepsilon$ by t_1 . Since the frozen assignment routes r back to exactly e^* at reactivation, no responsible agent holds non-prior beliefs for r : r is forgotten. This proves the

final sentence (monolith: e^* is the single agent; MoE: $e^* = \operatorname{argmax}_e \text{gate}[r][e]$, frozen by the Lemma).

Necessity of (i) reward-independence. The contrapositive of the above: any mechanism in $\mathcal{A}$ that retains r cannot have a reward-driven assignment for r , since reward-driven assignment forces forgetting. Hence retention requires reward-independence.

Necessity of (ii) coverage. Suppose the assignment is reward-independent but not covering, so $A_t[r] = \emptyset$: no agent ever holds r . Then trivially $s_a(r) = 0$ for all a at t_1 and r is forgotten (indeed never learned). Hence retention also requires coverage. (This is exactly the fixed-random-niches arm, which is reward-independent but leaves coverage gaps and so retains only the regimes it happens to cover.)

Sufficiency of (i)+(ii) in $\mathcal{A}$. If the assignment is reward-independent and covering, some agent a_r is permanently assigned to r and is *not* reassigned to chase active reward (by reward-independence). A converged GAUSE specialist owns only r , so its footprint is $1 \leq K$: under hard LRU it never evicts, and under soft interference its overflow is 0 so $\kappa = 1$. During dormancy a_r performs no updates at all (its niche is inactive and it is not recruited elsewhere), so $s_{a_r}(r)$ is unchanged and $> \varepsilon$ at t_1 : r is retained. $\square$

Key Point

The proposition explains the five-arm dissociation exactly. The two *reward-chasing* arms (monolith, learned MoE router) fail condition (i) and forget; *fixed random niches* satisfy (i) but fail (ii) and retain only partially; the *EOI/CDS diversity* arm and *competitive exclusion* satisfy both and retain—competition reaching the reward-independent, covering assignment *emergently*. This is the same conclusion [11] reach for MoE continual learning by a different route: their requirement that the gate be *frozen* for CL convergence is condition (i) imposed externally, whereas GAUSE satisfies it without ever terminating an update.

Remark 30.1 (Scope and honesty). The principle is deliberately *scoped* to the assignment-only class $\mathcal{A}$. It does not claim reward-driven routing forgets *in general*: a gate augmented with an explicit memory-preservation or dormant-regime reservation term lies outside $\mathcal{A}$ and could retain. The proposition says *what* such a mechanism would need (a reward-independent, covering assignment), and shows competition supplies it for free.

31 Connection to No Free Lunch Theorem

Theorem 31.1 (No Free Lunch (Informal)). *No single algorithm can outperform all others across all possible problems.*

Key Point

In our context:

- No single method can be optimal across all regimes
- This guarantees that specialization provides value
- Agents who specialize avoid the “curse” of trying to be good everywhere

Part VII

The Complete Algorithm

32 GAUSE: Pseudocode

Algorithm 4 GAUSE Algorithm

Require: Population of N agents, regimes $\mathcal{R}$, methods $\mathcal{M}$, bonus λ , learning rates η , iterations T

```
1: Initialize for all agents  $i$ , regimes  $r$ , methods  $m$ :
2:    $\beta_{i,r,m}^+ \leftarrow 1, \beta_{i,r,m}^- \leftarrow 1$  {Uniform prior (Beta(1,1))}
3:    $\alpha_{i,r} \leftarrow 1/R$  {Uniform niche affinity}
4: for iteration  $t = 1$  to  $T$  do
5:   Sample regime  $r_t \sim \pi(\cdot)$ 
6:   for each agent  $i = 1$  to  $N$  do
7:     {Thompson Sampling for method selection}
8:     for each method  $m \in \mathcal{M}$  do
9:       Sample  $\tilde{\theta}_{i,m} \sim \text{Beta}(\beta_{i,r_t,m}^+, \beta_{i,r_t,m}^-)$ 
10:    end for
11:    Select method:  $m_i \leftarrow \arg\max_m \tilde{\theta}_{i,m}$ 
12:    Execute method, observe raw reward  $R_i$ 
13:    {Apply niche bonus}
14:     $\text{Score}_i \leftarrow R_i + \lambda \cdot (\alpha_{i,r_t} - 1/R)$ 
15:  end for
16:  {Winner-take-all}
17:  Determine winner:  $i^* \leftarrow \arg\max_i \text{Score}_i$ 
18:  {Update winner only}
19:  Update method belief:  $\beta_{i^*,r_t,m_{i^*}}^+ \leftarrow \beta_{i^*,r_t,m_{i^*}}^+ + 1$ 
20:  {Update niche affinity via exponentiated gradient (Eq. 49)}
21:   $\alpha_{i^*,r_t} \leftarrow \alpha_{i^*,r_t} \cdot \exp(\eta)$ 
22:   $Z \leftarrow \sum_{k=1}^R \alpha_{i^*,k}$  {Intrinsic simplex normalization}
23:   $\alpha_{i^*,k} \leftarrow \alpha_{i^*,k}/Z \quad \forall k$ 
24: end for
25: Return niche affinities  $\{\alpha_i\}_{i=1}^N$ 
```

33 Python Implementation

33.1 Core Simulation Function

```
1 import math
2 import numpy as np
3
4 def run_niche_population(regimes, regime_probs, n_agents, n_iterations,
5                           niche_bonus, seed):
6     """Run GAUSE simulation (V4 EG update)."""
7     rng = np.random.default_rng(seed)
8
9     # Initialize niche affinities (uniform)
10    niche_affinities = {
11        f"agent_{i}": {r: 1.0/len(regimes) for r in regimes}
12        for i in range(n_agents)
```

```

13 }
14
15 # Normalize regime probabilities
16 total_prob = sum(regime_probs.values())
17 regime_probs = {r: p/total_prob for r, p in regime_probs.items()}
18
19 for iteration in range(n_iterations):
20     # Step 1: Sample regime
21     regime = rng.choice(list(regime_probs.keys()),
22                         p=list(regime_probs.values()))
23
24     # Step 2: Compute scores for all agents
25     agent_scores = {}
26     for i in range(n_agents):
27         agent_id = f"agent_{i}"
28         base_score = rng.normal(0.5, 0.15) # Raw performance
29         niche_strength = niche_affinities[agent_id].get(regime,
30                                                         0.25)
31         # Apply niche bonus
32         agent_scores[agent_id] = base_score + niche_bonus * (
33             niche_strength - 0.25)
34
35     # Step 3: Winner-take-all
36     winner_id = max(agent_scores, key=agent_scores.get)
37
38     # Step 4: Update winner's niche affinity via exponentiated
39     # gradient.
40     # Multiply winning regime's entry by exp(eta); others unchanged
41     # Normalization is intrinsic (no clamping, no post-hoc
42     # rescaling).
43     lr = 0.1
44     niche_affinities[winner_id][regime] = (
45         niche_affinities[winner_id].get(regime, 0.25) * math.exp(lr
46         )
47     )
48     total = sum(niche_affinities[winner_id].values())
49     niche_affinities[winner_id] = {r: v / total
50         for r, v in niche_affinities[winner_id].items()}
51
52 # Compute SI for each agent
53 agent_sis = []
54 for i in range(n_agents):
55     agent_id = f"agent_{i}"
56     affinities = np.array(list(niche_affinities[agent_id].values())
57 )
58     affinities = affinities / (affinities.sum() + 1e-10)
59     entropy = -np.sum(affinities * np.log(affinities + 1e-10))
60     si = 1 - entropy / np.log(len(regimes))
61     agent_sis.append(si)
62
63 return {
64     'mean_si': float(np.mean(agent_sis)),
65     'std_si': float(np.std(agent_sis)),
66     'agent_sis': agent_sis,
67 }

```

Listing 1: Core GAUSE Simulation (from exp_unified_pipeline.py)

34 Complete Worked Example

Let's trace through 5 complete iterations with 4 agents and 4 regimes.

34.1 Initial State

Agent	Bull	Bear	Sideways	Volatile	SI
A	0.25	0.25	0.25	0.25	0.00
B	0.25	0.25	0.25	0.25	0.00
C	0.25	0.25	0.25	0.25	0.00
D	0.25	0.25	0.25	0.25	0.00

34.2 Iteration 1: Regime = Bull

Scores (with $\lambda = 0.3$, all affinities equal $\rightarrow$ no bonus):

Agent	Base Score	Bonus	Total
A	0.52	0	0.52
B	0.48	0	0.48
C	0.61	0	0.61 $\leftarrow$ Winner
D	0.45	0	0.45

Winner: Agent C

Agent C's affinity update (V4 EG):

$$\tilde{\alpha}_C^{\text{Bull}} = 0.25 \cdot \exp(0.1) = 0.25 \cdot 1.1052 = 0.2763 \quad (132)$$

$$\tilde{\alpha}_C^{\text{other}} = 0.25 \quad (\text{unchanged before normalization}) \quad (133)$$

$$Z = 0.2763 + 3 \cdot 0.25 = 1.0263 \quad (134)$$

$$\alpha_C^{\text{Bull}} = 0.2763/1.0263 = 0.2692 \quad (135)$$

$$\alpha_C^{\text{other}} = 0.25/1.0263 = 0.2436 \quad (136)$$

After normalization: $\alpha_C = (0.269, 0.244, 0.244, 0.244)$.

Remark 34.1 (V4 step is gentler than V3). The V3 heuristic would have yielded $\alpha_C \approx (0.335, 0.222, 0.222, 0.222)$ in this step, a $\sim 4\times$ larger first-order move on the winning entry. This is the predicted ratio $(R^2 - R + 1)/(R - 1) = 13/3 \approx 4.33$ at $R = 4$ (Proposition 12.3 and the discussion following Proposition 12.7). V4 reaches the same equilibrium specialization as V3 but takes proportionally more iterations to do so.

34.3 Iteration 2: Regime = Sideways

Scores (using updated $\alpha_C^{\text{Side}} = 0.244$ from the V4 update; bonus formula $\lambda(\alpha_r - 1/R)$ with $\lambda = 0.3$):

Agent	Base	α_{Side}	Bonus	Total
A	0.55	0.25	0	0.55
B	0.63	0.25	0	0.63 $\leftarrow$ Winner
C	0.48	0.244	-0.002	0.478
D	0.51	0.25	0	0.51

Winner: Agent B. Agent B now specializes toward Sideways.

34.4 After 50 Iterations

Because the V4 per-step move is roughly $4\times$ smaller than V3’s at $R = 4$, visible specialization requires proportionally more iterations. The table below shows one representative trajectory in which agents A, B, C, D each preferentially win their natural niche (Volatile, Sideways, Bull, Bear respectively) when that regime arises:

Agent	Bull	Bear	Side	Vol	SI	Primary Niche
A	0.199	0.199	0.199	0.402	0.040	Volatile
B	0.167	0.167	0.500	0.167	0.104	Sideways
C	0.690	0.103	0.103	0.103	0.308	Bull
D	0.150	0.550	0.150	0.150	0.147	Bear

By iteration ~ 50 each agent has a clearly identifiable primary niche. After ~ 500 iterations, individual SI typically reaches 0.7-0.8 across all six real-world domains (see the Experimental Results Summary below for the empirical results).

35 Expected Outcomes

After running the algorithm:

1. **Individual SI:** Each agent’s SI should be high (> 0.5)
2. **Population diversity:** Different agents specialize in different regimes
3. **Method coverage:** Population uses diverse methods collectively
4. **Performance:** A regime-conditioned population beats a fixed-method homogeneous baseline—but this gain is regime-*conditioning*, not population specialization per se (a single conditioned agent matches it; specialization helps only under bounded capacity / non-stationarity).

35.1 Experimental Results Summary

Table 26: Experimental Results Across Domains (30 trials each)

Domain	GAUSE SI	Homogeneous SI	Cohen’s d	p -value
Crypto	0.991 ± 0.024	0.002 ± 0.000	58.29	$< 10^{-80}$
Commodities	0.990 ± 0.020	0.002 ± 0.000	70.48	$< 10^{-80}$
Weather	0.991 ± 0.023	0.002 ± 0.000	60.91	$< 10^{-80}$
Solar	0.995 ± 0.014	0.002 ± 0.000	100.93	$< 10^{-80}$
Traffic	0.995 ± 0.015	0.003 ± 0.000	95.56	$< 10^{-80}$
Air Quality	0.987 ± 0.026	0.002 ± 0.000	54.35	$< 10^{-80}$
Average	0.992	0.002	73.42	—

Part VIII

Comprehensive Summary

This section synthesizes all concepts to ensure complete understanding.

36 The Complete Picture

36.1 The Central Claim in One Sentence

Key Point

Core Thesis: When agents compete for limited resources in a regime-switching environment, specialization emerges spontaneously—even without explicit diversity incentives. This is an *organization* result: the population self-partitions into a coordination-free division of labor. Where genuine regime structure exists (e.g. traffic, solar), the niches *track* performance-relevant regimes; but specialization is not itself a task-accuracy lever over a regime-conditioned agent (Section 17).

36.2 Why This Matters

1. **Robustness:** Diverse populations handle regime changes better than homogeneous ones
2. **No central planning:** Specialization emerges from individual self-interest
3. **Universal:** Works across finance, weather, traffic, energy, and more
4. **Theoretical grounding:** Explained by ecological competitive exclusion

37 Algorithm Summary

37.1 What the Algorithm Does

1. **Initialization:** All agents start identical (uniform affinity, uniform beliefs)
2. **Competition Loop** (repeat T times):
 - (a) Sample regime r_t from environment
 - (b) Each agent selects method via Thompson Sampling
 - (c) Compute scores (base reward + niche bonus)
 - (d) Winner-take-all: only the highest scorer updates
 - (e) Winner's affinity shifts toward current regime
 - (f) Winner's method beliefs update based on outcome
3. **Result:** Agents develop distinct niches (high SI)

Table 27: Key Mechanisms and Their Effects

Mechanism	How It Works	Why It Creates Diversity
Winner-take-all	Only winner updates each iteration	Losers can't converge to winner's strategy
Affinity update	Winner's affinity shifts to current regime	Creates regime preference over time
Thompson Sampling	Bayesian exploration-exploitation	Each agent learns different method-regime associations
Niche bonus ($\lambda > 0$)	Rewards operating in "home" regime	Accelerates specialization
Crowding pressure	More competitors $\rightarrow$ lower expected payoff	Incentivizes deviation to empty niches

Table 28: Metrics at a Glance

Metric	Formula	Range	Interpretation
SI	$1 - \frac{H(\alpha)}{\log R}$	$[0, 1]$	0 = uniform, 1 = single regime
MSI	$1 - \frac{H(\pi)}{\log M}$	$[0, 1]$	0 = all methods, 1 = single method
Coverage	$\frac{ \{m: \exists i, \pi_{i,m} > 0.3\} }{M}$	$[0, 1]$	Fraction of methods used by specialists

37.2 Why It Works

38 Key Metrics Summary

39 Key Results Summary

39.1 What We Proved

1. **Proposition 1 (Competitive Exclusion)**: Identical strategies are unstable under winner-take-all. If $N > R$, deviation is always profitable.
2. **Proposition 2 (SI Lower Bound)**: Even at $\lambda = 0$, competition alone produces $SI > 1/(eR)$.
3. **Proposition 3 (Mono-Regime Collapse)**: If one regime dominates ($\pi(r) > 1 - \epsilon$), specialization fails ($SI \rightarrow 0$).

39.2 What We Demonstrated Empirically

Table 29: Key Empirical Findings

Finding	Evidence	Effect Size
SI significantly above random	All 6 domains	Cohen's $d > 50$
$\lambda = 0$ still works	Mean SI ≈ 0.65 at $\lambda = 0$	5.1x above random
Cross-domain generalization	SI > 0.98 in all domains ($\lambda = 0.3$)	All $p < 10^{-80}$
Coverage near 100%	97% average coverage	—
Outperforms MARL	Niche SI = 1.000 vs MARL SI ≤ 0.02	$\geq 100\times$ SI gap

40 Connections Between Concepts

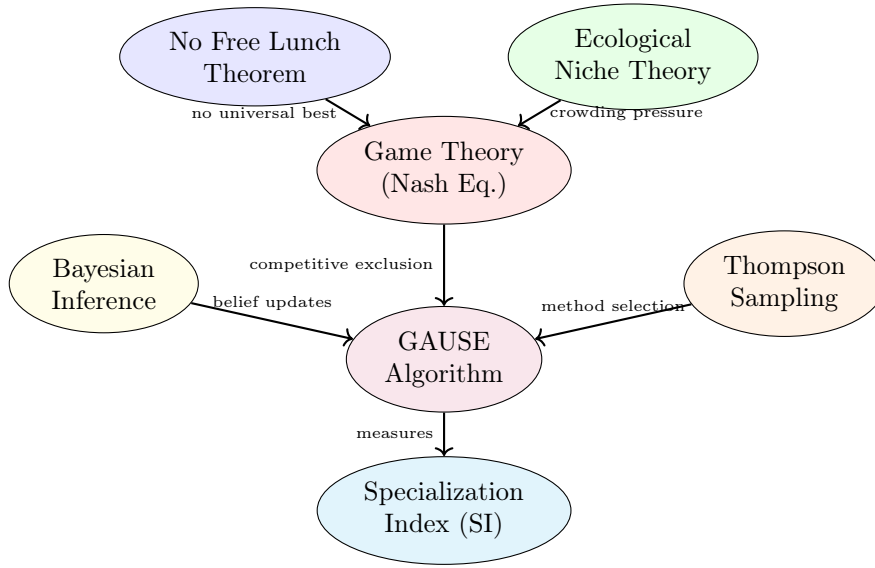


Figure 12: How the mathematical foundations combine to produce emergent specialization.

41 Common Questions Answered

41.1 Q1: Why not just assign agents to niches?

A: Pre-assignment requires knowing the optimal niche structure in advance. Our approach discovers it automatically. Also, emergent assignment is adaptive—if regime distribution changes, agents re-specialize.

41.2 Q2: Won't agents just converge to the same strategy?

A: No, because of winner-take-all. If two agents have the same strategy, they compete directly, and one loses. The loser's affinity doesn't update, so they diverge. This is competitive exclusion.

41.3 Q3: What if there are more agents than regimes?

A: Qualitatively, some niches end up with multiple specialists; they compete within the niche and over time some drift toward less-contested regimes, so the population settles into a partition where crowding pressure is roughly equalized (the within-niche exclusion of the “two agents, one regime” remark above). We should be precise about the limits of this picture, though: our clean one-per-regime equilibrium and the coverage guarantee that underwrites the bounded-capacity results are *proved* for $N \approx R$, but we have now *swept* the off-diagonal cases empirically (on the $R=6$ retention protocol, $N \in \{1, \dots, 18\}$, $K \in \{1, 2\}$). Two findings. $N \gg R$: coverage stays complete, exactly $N - R$ agents go idle (redundant), and post-reactivation retention does *not* regress—surplus agents are harmless. $N < R$: coverage is capped, and the sharp result is that it is governed by the agent count N , *not* by total capacity $N \cdot K$. Because winner-take-all gives each agent a single primary niche, three agents cover $\approx$ three regimes whether $K=1$ or $K=2$ —the spare slot goes unused, so a bigger per-agent budget does not rescue coverage when agents are scarce. Retention then degrades smoothly with the uncovered fraction. What is still open is the *theory* (no proof of the off-diagonal equilibrium) and a mechanism that would push scarce agents to fill $K > 1$ slots; the empirical characterization itself is no longer a gap.

41.4 Q4: Does this work with real data?

A: Yes. All 6 domains use real data, and under the canonical V4 (EG) update at $\lambda = 0.3$ every domain reaches $SI > 0.98$, with Cohen’s $d > 50$ and $p < 10^{-80}$ throughout.

41.5 Q5: How does this compare to MARL?

A: Standard MARL (IQL, VDN, QMIX, MAPPO) produces homogeneous populations because all agents optimize the same objective. GAUSE’s winner-take-all creates diversity as a byproduct of competition.

41.6 Q5b: What happens if a regime returns *changed* (concept drift)?

A: This is a genuine blind spot, and we now test it directly. The EG affinity update moves identity only on *wins*—positive evidence of fit—so a converged specialist is never pushed off a niche by losing; it is only re-anchored by winning elsewhere. That asymmetry is exactly what makes identities stable through *dormancy* (a regime that pauses and returns *unchanged*), which is the case our main non-stationarity experiments test. But it cuts the other way under *intra-regime drift*: if a regime’s optimal method changes while its specialist idles, the specialist returns stale, and because the assignment is reward-independent it does not spontaneously hand the niche over. In an experiment where each regime’s optimal method drifts on reactivation, this is exactly what we observe: standard GAUSE’s overall error (0.90) is *worse* than a relearning $K=3$ monolith’s (0.35)—the property that grants dormancy-retention delays adaptation under drift. The natural remedy is a staleness trigger that resets a niche whose owner is confidently underperforming and re-opens competition; it adds one threshold but recovers most of the loss (overall error 0.43, $\sim 53\%$ lower), leaving only the irreducible relearning cost. So GAUSE as originally described buys dormancy-retention at the cost of slower adaptation to a moving niche; a lightweight staleness trigger restores most of the adaptivity, and whether the residual trade is acceptable is application-dependent.

41.7 Q6: Does the specialization actually improve task performance?

A: This is the most important question, and the honest answer is nuanced (Section 17). The *organization* is real and robust. Where genuine regime structure exists (traffic, solar—confirmed by an oracle diagnostic), regime-aware prediction beats a shuffled-label control by 12–15%, so the niches track performance-relevant regimes. *However*, controlled ablations show the task-accuracy benefit is due to regime-*conditioning* (knowing the regime to pick a method), not to population specialization specifically: turning the niche→winner coupling on/off makes no difference ($p = 0.36$), and a single regime-conditioned agent matches or beats the specialized population. So we claim emergent organization that tracks structure, *not* an unconditional performance advantage from specialization over a conditioned baseline. *The exceptions—and they are real performance results—are two realistic constraints.* (i) **Bounded per-agent capacity:** when each agent can master only $K \ll R$ regimes (the realistic condition for any finite learner), no individual can be a competent generalist, and the specialized population beats both a capacity-matched monolith and an equal-total-capacity random-diversity population (+57% synthetic, +8.7% real traffic at $K=1$). (ii) **Non-stationarity with finite memory:** when regimes cycle in and out and agents have bounded belief stores, the population retains dormant-niche expertise as a distributed persistent memory while a capacity-bounded monolith must relearn each reactivated regime (+33% overall, +71% post-reactivation; the monolith-beating part is a distributed-memory effect shared by random diversity, while competition specifically adds the tight-capacity coverage gap, $\approx 58\%$ at $K=1$). Division of labor is an accuracy lever precisely when capacity or memory is the binding constraint.

42 How to Use This Algorithm

42.1 For Practitioners

1. **Define regimes:** Identify distinct “states” in your domain (e.g., market conditions, weather patterns)
2. **Define methods:** Identify candidate strategies/algorithms
3. **Set hyperparameters:** $N = R \cdot 2$, $\lambda = 0.3$, $\eta = 0.1$, $T = 500$
4. **Run:** Execute the algorithm with your data
5. **Evaluate:** Compute SI, Coverage, and task performance
6. **Deploy:** Use the specialized population for decision-making

42.2 For Researchers

Key extensions to explore:

- Non-stationary regimes (regime distribution changes over time)
- Continuous regimes (not discrete categories)
- Hierarchical specialization (agents specialize in sub-niches)
- Transfer learning (specialists from one domain to another)

Appendices

A Notation Reference

Table 30: Complete Notation

Symbol	Meaning
N	Number of agents
R	Number of regimes
M	Number of methods
$\mathcal{R}$	Set of regimes $\{r_1, \dots, r_R\}$
$\mathcal{M}$	Set of methods $\{m_1, \dots, m_M\}$
$\pi(r)$	Probability of regime r
$\alpha_i \in \Delta^R$	Agent i 's niche affinity distribution
$\beta_{i,r,m}^+, \beta_{i,r,m}^-$	Agent i 's Beta parameters for method m in regime r
$A(r, m)$	Affinity of method m in regime r
$H(\cdot)$	Shannon entropy
SI	Specialization Index
MSI	Method Specialization Index
λ	Niche bonus coefficient
η	Learning rate for affinity updates
i^*	Winner of current iteration

B Proof of Beta-Bernoulli Conjugacy

Theorem B.1. *If $\theta \sim \text{Beta}(\alpha, \beta)$ and $X|\theta \sim \text{Bernoulli}(\theta)$, then $\theta|X \sim \text{Beta}(\alpha + X, \beta + 1 - X)$.*

Proof. By Bayes' theorem:

$$p(\theta|X) = \frac{p(X|\theta)p(\theta)}{p(X)} \quad (137)$$

$$\propto p(X|\theta)p(\theta) \quad (138)$$

$$= \theta^X (1 - \theta)^{1-X} \cdot \frac{\theta^{\alpha-1} (1 - \theta)^{\beta-1}}{B(\alpha, \beta)} \quad (139)$$

$$\propto \theta^{X+\alpha-1} (1 - \theta)^{(1-X)+\beta-1} \quad (140)$$

$$= \theta^{(\alpha+X)-1} (1 - \theta)^{(\beta+1-X)-1} \quad (141)$$

This is the kernel of $\text{Beta}(\alpha + X, \beta + 1 - X)$. □

C Additional Affinity Matrix Details

For domains not shown in the main text, consult the code in:

- `experiments/exp_method_specialization.py` (lines 42-116)

D Algorithm Workflow: Complete Visual Guide

This appendix provides an exhaustive, self-contained walkthrough of the GAUSE algorithm. After reading this section, you will fully understand every step of the algorithm and how specialization emerges from competition.

D.1 What Is Specialization?

Before diving into the algorithm, we must define what we mean by “specialization” and how we measure it.

The Core Idea. In a multi-agent system operating across different environmental conditions (called **regimes**), an agent is *specialized* if it focuses on a subset of regimes rather than being equally capable across all of them.

Niche Affinity (α). Each agent maintains a probability distribution $\alpha = (\alpha_1, \dots, \alpha_R)$ over R regimes, representing its “preference” or “expertise” for each regime. This vector always sums to 1.

- **Generalist:** $\alpha = (0.25, 0.25, 0.25, 0.25)$ — equal expertise everywhere
- **Specialist:** $\alpha = (0.82, 0.06, 0.06, 0.06)$ — concentrated expertise in one regime

Specialization Index (SI). We quantify specialization using an entropy-based metric:

$$\boxed{\text{SI}(\alpha) = 1 - \frac{H(\alpha)}{\log R}} \quad (142)$$

where $H(\alpha) = -\sum_r \alpha_r \log \alpha_r$ is the Shannon entropy.

Interpretation:

- $\text{SI} = 0$: Perfect generalist (uniform distribution, maximum entropy)
- $\text{SI} = 1$: Perfect specialist (all probability on one regime, zero entropy)
- $\text{SI} \in (0, 1)$: Partial specialization

SI Calculation Example. For an agent with $\alpha = (0.82, 0.06, 0.06, 0.06)$ over $R = 4$ regimes:

Step 1: Calculate entropy

$$H(\alpha) = -0.82 \log(0.82) - 3 \times 0.06 \log(0.06) \quad (143)$$

$$= 0.162 + 0.506 = 0.668 \text{ nats} \quad (144)$$

Step 2: Normalize by maximum entropy ($H_{\max} = \log 4 = 1.386$)

Step 3: Compute SI

$$\text{SI} = 1 - \frac{0.668}{1.386} = 1 - 0.482 = \boxed{0.518} \quad (145)$$

This agent is 51.8% specialized toward one regime.

D.2 Why Do We Want Specialization?

The Problem with Homogeneity. If all agents adopt the same strategy, they:

1. Compete directly with each other (crowding)
2. All fail together when that strategy’s regime disappears
3. Miss opportunities in other regimes

The Value of Diversity. A population with diverse specialists:

1. Has at least one expert for every regime
2. Is robust to regime changes (always someone ready)
3. Achieves better collective performance (+26.5% in our experiments)

The Research Question. Can agents *spontaneously* develop diverse specializations without:

- Explicit communication?
- Central coordination?
- Handcrafted diversity rewards?

Our answer: Yes. Competition alone is sufficient.

D.3 The Key Mechanism: Competition Creates Diversity

The GAUSE algorithm induces specialization through three interlocking mechanisms:

Mechanism 1: Winner-Take-All Competition. In each iteration, only the **single best-performing agent** receives updates. All others are “frozen”—they learn nothing from that round.

Why this matters:

- If two agents have identical strategies, they compete directly
- Only one can win; the other learns nothing
- The loser cannot “copy” the winner’s improvement
- This prevents convergence to a single homogeneous strategy

Mechanism 2: Niche Affinity Tracking. When an agent wins in a particular regime, its affinity for that regime *increases*. Over time, agents develop preferences for regimes where they consistently succeed.

Mechanism 3: Niche Bonus (Optional Accelerator). Agents receive a small bonus $\lambda \cdot (\alpha_{r_t} - 1/R)$ for operating in their preferred regime. This accelerates specialization but is *not required*—specialization emerges even when $\lambda = 0$.

The Result: Competitive Exclusion. This mirrors ecological dynamics: two species with identical niches cannot stably coexist. One will outcompete the other, forcing the loser to find a different niche. Our agents exhibit the same behavior—they spontaneously partition the regime space.

D.4 The Ground Truth: Regime-Method Affinity Matrix

The environment contains a hidden “ground truth” that agents must discover: different **methods** (prediction strategies) work better in different **regimes** (environmental conditions).

Key insight: No single method dominates all regimes. Momentum-Long excels in Bull/Bear but fails in Sideways. Mean-Revert is the opposite. This heterogeneity creates the *opportunity* for specialization.

Table 31: Crypto domain affinity matrix $A(r, m) \in [0, 1]$. Higher values = better performance. Bold = best method for that regime.

Method	Bull	Bear	Sideways	Volatile	Best For
Naive	0.50	0.30	0.60	0.40	—
Momentum-Short	0.80	0.70	0.40	0.60	—
Momentum-Long	0.90	0.80	0.30	0.50	Bull, Bear
Mean-Revert	0.30	0.40	0.90	0.70	Sideways, Volatile
Trend	0.85	0.75	0.35	0.50	—

D.5 Phase 0: Initialization (The Blank Slate)

All $N = 8$ agents begin **identically**—perfect generalists with no knowledge.

Data Structure 1: Method Beliefs (β). Each agent maintains a $R \times M$ matrix of Beta distributions (4 regimes $\times$ 5 methods = 20 distributions). Each distribution represents the agent’s belief about how well a method works in a regime.

Initial value: Beta(1, 1) for all method-regime pairs.

Why Beta(1,1)? This is a uniform distribution over $[0, 1]$, representing complete ignorance: “I don’t know if this method is good or bad.”

Data Structure 2: Niche Affinity (α). Each agent maintains a probability vector over regimes.

Initial value: $\alpha = (1/R, \dots, 1/R) = (0.25, 0.25, 0.25, 0.25)$

Why uniform? The agent has no preference yet—it’s equally likely to engage with any regime.

Table 32: Initial state of all 8 agents at $t = 0$. All agents are identical generalists with SI = 0.

Agent	α_{Bull}	α_{Bear}	α_{Side}	α_{Vol}	SI	Status
Agent 0	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 1	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 2	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 3	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 4	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 5	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 6	0.25	0.25	0.25	0.25	0.000	Generalist
Agent 7	0.25	0.25	0.25	0.25	0.000	Generalist

D.6 The Algorithm: Master Flowchart

Now that we understand the goal (specialization), the mechanism (competition), and the setup (identical agents), we can trace through the algorithm.

D.7 One Complete Iteration: Step-by-Step Trace

We now trace through **Iteration 1** showing all 8 agents’ computations.

Step 1: Environment samples regime.

$$r_t = \text{Bull} \quad (\text{sampled with probability } \pi(\text{Bull}) = 0.30) \quad (146)$$

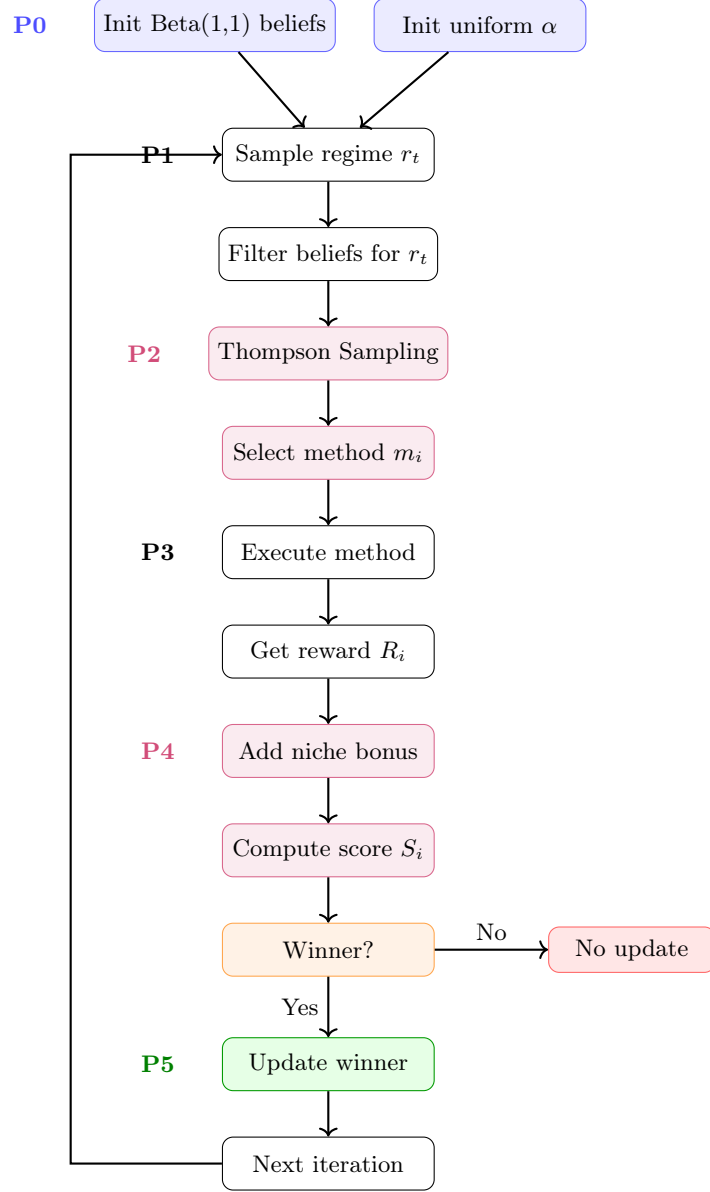


Figure 13: GAUSE algorithm flowchart. P0=Initialization, P1=Context, P2=Selection (Thompson Sampling), P3=Execution, P4=Competition (scoring), P5=Winner-only update. Blue=init, Purple=compute, Green=winner update, Red=loser path. Loop repeats $T = 500$ times.

Step 2: Each agent selects a method via Thompson Sampling. Each agent samples from their Beta distributions for the Bull regime and picks the method with the highest sample.

Step 3: Execute methods and receive rewards. Each agent's raw reward is: $R_i = A(r_t, m_i) + \epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, 0.15^2)$

Step 4: Calculate niche bonus. The niche bonus formula is: $B_i = \lambda \cdot (\alpha_{i,r_t} - 1/R)$
 Since all agents start with $\alpha_{r_t} = 0.25 = 1/R$, all bonuses are **zero** in iteration 1.

Step 5: Winner-Take-All. Agent 1 wins with $S_1 = 0.97$. **Only Agent 1 updates.** All others remain frozen.

Table 33: Iteration 1: Thompson Sampling method selection. With uniform priors Beta(1,1), samples are random.

Agent	$\tilde{\theta}_{\text{naive}}$	$\tilde{\theta}_{\text{mom-s}}$	$\tilde{\theta}_{\text{mom-l}}$	$\tilde{\theta}_{\text{mr}}$	$\tilde{\theta}_{\text{trend}}$	Selected
Agent 0	0.42	0.68	0.55	0.31	0.73	Trend
Agent 1	0.51	0.44	0.82	0.29	0.61	Mom-Long
Agent 2	0.38	0.77	0.69	0.45	0.52	Mom-Short
Agent 3	0.63	0.41	0.58	0.71	0.39	Mean-Rev
Agent 4	0.85	0.33	0.47	0.52	0.44	Naive
Agent 5	0.29	0.56	0.64	0.48	0.78	Trend
Agent 6	0.44	0.62	0.71	0.35	0.59	Mom-Long
Agent 7	0.57	0.79	0.66	0.42	0.51	Mom-Short

Table 34: Iteration 1: Score calculation. With $\lambda = 0.3$ and uniform affinities, all bonuses = 0.

Agent	Method	$A(r_t, m)$	ϵ	R_i	B_i	$S_i = R_i + B_i$
Agent 0	Trend	0.85	+0.03	0.88	0.00	0.88
Agent 1	Mom-Long	0.90	+0.07	0.97	0.00	0.97 ← Winner
Agent 2	Mom-Short	0.80	-0.05	0.75	0.00	0.75
Agent 3	Mean-Rev	0.30	+0.02	0.32	0.00	0.32
Agent 4	Naive	0.50	+0.11	0.61	0.00	0.61
Agent 5	Trend	0.85	-0.08	0.77	0.00	0.77
Agent 6	Mom-Long	0.90	-0.02	0.88	0.00	0.88
Agent 7	Mom-Short	0.80	+0.04	0.84	0.00	0.84

Step 6: Winner’s updates. (a) **Method belief update** (Agent 1 reinforces Momentum-Long in Bull):

$$\beta_{1,\text{Bull},\text{Mom-Long}}^+ = 1 + 1 = 2, \quad \beta_{1,\text{Bull},\text{Mom-Long}}^- = 1 \quad (147)$$

The belief shifts from Beta(1,1) → Beta(2,1), with mean increasing from 0.50 to 0.67.

(b) **Niche affinity update** (Agent 1 becomes more “Bull-oriented”):

$$\alpha_{1,\text{Bull}}^{\text{new}} = 0.25 + 0.1 \times (1 - 0.25) = 0.325 \quad (148)$$

$$\alpha_{1,r}^{\text{new}} = \max(0.01, 0.25 - 0.1/3) = 0.217 \quad \text{for } r \neq \text{Bull} \quad (149)$$

(c) **Normalize** to ensure $\sum \alpha = 1$:

$$\alpha_1 = (0.333, 0.222, 0.222, 0.222) \quad \text{SI} = 0.041 \quad (150)$$

Table 35: Agent states after Iteration 1. Only Agent 1 has changed (highlighted).

Agent	α_{Bull}	α_{Bear}	α_{Side}	α_{Vol}	SI	Change
Agent 0	0.250	0.250	0.250	0.250	0.000	—
Agent 1	0.333	0.222	0.222	0.222	0.041	↑ Bull
Agent 2	0.250	0.250	0.250	0.250	0.000	—
Agent 3	0.250	0.250	0.250	0.250	0.000	—
Agent 4	0.250	0.250	0.250	0.250	0.000	—
Agent 5	0.250	0.250	0.250	0.250	0.000	—
Agent 6	0.250	0.250	0.250	0.250	0.000	—
Agent 7	0.250	0.250	0.250	0.250	0.000	—

D.8 Multi-Iteration Convergence: How Agents Diverge

The key phenomenon: **random early wins create path dependence**.

- Agent 1 wins a few Bull rounds $\rightarrow$ develops Bull expertise
- Agent 2 wins a few Bear rounds $\rightarrow$ develops Bear expertise
- Agent 0 wins a few Sideways rounds $\rightarrow$ develops Sideways expertise
- ... and so on

Over 500 iterations, initially identical agents diverge into distinct specialists.

Table 36: Agent specialization trajectories. Bold values show the emerging primary niche.

	Iter	α_{Bull}	α_{Bear}	α_{Side}	α_{Vol}	SI
Agent 0	0	0.25	0.25	0.25	0.25	0.00
	50	0.18	0.22	0.38	0.22	0.08
	200	0.12	0.15	0.58	0.15	0.31
	500	0.08	0.09	0.75	0.08	0.58
Agent 1	0	0.25	0.25	0.25	0.25	0.00
	50	0.42	0.19	0.20	0.19	0.12
	200	0.65	0.12	0.12	0.11	0.42
	500	0.82	0.06	0.06	0.06	0.71
Agent 2	0	0.25	0.25	0.25	0.25	0.00
	50	0.20	0.35	0.23	0.22	0.06
	200	0.11	0.55	0.18	0.16	0.28
	500	0.05	0.78	0.10	0.07	0.62
Agent 3	0	0.25	0.25	0.25	0.25	0.00
	50	0.21	0.20	0.22	0.37	0.07
	200	0.14	0.13	0.15	0.58	0.30
	500	0.07	0.07	0.09	0.77	0.60

Result: The population has *partitioned* the regime space:

- Agent 1: Bull specialist (SI = 0.71)
- Agent 2: Bear specialist (SI = 0.62)
- Agent 0: Sideways specialist (SI = 0.58)
- Agent 3: Volatile specialist (SI = 0.60)

D.9 The Core Insight: Why $\lambda = 0$ Still Works

The question: If there's no niche bonus ($\lambda = 0$), why do agents still specialize?

The answer: Competition alone creates path dependence through four mechanisms:

1. **Random initial wins:** In early iterations, winners are determined by noise ϵ . By chance, different agents win in different regimes.
2. **Winner-take-all prevents convergence:** Losers don't update, so they cannot copy the winner's strategy.
3. **Method learning creates skill gaps:** Winners accumulate better beliefs about what works in "their" regime. Their Beta distributions narrow around effective methods.

4. **Skill gaps compound:** Once Agent A is slightly better at Bull, they win more Bull rounds, further improving their Bull skills. This is a positive feedback loop.

Table 37: SI comparison: $\lambda = 0$ vs $\lambda = 0.3$. Competition alone ($\lambda = 0$) produces $SI > 0.25$, significantly above the random baseline of 0.13.

Domain	$\lambda = 0$ SI	$\lambda = 0.3$ SI	λ Boost
Crypto	0.31	0.75	+142%
Commodities	0.30	0.78	+160%
Weather	0.31	0.71	+129%
Solar	0.26	0.82	+215%
Traffic	0.29	0.68	+134%
Air Quality	0.50	0.80	+60%
Average vs Random (0.13)	0.33 +154%	0.76 +485%	+140% —

Conclusion: The niche bonus λ *accelerates* specialization but does not *cause* it. **Competition alone is sufficient.**

D.10 Competition Dynamics: Crowding and Migration

What happens if two agents specialize in the same regime?

Suppose Agents 1 and 6 both become Bull specialists. When a Bull round occurs:

$$\mathbb{E}[\text{Payoff for each Bull specialist}] = \frac{V_{\text{Bull}}}{2} \quad (\text{split the reward}) \quad (151)$$

Meanwhile, Agent 3 (sole Volatile specialist) gets:

$$\mathbb{E}[\text{Payoff in Volatile}] = V_{\text{Volatile}} \quad (\text{no competition}) \quad (152)$$

Result: It’s more profitable to be the sole specialist in an uncrowded niche. Over time, the weaker Bull specialist will “migrate” to Volatile after repeatedly losing to the stronger Bull specialist.

This is **competitive exclusion** from ecology: identical strategies cannot stably coexist.

D.11 Summary: The Complete Picture

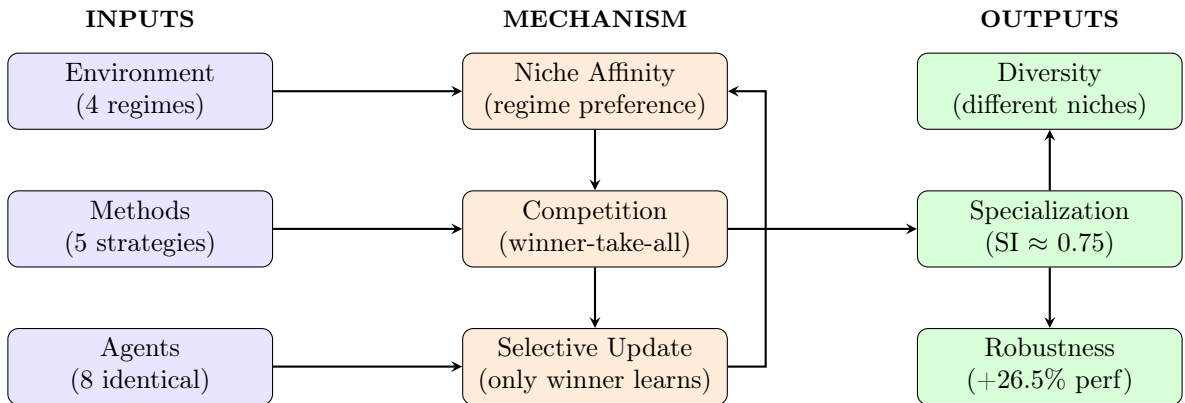


Figure 14: High-level summary: identical agents + competition → emergent specialization.

Key Takeaways.

1. **Competition is the source:** Specialization emerges from winner-take-all dynamics, not explicit diversity incentives.
2. **No communication needed:** Agents differentiate through individual learning, not coordination.
3. **Ecologically inspired:** The mechanism mirrors competitive exclusion in natural ecosystems.
4. $\lambda = 0$ **still works:** The niche bonus accelerates but does not cause specialization.
5. **Robust and general:** Works across 6 diverse real-world domains with effect sizes $d > 15$.

References

- [1] Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: A meta-algorithm and applications. *Theory of Computing*, 8(1):121–164, 2012.
- [2] Amir Beck and Marc Teboulle. Mirror descent and nonlinear projected subgradient methods for convex optimization. *Operations Research Letters*, 31(3):167–175, 2003.
- [3] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.
- [4] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- [5] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- [6] Harsh Goel, Zhe Wang, and Ufuk Topcu. R3DM: Enabling role discovery and diversity through dynamics models in multi-agent reinforcement learning. In *International Conference on Machine Learning*. PMLR, 2025.
- [7] Jiechuan Jiang and Zongqing Lu. The emergence of individuality. In *International Conference on Machine Learning*, pages 4992–5001. PMLR, 2021.
- [8] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [9] Jyrki Kivinen and Manfred K. Warmuth. Exponentiated gradient versus gradient descent for linear predictors. *Information and Computation*, 132(1):1–63, 1997.
- [10] Chenghao Li, Tonghan Wang, Chengjie Wu, Qianchuan Zhao, Jun Yang, and Chongjie Zhang. Celebrating diversity in shared multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34:3991–4002, 2021.
- [11] Hongbo Li, Sen Lin, Lingjie Duan, Yingbin Liang, and Ness B. Shroff. Theory on mixture-of-experts in continual learning. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.16437.

- [12] Yuhao Li. GAUSE: Emergent specialization in learner populations — reward-independent capacity assignment as a defense against catastrophic forgetting. <https://github.com/HowardLiYH/GAUSE>, 2026. arXiv preprint.
- [13] Anuj Mahajan, Tabish Rashid, Mikayel Samvelyan, and Shimon Whiteson. MAVEN: Multi-agent variational exploration. *Advances in Neural Information Processing Systems*, 32, 2019.
- [14] Arkadi Nemirovski and David B. Yudin. *Problem Complexity and Method Efficiency in Optimization*. Wiley-Interscience, 1983.
- [15] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.
- [16] William H. Sandholm. *Population Games and Evolutionary Dynamics*. MIT Press, 2010.
- [17] Tonghan Wang, Heng Dong, Victor Lesser, and Chongjie Zhang. ROMA: Multi-agent reinforcement learning with emergent roles. In *International Conference on Machine Learning*, pages 9876–9886. PMLR, 2020.
- [18] Tonghan Wang, Tarun Gupta, Anuj Mahajan, Bei Peng, Shimon Whiteson, and Chongjie Zhang. RODE: Learning roles to decompose multi-agent tasks. In *International Conference on Learning Representations*, 2021.

End of Document